Tcl/Tk unter Linux

X11, XDG, Locale, Xfce, Tests unter Xvfb

Gregor Ebbing

4. November 2026

Inhaltsverzeichnis

1 Was dieses Buch ist 7
1.1 Einführung 7
1.2 Womit die Beispiele arbeiten 7
1.3 Wie das Buch zu lesen ist 8
2 tcl_platform und Distribution 8
2.1 Einführung 8
2.2 Die Schlüssel 8
2.2.1 threaded unter Tcl 9 9
2.2.2 osVersion ist der Kernel 9
2.2.3 user ist nicht USER 9
2.3 Distribution: /etc/os-release 10
2.4 Der laufende Prozess: /proc/self 11
2.5 Zusammenfassung 11
3 Welches Tcl? 11
3.1 Einführung 11
3.2 Die drei Herkünfte 11
3.3 Wer antwortet? 12
3.4 Tk zu Tcl 9.1 13
3.5 Ein Quellbau, der Debians Bibliothek lädt 13
3.6 tclsh und wish im PATH 14
3.7 Die Shebang-Zeile 14
3.8 Woran ein Tcl 9 erkennbar ist 14
3.9 Zusammenfassung 15
4 Pfade, Umgebung, XDG 15
4.1 Einführung 15
4.2 HOME und die Tilde 15
4.3 Umgebungsvariablen 16
4.4 XDG Base Directory 17
4.5 Benutzerordner: Dokumente und Desktop 17
4.6 Dateisystem 18
4.7 Zusammenfassung 18
5 Suchpfade 18
5.1 Einführung 18
5.2 auto_path des Distributions-Tcl 18
5.3 auto_path der Debian-Pakete für Tcl 9 19
5.4 auto_path eines Quellbaus 19
5.5 sqlite3 unter Debians Tcl 9 20
5.5.1 Mit 3.53.4-2 oder älter 20
5.5.2 Eigener Bau und Paket 21
5.6 TCLLIBPATH 21
5.7 tm-Pfade 22
5.8 Welche Kopie gewinnt 22
5.9 Welches Paket wurde geladen? 23
5.10 Zusammenfassung 23
6 Kodierung und Locale 24

6.1 Einführung 24
6.2 encoding system folgt der Locale 24
6.3 Dateien: immer mit Kodierung 25
6.4 Skripte: die Wache unter Tcl 8.6 25
6.5 Terminal und exec 26
6.6 msgcat und Sprache 26
6.7 Zusammenfassung 26
7 exec, xdg-open, Prozesse 26
7.1 Einführung 26
7.2 Programm finden: auto_execok 27
7.3 Fehler von exec auseinanderhalten 27
7.4 Im Hintergrund und über Kanäle 28
7.4.1 Die Falle mit der eigenen Auswahl 28
7.5 xdg-open: Datei oder Adresse öffnen 28
7.6 Prozessnummer und Signale 29
7.7 Zusammenfassung 29
8 X11, Fenstermanager und Wayland 30
8.1 Einführung 30
8.2 DISPLAY und der Start ohne Anzeige 30
8.3 Fensterklasse und WM_CLASS 30
8.4 Was der Fenstermanager beisteuert 31
8.5 Einen Fenstermanager erkennen 32
8.6 Wayland 33
8.6.1 XWayland unter Weston 33
8.7 Zusammenfassung 34
9 Schriften, Skalierung, Themes 34
9.1 Einführung 34
9.2 Schriften: Xft und fontconfig 34
9.3 tk scaling und DPI 35
9.3.1 Woher Tk 9 die Skalierung nimmt 36
9.4 Themes 37
9.5 X-Ressourcen 37
9.6 Zusammenfassung 37
10 Maus, Tastatur, Zwischenablage 38
10.1 Einführung 38
10.2 Das Mausrad 38
10.3 Modifier und Tasten 39
10.4 Zwei Zwischenablagen 40
10.4.1 Virtuelle Ereignisse 41
10.5 Zusammenfassung 41
11 Native Dialoge und Desktop-Integration 41
11.1 Einführung 41
11.2 Die Standarddialoge sind Tk-Dialoge 42
11.3 Die .desktop-Datei 43
11.4 Fenstersymbol 43
11.5 Benachrichtigungen 44
11.6 Symbol in der Leiste 44

11.7 Drucken 45
11.8 Zusammenfassung 45
12 Xfce aus Tcl 45
12.1 Einführung 45
12.2 Xfce erkennen 45
12.3 xfconf: Einstellungen lesen 46
12.4 xfconf: Einstellungen schreiben 46
12.5 Bevorzugte Programme: exo-open 47
12.6 Benachrichtigungen 47
12.7 Fenstermanager, Panel, Sitzung 47
12.8 Zusammenfassung 47
13 GUI-Tests mit Xvfb 48
13.1 Einführung 48
13.2 Xvfb und xvfb-run 48
13.2.1 Eigener Server statt xvfb-run 48
13.3 Bildschirm, DPI und Schriften 49
13.4 Rückgabewert und Abbruch 49
13.5 tclsh oder wish 50
13.6 Zusammenfassung 50
14 Was der Test selbst herstellt 50
14.1 Einführung 50
14.2 Übersicht 50
14.3 Mappen: erst update 51
14.4 Tasten erzeugen: Fokus herstellen 51
14.5 Grab 52
14.6 Lage von Dialogen 52
14.6.1 Der Rahmen kommt spät 52
14.6.2 openbox setzt mittige Dialoge selbst 53
14.7 Tests, die einen Fenstermanager brauchen 54
14.8 Zusammenfassung 54
15 Tk bauen: Abhängigkeiten 54
15.1 Einführung 54
15.2 Was Debian zum Bau von Tk braucht 55
15.3 TK_LIBS und TCL_LIBS 55
15.4 Was wirklich gelinkt ist 55
15.5 tclConfig.sh unter Debian 56
15.6 Threads und shared 56
15.7 Zwei Fallen beim Bau 56
15.8 Zusammenfassung 57
16 Weitergeben 57
16.1 Einführung 57
16.2 Welches Tcl hat der Zielrechner? 57
16.3 Ein Image mit zipfs mkimg 58
16.4 Zipkit: statische Basis 58
16.5 Desktop: Starter und Symbol 59
16.6 Zusammenfassung 59
17 FAQ 59

17.1 Plattform und Interpreter 60
17.2 Kodierung 60
17.3 Prozesse 60
17.4 Oberfläche 60
17.5 Tests 61
17.6 Bauen und Weitergeben 61
18 Fallen 62
18.1 Plattform und Interpreter 62
18.2 Pfade und Suchpfade 62
18.3 Kodierung 63
18.4 Prozesse 63
18.5 X11 und Fenstermanager 63
18.6 Schrift, Skalierung, Themes 63
18.7 Maus, Tastatur, Zwischenablage 64
18.8 Desktop 64
18.9 Tests 64
18.10 Bauen und Weitergeben 64
19 CLI 64
19.1 Zur Laufzeit 65
19.2 Shell 65
19.3 Prüfskript 65
19.4 Weitere Beispielskripte 66
19.5 Howto-Serien 67
20 Glossar 67
20.1 Zeichen 67
20.2 A 68
20.3 C 68
20.4 D 68
20.5 E 68
20.6 F 68
20.7 L 69
20.8 M 69
20.9 O 69
20.10 P 69
20.11 R 69
20.12 S 69
20.13 T 70
20.14 W 70
20.15 X 70
20.16 Z 71
21 Fehlermeldungen 71
21.1 no display name and no $DISPLAY environment variable 71
21.2 couldn't connect to display 71
21.3 couldn't execute … no such file or directory 71
21.4 child process exited abnormally 71
21.5 child killed: software termination signal 71
21.6 (Text auf stderr) 72

21.7 invalid or incomplete multibyte or wide character 72
21.8 unknown or ambiguous subcommand "home" 72
21.9 unknown or ambiguous subcommand "systray" 72
21.10 bad button number "8" 72
21.11 bad event type or keysym "Super" 72
21.12 CLIPBOARD selection doesn't exist or form "STRING" not defined 72
21.13 Error: target STRING not available 72
21.14 Unable to obtain the list of printers 72
21.15 Failed to execute child process "dbus-launch" 73
21.16 Verbindung ist gescheitert 73
21.17 SKIP: serie-xfce braucht XFCE 73
21.18 WARNING: unrecognized options 73
21.19 file format not recognized 73
21.20 XInput2.h fehlt 73
21.21 this extension is compiled for Tcl 8.6 73
21.22 Keine Berechtigung beim Installieren nach /usr/bin 73
22 Quellen 74
22.1 Einführung 74
22.2 Eigene Läufe 74
22.2.1 Howto-Serien des Autors 74
22.2.2 Container-Prüfung 74
22.2.3 Prüfung auf lxpro 75
22.2.4 Was im Container fehlte 75
22.3 Tcl und Tk 75
22.4 Debian 76
22.5 Spezifikationen 76
22.6 Nachbarbücher derselben Reihe 76
22.7 Widersprüche zwischen den Quellen 77
22.8 Marken 77
23 Testumgebung 77
23.1 Einführung 77
23.2 Rechner und Versionen 77
23.3 Prüfskripte 78
23.4 Noch nicht getestet 78
24 Lizenz 79

1 Was dieses Buch ist

1.1 Einführung

Tcl/Tk läuft unter Linux zu Hause: Die Distributionen bringen es als Paket mit, `wish` zeichnet mit dem Fenstersystem X11, und die meisten Tcl-Beispiele im Netz setzen stillschweigend Linux voraus. Trotzdem verhält sich ein Tcl-Programm auf einem Linux-Arbeitsplatz an vielen Stellen anders als unter macOS oder Windows: mehrere Interpreter liegen nebeneinander, Pfade folgen XDG, die Kodierung kommt aus der Locale, ein Fenstermanager entscheidet über Fokus und Größe, und es gibt zwei Zwischenablagen.

Dieses Buch behandelt diese Besonderheiten aus Sicht der Tcl/Tk-Entwicklung: den Plattformwert, welches Tcl gerade läuft, Pfade und Suchpfade der Distribution, Kodierung, `exec`, X11 und Fenstermanager, Schriften und Skalierung, Mausrad und Zwischenablage, die Einbindung in den Desktop, Xfce, GUI-Tests unter Xvfb, die Bauabhängigkeiten von Tk und die Weitergabe.

Es ist ein Werkstattbuch. Die Kapitel zeigen Vorgehen und Befehle, die unter Linux ausprobiert sind, mit der Ausgabe, die dabei herauskommt, und mit den Stellen, an denen etwas nicht wie erwartet lief. Allgemeine Tcl/Tk-Kenntnisse setzt es voraus. Ein Linux-Handbuch ist es nicht.

1.2 Womit die Beispiele arbeiten

Die Beispiele verwenden Tcl 8.6 aus dem Paket der Distribution und Tcl/Tk 9.0.4, selbst gebaut unter `/opt/tcl9`, auf Debian und Ubuntu mit X11. Wo sich 8.6 und 9.0 unterscheiden, steht es im Text, ebenso, wenn ein Wert nur mit einem Fenstermanager oder nur unter Xfce gilt.

Grundlage sind fünf Howto-Serien des Autors. `serie-linux`, `serie-xvfb` und `serie-xfce` sind veröffentlicht, im Repository `tcltk-manuals` unter Serien/; `serie-tcltk-bauen` und `serie-tcl9-app` nicht. Angaben wie "(`serie-linux`, Howto 08)" nennen das Skript, mit dem ein Wert geprüft wurde. Jede Serie hat ein `README.md` mit den geprüften Werten, ein `TUTORIAL.md`, eine Kurzreferenz `howto/00-referenz.md` und je Howto eine Erklärung `NN-*.md` mit einer lauffähigen Demo `NN-*.tcl`. Die Demos prüfen ihre Aussagen selbst und melden OK, SKIP oder FAIL.

Serie	Inhalt	Kapitel hier
serie-linux	Pfade, XDG, Umgebung, exec, Kodierung, Dateisystem, tcl_platform, Dialoge	2, 4, 6, 7, 11
serie-xvfb	Xvfb, Fenstermanager, Fokus, Iconify, Grab, Tests ohne Dekorator (20 Howtos)	8, 13, 14
serie-xfce	Xfce 4.20 aus Tcl: xfconf, DBus, Thunar, exo-open, notify-send	11, 12
serie-tcltk-bauen	Bauabhängigkeiten von Tk unter Debian, TK_LIBS, ldd	2, 3, 5, 15
serie-tcl9-app	Programme mit Tcl 9 und zipfs	16

Das Buch übernimmt die Howtos nicht Stück für Stück. Es ordnet sie nach Themen, erklärt den Zusammenhang und behält die Codezeilen, auf die es ankommt. Laufen die Serien gesammelt,

ist `SERIES_AUTO=1` gesetzt; dann öffnet keine Demo einen Dialog, einen Browser oder ein Benachrichtigungsfenster.

Wer eine Aussage selbst prüfen will, nimmt die Skripte in `beispiele/` neben dem Buchordner, allen voran messprobe-linux.tcl, das alle Werte der Tabellen dieses Buches druckt.

Rechner, genaue Versionen und was noch nicht getestet ist, nennt Anhang F: Testumgebung. Werte aus dem Ubuntu-Container heißen im Text "Container-Prüfung"; dort läuft Xvfb ohne Fenstermanager, ohne Desktop-Sitzung und ohne Xfce.

1.3 Wie das Buch zu lesen ist

Teil 1 (Kapitel 1 bis 7) erklärt die Plattform: `tcl_platform`, welches Tcl, Pfade und XDG, Suchpfade, Kodierung und Locale, `exec`. Teil 2 (Kapitel 8 bis 12) geht an die Oberfläche: X11 und Fenstermanager, Schriften und Skalierung, Maus, Tastatur und Zwischenablage, Dialoge und Desktop, Xfce. Teil 3 (Kapitel 13 bis 17) behandelt Tests unter Xvfb, die Bauabhängigkeiten von Tk, die Weitergabe und häufige Fragen.

Die Anhänge sind zum Nachschlagen: Fallen (A), Befehle (B), Glossar (C), Fehlermeldungen (D), Quellen (E) und die Testumgebung (F). Anhang A lohnt sich beim Lesen nebenbei offen zu halten.

Andere Themen stehen in eigenen Büchern derselben Reihe: Debian als Betriebssystem in *Debian unter Linux* (dieses Buch beschreibt den Interpreter auf der Distribution, jenes die Distribution), die Schwesterbücher *Tcl/Tk unter macOS* und *Tcl/Tk unter Windows*, Tcl/Tk selbst bauen in *Tcl/Tk 9.0 bauen* und *Tcl/Tk 9.1 bauen*, Kits, zipfs, `.deb` und `.rpm` in *Tcl-Anwendungen ausliefern* (im Text kurz *Ausliefern*), C-Erweiterungen in *Tcl-Erweiterungen*, Pakete und Module in *Pakete und Module*.

Nächstes Kapitel: tcl_platform und Distribution

2 tcl_platform und Distribution

2.1 Einführung

tcl_platform sagt, welches Betriebssystem Tcl sieht. Unter Linux und unter macOS lautet `platform` gleich: `unix`. Die Trennung macht `os`. Welche Distribution läuft, sagt `tcl_platform` nicht. Dafür gibt es /etc/os-release.

2.2 Die Schlüssel

```
set ::tcl_platform(platform)   ;# unix
set ::tcl_platform(os)         ;# Linux
set ::tcl_platform(osVersion)  ;# Kernel, nicht Distribution
set ::tcl_platform(machine)    ;# x86_64
set ::tcl_platform(user)       ;# Benutzer des Prozesses
```

Werte aus `serie-linux` (Debian forky, Tcl 8.6.18, Howto 10) und aus der Container-Prüfung (– = dort nicht erhoben):

Schlüssel	Debian forky, 8.6.18	Container, 8.6.14	Container, 9.0.4
platform	unix	unix	unix
os	Linux	Linux	Linux
osVersion	7.2.14+deb14-amd64	6.18.44-fc-v139	6.18.44-fc-v139
machine	x86_64	x86_64	x86_64
wordSize	8	8	8
pointerSize	–	8	8
byteOrder	–	littleEndian	littleEndian
threaded	1 (8.6)	1	Schlüssel fehlt

Linux-Code prüft also:

```
if {$::tcl_platform(os) eq "Linux"} { ... }
```

`platform eq "unix"` trifft auch macOS (`os` ist dort `Darwin`) und die BSDs. Für "X11 oder nicht" ist `tcl_platform` ohnehin der falsche Zeuge, das sagt `tk windowingsystem`, siehe X11, Fenstermanager und Wayland.

2.2.1 threaded unter Tcl 9

Tcl 9 ist immer mit Threads gebaut. Der Schlüssel threaded fehlt deshalb. Code, der ihn liest, braucht `info exists`:

```
set threads [expr {![info exists ::tcl_platform(threaded)]
                   || $::tcl_platform(threaded)}]
```

`serie-tcltk-bauen` (Howto 05) bestätigt das für 9.0.4 auf Debian: `TCL_THREADS=1` in der `tclConfig.sh` von 8.6, unter 9.0 kein Schlüssel, `--enable-threads` ist dort ein unbekannter Schalter.

2.2.2 osVersion ist der Kernel

`osVersion` ist die Ausgabe von `uname -r` und wechselt mit jedem Kernel-Update. Auf dem Debian-Arbeitsplatz war es bei der letzten Prüfung `7.2.14+deb14-amd64` (`serie-linux`, Howto 10, nennt noch `7.1.9+deb14-amd64`), im Container `6.18.44-fc-v139`. Der zweite Wert ist der Kernel des Wirts, nicht der von Ubuntu: Ein Container bringt keinen eigenen Kernel mit. Eine Versionsprüfung auf "Debian 13" oder "Ubuntu 24.04" über `osVersion` geht daher immer schief.

2.2.3 user ist nicht USER

`tcl_platform(user)` ist der Name zur effektiven Benutzerkennung des Prozesses. `$env(USER)` ist geerbt und kann fehlen oder etwas anderes sagen, etwa nach `sudo` oder in einer Sandbox. Im Container mit `env -i` gestartet: `tcl_platform(user)` ist `root`, `env(USER)` gibt es nicht. `serie-linux` (Howto 03) schlägt deshalb im Container mit "env USER gesetzt" fehl, auf dem Arbeitsplatz nicht.

```
set wer $::tcl_platform(user)             ;# immer da
set wer2 [expr {[info exists ::env(USER)] ? $::env(USER) : $wer}]
```

2.3 Distribution: /etc/os-release

Die Distribution steht in `/etc/os-release`. Die Datei besteht aus Zeilen `SCHLÜSSEL=Wert`, Werte oft in Anführungszeichen. Sie ist Shell-Syntax, aber für die üblichen Schlüssel reicht ein einfacher Leser:

```
proc osRelease {} {
    set d [dict create]
    foreach datei {/etc/os-release /usr/lib/os-release} {
        if {![file readable $datei]} continue
        set ch [open $datei]
        fconfigure $ch -encoding utf-8
        foreach zeile [split [read $ch] \n] {
            if {[regexp {^([A-Z_]+)=(.*)$} $zeile -> k v]} {
                dict set d $k [string trim $v {"'}]
            }
        }
        close $ch
        break
    }
    return $d
}
set r [osRelease]
puts "[dict get $r ID] [dict getdef $r VERSION_CODENAME -]"
```

`dict getdef` gibt es ab Tcl 8.7; unter 8.6 `dict exists` vorher prüfen. Ergebnis (– = nicht erhoben):

Schlüssel	Debian-Arbeitsplatz	Container
ID	debian	ubuntu
ID_LIKE	–	debian
VERSION_ID	fehlt	24.04
VERSION_CODENAME	forky	noble
PRETTY_NAME	Debian GNU/Linux forky/sid	Ubuntu 24.04.5 LTS

Die Debian-Spalte stammt aus `serie-linux` (Howto 06: `ID=debian`, `VERSION_CODENAME=forky`). Ubuntu trägt `ID_LIKE=debian`, wie das Debian-Buch im Kapitel *Debian erkennen* schreibt. Debian Testing setzt `VERSION_ID` nicht (Arbeitsplatz, forky/sid). Code verlässt sich deshalb nicht darauf und nimmt `VERSION_CODENAME`. Wer "Debian oder abgeleitet" prüft, liest `ID` **und** `ID_LIKE`.

```
set familie [concat [dict getdef $r ID ""] [dict getdef $r ID_LIKE ""]]
if {"debian" in $familie} { ... }
```

Im Container liefert `/etc/debian_version` übrigens `trixie/sid`, obwohl das System Ubuntu ist. Ubuntu übernimmt die Datei aus Debian. Sie taugt nicht, um Debian von Ubuntu zu trennen.

2.4 Der laufende Prozess: /proc/self

Was Windows aus der Registry und macOS aus `sysctl` liest, liegt unter Linux im Dateisystem /proc. `serie-linux` (Howto 06):

```
file readlink /proc/self/exe      ;# z. B. /usr/bin/tclsh8.6
set ch [open /proc/self/comm]
set name [string trim [read $ch]]
close $ch
```

`/proc/self/exe` ist der aufgelöste Pfad des Interpreters, auch wenn er über einen Symlink gestartet wurde. `info nameofexecutable` liefert dagegen den Pfad, unter dem der Interpreter aufgerufen wurde. Für die Frage "welches Tcl läuft hier" ist das nächste Kapitel zuständig.

2.5 Zusammenfassung

- `platform` ist `unix` auf Linux und macOS. `os` trennt: `Linux`.
- `osVersion` ist der Kernel, im Container der des Wirts.
- `threaded` fehlt unter Tcl 9. Vor dem Lesen `info exists`.
- `tcl_platform(user)` ist verlässlich, `$env(USER)` nicht.
- Distribution aus `/etc/os-release`, Familie über `ID` und `ID_LIKE`. `/etc/debian_version` steht auch auf Ubuntu.

Nächstes Kapitel: Welches Tcl?

3 Welches Tcl?

3.1 Einführung

Auf einem Linux-Arbeitsplatz liegen oft mehrere Tcl nebeneinander: das der Distribution unter `/usr` und ein selbst gebautes unter `/opt`. Beide heißen `tclsh` mit Versionsnummer. Welches antwortet, entscheidet der PATH oder der volle Pfad. Dieses Kapitel zeigt, woran man das laufende Tcl erkennt und wie sich die beiden Sorten unterscheiden.

3.2 Die drei Herkünfte

Herkunft	Ort	Aufruf	geprüft mit
Distributionspaket Tcl 8.6	/usr/bin	tclsh8.6, wish8.6, oft tclsh	Debian forky 8.6.18, Ubuntu 24.04 8.6.14
Distributionspaket Tcl 9.0	/usr/bin	tclsh9.0, wish9.0	Debian forky 9.0.4 (serie-tcltk-bauen)
Distributionspaket Tcl 9.1	/usr/bin	tclsh9.1, wish9.1	Debian forky und unstable 9.1.0
Quellbau	/opt/tcl9	/opt/tcl9/bin/tclsh9.0, wish9.0	Container 9.0.4

Welche Pakete die Debian-Zweige führen, darunter tcl9.0 (Quellpakete, Stand 2026-11-02,

qa.debian.org/madison.php):

Paket	Stable (trixie)	Testing (forky)	unstable (sid)
tcl8.6	8.6.16	8.6.18	8.6.18
tcl9.0	9.0.1	9.0.4	9.0.4
tcl9.1	–	9.1.0	9.1.0

Zu jedem `tcl…` gibt es das passende `tk…` in derselben Fassung. Der Betreuer der Tcl-Pakete hat im November 2026 angekündigt, Tcl/Tk 9.0 für forky zu streichen; dann bliebe 9.1 neben 8.6. Standard bleibt 8.6: Pakete mit C-Teil baut Debian nur gegen 8.6, solange die Pakete `tcltk-defaults` nichts anderes sagen. Das Debian-Buch, Kapitel *Tcl unter Linux*, nennt die Lage auf den Rechnern des Autors: auf dem Debian-Arbeitsplatz die Pakete für 8.6.18, 9.0.4 und 9.1.0 unter `/usr` und ein selbst gebautes 9.1.0 unter `/opt/tcl91`, auf einem zweiten Rechner ein selbst gebautes 9.0.4 unter `/opt/tcl9`. Ubuntu 24.04 im Container hat nur 8.6.14 als Paket.

Die Regeln aus dem Debian-Buch gelten hier unverändert: Ein eigenes Tcl nie nach `/usr` installieren, sondern nach `/opt`. Das Bauen selbst beschreiben *Tcl/Tk 9.0 bauen* und *Tcl/Tk 9.1 bauen*, Kapitel *Mehrere Installationen*.

3.3 Wer antwortet?

In der Shell:

```
type -a tclsh tclsh8.6 tclsh9.0 wish9.0
ls -l /usr/bin/tclsh /usr/bin/wish
dpkg -S /usr/bin/tclsh8.6          # nur Debian und Ubuntu
```

Im Container ist `/usr/bin/tclsh` ein Symlink auf `tclsh8.6`, `/usr/bin/wish` einer auf `wish8.6`. `update-alternatives --display tclsh` meldet dort "no alternatives for tclsh": Der Link gehört dem Paket `tcl`, nicht dem Alternativen-System.

Im Interpreter:

```
puts [info nameofexecutable]    ;# Pfad des Aufrufs
puts [info patchlevel]
puts $tcl_library
puts [tcl::pkgconfig get libdir,runtime]
```

Container-Prüfung:

Wert	tclsh8.6 (Ubuntu)	tclsh9.0 (/opt/tcl9)
info patchlevel	8.6.14	9.0.4
$tcl_library	/usr/share/tcltk/tcl8.6	//zipfs:/lib/tcl/tcl_library
libdir,runtime	/usr/lib/x86_64-linux-gnu	/opt/tcl9/lib
scriptdir,runtime	/usr/share/tcltk/tcl8.6	/opt/tcl9/lib/tcl9.0
zipfs mount	Befehl fehlt	//zipfs:/lib/tcl an libtcl9.0.so

Zwei Dinge fallen auf. Debian und Ubuntu legen die Tcl-Bibliothek nach `/usr/share/tcltk`,

nicht nach `/usr/lib/tcl8.6` wie ein Quellbau von 8.6. Und ein Quellbau von Tcl 9 hängt die Skriptbibliothek als ZIP an `libtcl9.0.so`. `$tcl_library` zeigt dann ins Zipfs. Den Ordner `/opt/tcl9/lib/tcl9.0` aus `scriptdir,runtime` gibt es im Container nicht; er ist ein Rest der Bauvorgabe.

Die Debian-Pakete für Tcl 9 legen die Bibliothek ohne Zipfs ab. Auf dem Debian-Arbeitsplatz meldet `tclsh9.0` den Ordner `/usr/lib/tcl9.0`, `tclsh9.1` den Ordner `/usr/lib/tcl9.1`.

3.4 Tk zu Tcl 9.1

Debian trennt Tcl und Tk in eigene Pakete. Wer nur `tcl9.1` installiert, hat kein `wish9.1`, und `package require Tk` in `tclsh9.1` meldet trotzdem eine Fassung. Auf dem Debian-Arbeitsplatz, ohne das Paket `tk9.1`:

```
% package require Tk
9.0.4
```

`tclsh9.1` findet über `auto_path` den Index von Debians Tk 9.0 unter `/usr/lib/tcltk/x86_64-linux-gnu/tk9.0` und lädt ihn. Das geht, weil Tk 9.0 sich über die Stub-Tabelle an Tcl bindet; die Release-Notes von Tk 9.1.0 nennen ausdrücklich Tcl 9.0 oder 9.1 als Grundlage. Es ist aber nicht Tk 9.1: Die Neuerungen von 9.1 fehlen. Abhilfe:

```
sudo apt install tk9.1
```

Danach liefert `package require Tk` in `tclsh9.1` die Fassung 9.1.0, und `wish9.1` ist da.

3.5 Ein Quellbau, der Debians Bibliothek lädt

Ein Tcl unter `/opt` ist nur dann getrennt, wenn es auch seine eigene `libtcl` lädt. Die Datei heißt bei Debian und beim Quellbau gleich, `libtcl9.0.so`. Findet der Lader zuerst Debians Datei, läuft das Programm aus `/opt` mit Debians Bibliothek. Auf dem Debian-Arbeitsplatz mit einem BAWT-Build 9.0.1 unter `/opt/Tcl`:

Aufruf	info patchlevel	$tcl_library
/opt/Tcl/bin/tclsh9.0	9.0.4	/usr/lib/tcl9.0
LD_LIBRARY_PATH=/opt/Tcl/lib /opt/Tcl/bin/tclsh9.0	9.0.1	//zipfs:/lib/tcl/tcl_library

Ohne `LD_LIBRARY_PATH` meldet das BAWT-Programm Debians Fassung und Debians Skriptbibliothek. Es sieht dann auch Debians Pakete zuerst. Prüfen lässt es sich mit `ldd`:

```
ldd /opt/Tcl/bin/tclsh9.0 | grep libtcl
```

Zeigt die Zeile nach `/usr/lib/x86_64-linux-gnu`, braucht das Programm eine kleine Startdatei, etwa `~/bin/tclsh9b`:

```
#!/bin/sh
LD_LIBRARY_PATH=/opt/Tcl/lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH} exec
    /opt/Tcl/bin/tclsh9.0 "$@"
```

Ein Alias reicht nicht: Er gilt nur in der interaktiven Shell, nicht in Skripten und Makefiles. Der

Quellbau im Container nach `/opt/tcl9` braucht keine Startdatei: `readelf -d /opt/tcl9/bin/tclsh9.0` zeigt `RUNPATH /opt/tcl9/lib`, das Programm findet seine eigene Bibliothek vor der des Systems. Beim BAWT-Build auf dem Debian-Arbeitsplatz greift kein solcher Eintrag; was dort steht, zeigt `readelf -d` auf dem Rechner selbst.

3.6 tclsh und wish im PATH

Ein Quellbau nach `/opt/tcl9` legt `tclsh9.0` und `wish9.0` in `/opt/tcl9/bin`. Im Container gab es dort zusätzlich einen Link `tclsh -> tclsh9.0`, aber keinen Link `wish`. `make install` legt diesen Link nicht an: Das Ziel `install-binaries` in `unix/Makefile.in` von 9.0.4 installiert nur `tclsh9.0`. Der Link ist also von Hand entstanden, so wie *Tcl/Tk 9.0 bauen* im Kapitel *Mehrere Installationen* es vorschlägt. Folge: Mit `/opt/tcl9/bin` vorn im PATH startet `tclsh` das Tcl 9, `wish` aber weiter das `wish8.6` aus `/usr/bin`.

Deshalb immer mit Versionsnummer aufrufen. Für 9.1 neben 9.0 rät das Debian-Buch, nicht beide Prefixe in den PATH zu legen, sondern den vollen Pfad zu nehmen.

3.7 Die Shebang-Zeile

Ein ausführbares Skript nennt seinen Interpreter in der ersten Zeile. Unter Linux gibt es drei übliche Formen:

```
#!/usr/bin/tclsh8.6
#!/usr/bin/env tclsh9.0
#!/opt/tcl9/bin/wish9.0
```

`/usr/bin/env` sucht im PATH. Das macht das Skript beweglich, aber abhängig von der Sitzung: Ein Programm aus dem Startmenü sieht einen anderen PATH als ein Terminal (siehe Native Dialoge und Desktop-Integration). Der volle Pfad ist fest, aber an einen Rechner gebunden.

Der Linux-Kernel reicht alles nach dem Interpreterpfad als **ein** Argument weiter. `#!/usr/bin/env tclsh9.0 -encoding utf-8` sucht also ein Programm namens `tclsh9.0 -encoding utf-8`. Für Optionen gibt es `env -S` aus GNU coreutils; auf dem Zielrechner vorher prüfen.

Die alte Form für Systeme, auf denen der Pfad zu `wish` unbekannt ist, nutzt, dass Tcl einen Backslash am Kommentarende als Fortsetzung liest, die Shell aber nicht:

```
#!/bin/sh
# Die nächste Zeile liest nur die Shell, für Tcl ist sie Kommentar \
exec wish9.0 "$0" "$@"
```

`serie-tcl9-app` (Howto 02) nimmt für Linux `#!/usr/bin/env tclsh9.0`.

3.8 Woran ein Tcl 9 erkennbar ist

```
if {[package vsatisfies [info patchlevel] 9-]} { ... }
```

Ohne Versionsvergleich verraten Tcl 9 drei Dinge (im Container geprüft):

- `$tcl_library` beginnt mit `//zipfs:/` (beim Quellbau).

- `file volumes` liefert `//zipfs:/ /`, unter 8.6 nur `/`.
- `file attributes /tmp -permissions` liefert `0o41777`, unter 8.6 `041777`.

Das dritte trifft Code, der den Rechtewert als Zeichenkette vergleicht. Zahlen mit `0o` liest `expr` unter beiden Fassungen. Code, der Rechte vergleicht, rechnet deshalb mit dem Zahlenwert:

```
set r [file attributes $datei -permissions]
expr {$r & 0o777}        ;# unter 8.6 und 9 gleich
```

3.9 Zusammenfassung

- Debian Stable: 8.6.16 und Paket `tcl9.0` 9.0.1. Testing und unstable: 8.6.18, `tcl9.0` 9.0.4 und `tcl9.1` 9.1.0. Angekündigt: 9.0 fällt für forky weg, 9.1 bleibt. Eigenes Tcl nach `/opt`, nie nach `/usr`.
- Debian legt `$tcl_library` nach `/usr/share/tcltk/tcl8.6` und `/usr/lib/tcl9.x`. Ein Quellbau von Tcl 9 legt sie ins Zipfs an `libtcl9.0.so`.
- Ohne das Paket `tk9.1` lädt `tclsh9.1` Debians Tk 9.0.
- Ein Programm unter `/opt` kann Debians `libtcl` laden. `ldd` prüft das, eine Startdatei mit LD_LIBRARY_PATH behebt es.
- Immer mit Versionsnummer aufrufen. `wish` ohne Nummer ist meist 8.6.
- Shebang mit `env` folgt dem PATH der Sitzung.
- `file volumes` und das Format der Rechte unterscheiden sich zwischen 8.6 und 9.

Nächstes Kapitel: Pfade, Umgebung, XDG

4 Pfade, Umgebung, XDG

4.1 Einführung

Unter Linux hat jeder Prozess ein HOME, Pfade trennt der Schrägstrich, es gibt nur ein Wurzelverzeichnis. Wo ein Programm seine Einstellungen ablegt, regelt die XDG Base Directory Specification: nicht als Punktdatei direkt in `$HOME`, sondern in `~/.config/<Programm>/`. Dieses Kapitel zeigt die Variablen, die Vorgaben, wenn sie fehlen, und den Bruch bei der Tilde zwischen Tcl 8.6 und 9.

4.2 HOME und die Tilde

`$env(HOME)` ist unter Linux gesetzt, anders als oft unter Windows (`serie-linux`, Howto 02 und 03). Pfade baut man mit `file join`:

```
set konfig [file join $env(HOME) .config meineapp]
```

Bei der Tilde trennen sich 8.6 und 9. Tcl 8.6 ersetzt ein führendes ~ in jedem Pfad durch HOME. Tcl 9 tut das nicht mehr (TIP 602): ~ ist dort ein gewöhnlicher Dateiname. Dafür gibt es zwei neue Befehle. Container-Prüfung, Arbeitsordner `/tmp`, HOME=`/root`:

Ausdruck	Tcl 8.6.14	Tcl 9.0.4 und 9.1.0
file normalize ~ (gestartet in /tmp)	/root	/tmp/~
file exists ~	1	0

Ausdruck	Tcl 8.6.14	Tcl 9.0.4 und 9.1.0
file home	Fehler, unbekannt	/root
file tildeexpand ~/x	Fehler, unbekannt	/root/x

Auf dem Debian-Arbeitsplatz mit den Debian-Paketen, im Heimatordner gestartet, dasselbe Bild: `tclsh` 8.6.18 liefert für `file normalize ~` den Heimatordner und für `file exists ~` 1, `tclsh9.0` 9.0.4 und `tclsh9.1` 9.1.0 liefern `<Heimatordner>/~` und 0.

`file normalize ~` taugt also nur unter 8.6 als Weg zum Heimatordner. Unter Tcl 9 liefert derselbe Ausdruck einen Pfad im aktuellen Ordner. `serie-linux` zeigt in Howto 02 beide Fassungen; die Demo `02-pfade.tcl` prüft unter 8.6 "Tilde = HOME" und unter 9 `file home` und `file tildeexpand`.

Code für beide Fassungen nimmt `$env(HOME)` und keine Tilde:

```
proc heim {} {
    if {[info exists ::env(HOME)] && $::env(HOME) ne ""} {
        return $::env(HOME)
    }
    return [file home]          ;# nur Tcl 9
}
```

Eine Tilde, die von außen kommt, etwa aus einer Konfigurationsdatei, löst unter Tcl 9 erst `file tildeexpand` auf. Unter 8.6 erledigt das jeder Dateibefehl von selbst.

4.3 Umgebungsvariablen

`serie-linux` (Howto 03) liest die üblichen Variablen:

```
$env(HOME) $env(USER) $env(SHELL) $env(PATH)
$env(LANG) $env(DISPLAY)
```

Was davon gesetzt ist, hängt an der Sitzung. In einer grafischen Sitzung stehen `DISPLAY`, `XDG_CURRENT_DESKTOP`, `XDG_SESSION_TYPE` und `XDG_RUNTIME_DIR` bereit. In einem Cron-Lauf, unter `sudo` oder mit `env -i` fehlen sie. `HOSTNAME` und `TMPDIR` fehlen oft auch in der Sitzung. Für den Rechnernamen gibt es `info hostname`. Das temporäre Verzeichnis ist dann `/tmp`.

Robust ist ein Zugriff mit Vorgabe:

```
proc envOder {name vorgabe} {
    if {[info exists ::env($name)] && $::env($name) ne ""} {
        return $::env($name)
    }
    return $vorgabe
}
set tmp [envOder TMPDIR /tmp]
```

Tcl 9 hat dafür auch `file tempdir`. Es legt ein neues, leeres Verzeichnis an und liefert seinen Pfad, im Container `/tmp/tcl_KZSgAw`. Das Verzeichnis räumt das Programm selbst wieder weg.

4.4 XDG Base Directory

Die Spezifikation legt vier Orte für Benutzerdaten fest. Jede Variable darf fehlen, dann gilt die Vorgabe. `serie-linux` (Howto 04):

Variable	Vorgabe	Inhalt
XDG_CONFIG_HOME	~/.config	Einstellungen
XDG_DATA_HOME	~/.local/share	Daten, die bleiben sollen
XDG_CACHE_HOME	~/.cache	Wegwerfbares
XDG_STATE_HOME	~/.local/state	Verlauf, letzte Fenstergröße
XDG_RUNTIME_DIR	keine	Sockets, Sperrdateien; /run/user/UID

Auf dem Arbeitsplatz des Autors ist `XDG_DATA_HOME` oft nicht gesetzt, `XDG_RUNTIME_DIR` ist `/run/user/UID`. Im Container mit `env -i` ist keine dieser Variablen gesetzt. `XDG_STATE_HOME` prüfen die Serien nicht; die Vorgabe stammt aus der Spezifikation.

Ein Programm legt seine Dateien in einen eigenen Unterordner:

```
proc xdgOrdner {var vorgabe app} {
    set basis ""
    if {[info exists ::env($var)]} { set basis $::env($var) }
    if {$basis eq "" || [file pathtype $basis] ne "absolute"} {
        set basis [file join [heim] {*}$vorgabe]
    }
    return [file join $basis $app]
}
set konfig [xdgOrdner XDG_CONFIG_HOME .config meineapp]
file mkdir $konfig
```

Ein relativer Wert ist laut Spezifikation ungültig und wird übergangen. `beispiele/xdg-ordner.tcl` enthält die vier Varianten und lief im Container unter 8.6.14 und 9.0.4 mit demselben Ergebnis, auch mit `XDG_CONFIG_HOME=relativ`: `/root/.config/meineapp`.

Die Windows-Entsprechung ist `APPDATA`, die macOS-Entsprechung `~/Library/Application Support`. Ein Programm für alle drei fragt zuerst `tcl_platform(os)` und dann die jeweilige Quelle.

4.5 Benutzerordner: Dokumente und Desktop

Die Namen der sichtbaren Ordner im Heimatverzeichnis sind **lokalisiert**. Auf einem deutschen Desktop heißt der Ordner `Dokumente`, nicht `Documents`, der Desktop `Schreibtisch`. Die Zuordnung steht in ~/.config/user-dirs.dirs; das Werkzeug `xdg-user-dir` liest sie (`serie-linux`, Howto 05):

```
exec {*}[auto_execok xdg-user-dir] DOCUMENTS
exec {*}[auto_execok xdg-user-dir] DESKTOP
exec {*}[auto_execok xdg-user-dir] DOWNLOAD
```

`xdg-user-dir` gehört zum Debian-Paket `xdg-user-dirs`. Im Container fehlt es; Howto 05 überspringt dort. `beispiele/xdg-ordner.tcl` fällt in diesem Fall auf `HOME` zurück. Einen

Ordner wie `~/Documents` fest einzutragen ist der häufigste Fehler dieses Abschnitts.

4.6 Dateisystem

Unix-Attribute, keine Laufwerksbuchstaben (`serie-linux`, Howto 09):

```
file volumes                          ;# /
file attributes $f                    ;# -group -owner -permissions
file attributes $f -permissions 00644
```

Unter Tcl 9 liefert `file volumes` zusätzlich `//zipfs:/` (Container-Prüfung: `//zipfs:/ /`). Code, der `[lindex [file volumes] 0]` für die Wurzel hält, bekommt unter 9 das Zipfs. `/tmp` hat das Sticky-Bit, Rechte `1777`. Das Format des Rechtewerts unterscheidet sich zwischen 8.6 und 9, siehe Welches Tcl?.

`file nativename` liefert unter Linux denselben Pfad zurück. Groß- und Kleinschreibung zählt: `Daten.txt` und `daten.txt` sind zwei Dateien. Das unterscheidet Linux von macOS (APFS in der Grundeinstellung) und von Windows.

4.7 Zusammenfassung

- HOME ist gesetzt. Pfade mit `file join`.
- Tilde: unter 8.6 überall aufgelöst, unter 9 nicht mehr. `file normalize ~` liefert unter 9 einen Pfad im aktuellen Ordner. Für beide Fassungen `$env(HOME)`.
- XDG: Variable oder Vorgabe, eigener Unterordner, relative Werte übergehen.
- Benutzerordner lokalisiert, über `xdg-user-dir`.
- `file volumes` enthält unter 9 auch `//zipfs:/`.

Nächstes Kapitel: Suchpfade

5 Suchpfade

5.1 Einführung

`package require` sucht in auto_path, Module (`.tm`) im tm-Pfad. Beide Listen setzt jedes Tcl beim Start aus eingebauten Werten und Umgebungsvariablen zusammen. Debian und Ubuntu ändern die eingebauten Werte mit eigenen Patches. Ein selbst gebautes Tcl unter `/opt` kennt sie nicht. Wer beide nebeneinander hat, muss wissen, wer welche Pakete sieht.

Die Grundlagen von `package`, `pkgIndex.tcl` und `.tm` stehen im Buch *Pakete und Module*. Hier geht es nur um die Linux-Werte.

5.2 auto_path des Distributions-Tcl

Container-Prüfung, `tclsh8.6` aus Ubuntu 24.04, mit `env -i` ohne `TCLLIBPATH`:

```
/usr/share/tcltk/tcl8.6
/usr/share/tcltk
/usr/lib
/usr/local/lib/tcltk
/usr/local/share/tcltk
```

```
/usr/lib/tcltk/x86_64-linux-gnu
/usr/lib/tcltk
/usr/lib/tcltk/tcl8.6
```

Die Liste folgt der Debian-Tcl-Policy: architekturunabhängige Pakete unter /usr/share/tcltk, Pakete mit C-Teil unter `/usr/lib/tcltk/<multiarch>`, lokale Pakete unter `/usr/local/share/tcltk` und `/usr/local/lib/tcltk`. `tcl_pkgPath` nennt dieselben Paketordner in anderer Reihenfolge, aber nicht den Bibliotheksordner der Fassung. Der steht nur in `auto_path`, bei 8.6 vorn: `/usr/share/tcltk/tcl8.6`. Bei Debians `tclsh9.0` fehlt in `tcl_pkgPath` entsprechend `/usr/lib/tcl9.0`, bei `tclsh9.1` `/usr/lib/tcl9.1`. Debian unstable mit 8.6.18 liefert dieselbe Liste in derselben Reihenfolge.

Paketierte Bibliotheken liegen dort mit Versionsnummer im Ordnernamen. Im Container lädt `struct::list` aus `/usr/share/tcltk/tcllib1.21/struct/list.tcl`.

Eigene Pakete für das System-Tcl gehören nach `/usr/local/share/tcltk` (reines Tcl) oder `/usr/local/lib/tcltk` (mit `.so`). Dort findet das System-Tcl sie ohne weitere Einstellung, und `apt` fasst sie nicht an.

5.3 auto_path der Debian-Pakete für Tcl 9

Auf Debian unstable, `tclsh9.0` aus dem Paket `tcl9.0`, ohne die Einträge aus `TCLLIBPATH`:

```
/usr/lib/tcl9.0
/usr/lib
/usr/local/lib/tcltk
/usr/local/share/tcltk
/usr/lib/tcltk/x86_64-linux-gnu
/usr/lib/tcltk
/usr/share/tcltk
/usr/lib/tcltk/tcl9.0          (fehlt)
```

`tclsh9.1` meldet dieselbe Liste mit `9.1` statt `9.0`. Anders als ein Quellbau sieht Debians Tcl 9 also **alle** Paketordner der Distribution, auch die mit Bibliotheken nur für 8.6. Ob ein Paket unter Tcl 9 lädt, entscheidet dann allein sein `pkgIndex.tcl`. Pakete wie tdom, tcltls, Thread oder Img legen zwei Bibliotheken nebeneinander, `libtdom0.9.6.so` für 8.6 und `libtcl9tdom0.9.6.so` für 9, und ihr Index wählt nach der Fassung. Wo die zweite Datei fehlt, bricht `package require` unter Tcl 9 ab (Abschnitt *sqlite3 unter Debians Tcl 9*).

In der Liste der tm-Pfade stehen bei den Paketen für Tcl 9 zwei Ordner als ein einziger Eintrag:

```
/usr/lib/tcltk /usr/share/tcltk/tcl9/site-tcl
/usr/lib/tcltk /usr/share/tcltk/tcl9/9.0
```

Module unter `/usr/share/tcltk/tcl9` findet Tcl 9 so nicht. Auf dem geprüften Debian unstable gibt es diese Ordner nicht, es fehlt also derzeit nichts.

5.4 auto_path eines Quellbaus

Container-Prüfung, `/opt/tcl9/bin/tclsh9.0`, `env -i`:

```
//zipfs:/lib/tcl/tcl_library
//zipfs:/lib/tcl
/opt/tcl9/lib
```

Das ist alles. `/usr/share/tcltk` steht nicht darin. Ein Tcl 9 unter `/opt` sieht kein einziges Paket der Distribution, weder Tk-Erweiterungen noch Tcllib. Das ist gewollt: Ein Paket für 8.6 mit C-Teil lädt unter Tcl 9 ohnehin nicht. Reine Tcl-Pakete laufen oft unter beiden Fassungen, müssen aber ausdrücklich eingehängt werden.

Im Container brachte `/opt/tcl9/lib` eigene Fassungen mit: Tcllib 1.9 (`package versions struct::list`), Thread 3.0.6, sqlite3 3.53.0. Insgesamt kannte `tclsh9.0` nach einem `package require` auf ein fehlendes Paket 628 Paketnamen, `tclsh8.6` 584.

Was `package require` auf Debian forky/sid (Debian-Pakete, ohne `TCLLIBPATH`) liefert; `serie-tcltk-bauen`, Howto 10, nennt noch Thread 3.0.6 und tdbc 1.1.14:

Paket	Tcl 8.6.18	Tcl 9.0.4	Tcl 9.1.0
Thread	3.0.7	3.0.7	3.0.7
Itcl	4.3.8	4.3.8	4.3.8
sqlite3	3.53.4	fehlt	fehlt
tdbc	1.1.15	1.1.15	1.1.15

Thread kommt aus dem Paket `tcl-thread` 1:3.0.7-2, tdbc aus `/usr/lib/tcltk/x86_64-linux-gnu/tdbc1.1.15`. sqlite3 bricht unter 9.0 und 9.1 mit `this extension is compiled for Tcl 8.6` ab (Abschnitt *sqlite3 unter Debians Tcl 9*).

Ein Quellbau holt sqlite aus dem `pkgs/`-Ordner des Tcl-Baums; dem Debian-Paket `tcl9.0` fehlte es dort.

5.5 sqlite3 unter Debians Tcl 9

Das Paket libsqlite3-tcl enthält bis 3.53.4-2 nur eine Bibliothek, gebaut für Tcl 8.6. Unter `tclsh9.0` und `tclsh9.1` bricht `package require sqlite3` dann ab:

```
% package require sqlite3
this extension is compiled for Tcl 8.6
```

Seit 3.54.1-1 (unstable, November 2026) bringt das Paket eine zweite Bibliothek für Tcl 9 mit; dann ist nichts zu tun:

```
/usr/lib/tcltk/x86_64-linux-gnu/sqlite3/
    libsqlite3.54.1.so        für 8.6
    libtcl9sqlite3.54.1.so    für 9.0 und 9.1
```

Beide sind dynamisch gegen `libsqlite3.so.0` aus `libsqlite3-0` gebunden; die gegen 9.1 gebaute lädt auch unter 9.0.4.

5.5.1 Mit 3.53.4-2 oder älter

Ein zweites sqlite3 daneben hilft nur mit gleicher Nummer und vorn in `auto_path`; bei verschiedenen Nummern nimmt `package require` die höchste. Also dieselbe Fassung für beide Tcl bauen, aus den Quellen der Tcl-Distribution (für 9.1.0 `sqlite3.53.4.tar.gz`, ein TEA-Paket):

```
cd sqlite3.53.4
mkdir b86 b90
(cd b86 && ../configure --with-tcl=/usr/lib/tcl8.6 --libdir=$HOME/lib/tcltk \
        && make && make install-lib-binaries)
(cd b90 && ../configure --with-tcl=/usr/lib/tcl9.0 --libdir=$HOME/lib/tcltk \
        && make && make install-lib-binaries)
```

- `install-lib-binaries`, nicht `install-binaries`: Das zweite Ziel will `sqlite3_analyzer` nach `/usr/bin` legen.
- Gegen 9.0 bauen: Die Bibliothek lädt dann unter 9.0 und 9.1.
- Beide Bibliotheken gehören in denselben Ordner; der erzeugte Index wählt nach der Fassung.

`~/lib/tcltk` kommt über `TCLLIBPATH` nach vorn. Kommt später ein Paket mit höherer Nummer, gewinnt wieder das Paket.

5.5.2 Eigener Bau und Paket

Ein eigener Bau und Debians Paket haben nicht dieselben `PRAGMA compile_options`. Zwei davon ändern das Verhalten:

Option	eigener Bau 3.53.4	Debian 3.54.1-1
DQS=0	gesetzt: SELECT "abc" ist ein Fehler	fehlt: liefert abc
USE_URI	fehlt: file:… ohne -uri 1 wird ein Dateiname	gesetzt: URI

Das Programm legt das Verhalten selbst fest, in jedem Bau gleich:

```
sqlite3 db $datei -uri 1      ;# file:-Namen immer als URI
db config dqs_dml 0           ;# "Text" in SELECT/INSERT … ein Fehler
db config dqs_ddl 0           ;# ebenso in CREATE TABLE …
```

`tdbc::sqlite3` ist reines Tcl und braucht keinen eigenen Bau. Unter 8.6 lädt Debians Modul, unter Tcl 9 jedes `sqlite3-1.1.15.tm` in einem tm-Pfad der Fassung, etwa über `TCL9_0_TM_PATH`. Hat das Programm vorher ein Paket aus einem Index geladen, gewinnt bei gleicher Fassung der Index (Abschnitt *Welche Kopie gewinnt*).

5.6 TCLLIBPATH

TCLLIBPATH wird beim Start **vorn** an `auto_path` gestellt. Der Wert ist eine Tcl-Liste, getrennt durch Leerzeichen, nicht durch Doppelpunkte wie `PATH`. Container-Prüfung, beide Fassungen gleich:

TCLLIBPATH	Anfang von auto_path
/x/eins /x/zwei	/x/eins /x/zwei …
/x/eins:/x/zwei	/x/eins:/x/zwei …, ein einziger Eintrag

Der zweite Eintrag ist ein Pfad, den es nicht gibt. Tcl meldet nichts, das Paket bleibt unauffindbar.

TCLLIBPATH gilt für **jedes** Tcl, das die Sitzung startet. Wer darin Ordner mit 9er-Paketen einträgt, gibt sie auch dem `tclsh8.6`. Tcl sucht eine Ebene unter jedem Eintrag nach `pkgIndex.tcl`; eine `.so` für Tcl 9 bricht unter 8.6 beim `load` ab. Sauberer ist es, je Fassung einen eigenen Ordner zu führen und `auto_path` im Programm zu ergänzen:

```
set hier [file dirname [file normalize [info script]]]
set auto_path [linsert $auto_path 0 [file join $hier lib]]
```

5.7 tm-Pfade

Module suchen nicht in `auto_path`, sondern im tm-Pfad (`tcl::tm::path list`). Container-Prüfung, Tcl 9.0.4 aus `/opt/tcl9`:

```
/opt/tcl9/lib/tcl9/site-tcl
/opt/tcl9/lib/tcl9/9.0
//zipfs:/lib/tcl/tcl9/site-tcl
//zipfs:/lib/tcl/tcl9/9.0
```

Das Ubuntu-`tclsh8.6` meldet 25 Einträge: `site-tcl` und die Ordner `8.0` bis `8.6` jeweils unter `/usr/lib/tcltk/tcl8`, `/usr/lib/tcl8` und `/usr/share/tcltk/tcl8`, dazu `/usr/share/tcltk/tcl8.6/tcl8`.

Zusätzliche Orte kommen aus Umgebungsvariablen, deren Name die Fassung trägt: TCL9_0_TM_PATH für Tcl 9.0, `TCL8_6_TM_PATH` für 8.6, dazu `TCL8_5_TM_PATH` und so weiter für ältere Unterfassungen, die 8.6 ebenfalls liest. Tcl 9.1 liest `TCL9_1_TM_PATH` und ebenso TCL9_0_TM_PATH. Zu jedem Namen gibt es eine Schreibweise mit Punkt (TCL9.0_TM_PATH), die `tm.tcl` auch liest; in `bash` lässt sie sich nicht exportieren. Diese Variablen trennen mit **Doppelpunkt**, anders als `TCLLIBPATH`. Container-Prüfung:

- TCL9_0_TM_PATH=/x/tm90a:/x/tm90b: Tcl 9.0.4 setzt `/x/tm90b` und `/x/tm90a` an die Spitze, in dieser umgekehrten Reihenfolge.
- TCL9_TM_PATH und `TCL8_TM_PATH` (ohne Unterfassung) liest keine der beiden Fassungen.
- TCL8_6_TM_PATH sieht Tcl 9.0 nicht.
- Ubuntus 8.6.14 setzt die Einträge aus `TCL8_6_TM_PATH` nicht an die Spitze, sondern hinter die acht Ordner unter `/usr/lib/tcltk/tcl8` (geprüft mit `TCL8_5_TM_PATH` und TCL8_6_TM_PATH: Index 8 bis 10 von 28). Module gleichen Namens und gleicher Version in diesen Ordnern

gewinnen also; eine höhere Version gewinnt unabhängig vom Ordner.

Die umgekehrte Reihenfolge folgt daraus, dass `tcl::tm::path add` jeden Eintrag vorn einfügt. Wer eine bestimmte Reihenfolge braucht, setzt sie im Programm mit `tcl::tm::path add`.

5.8 Welche Kopie gewinnt

Liegt ein Paket als Modul im tm-Pfad und zugleich hinter einem `pkgIndex.tcl` in `auto_path`, hängt die geladene Kopie davon ab, was das Programm vorher verlangt hat:

- Ist noch kein Index gelesen, beendet ein passendes Modul die Suche.
- Die erste Suche nach einem Paket, das kein Modul ist, liest jeden `pkgIndex.tcl` in allen Ordnern von `auto_path`. Danach kennt Tcl alle Fassungen daraus. Passt eine davon, fragt ein späterer Aufruf nicht mehr nach Modulen.
- Bei gleicher Version gewinnt dann der Index. Lag das früher gesuchte Paket in einem anderen Namensraum-Ordner als das Modul, gewinnt der Index auch mit einer niedrigeren Version: Der Modul-Handler hat dabei nur in jenem Ordner gesucht, `struct/` für `struct::list`, die Wurzel für `tablelist`.

Geprüft mit 8.6.14, 9.0.4 und 9.1.0, mit gleichem Ergebnis. Die ganze Tabelle und ein Skript zum Nachmachen stehen in *Ausliefern*, Kapitel Suchpfade.

Unter Linux trifft das vor allem alte Kopien in Ordnern aus `TCLLIBPATH`. Ein typischer Fall: In `~/lib/tcltk` liegt noch eine frühere Installation mit `pkgIndex.tcl`, die neue liegt in derselben Version als Modul in `site-tcl`. Ein Programm, das zuerst `tablelist` lädt (hinter einem Index), bekommt danach die alte Kopie; ein Programm, das nur das Modul verlangt, die neue. Abhilfe ist, die alte Kopie wegzuräumen.

5.9 Welches Paket wurde geladen?

```
package require struct::list
puts [package ifneeded struct::list [package present struct::list]]
```

Die Zeile zeigt den Befehl, mit dem Tcl das Paket laden würde, also seinen Pfad. Das ist die schnellste Antwort auf "warum lädt mein Tcl 9 das alte Paket": Meist steht ein Ordner aus TCLLIBPATH davor.

Für Pakete mit C-Teil ist info loaded der Beweis. Es nennt jede Bibliothek, die der Interpreter tatsächlich geladen hat:

```
package require sqlite3
puts [info loaded]
```

Alles auf einmal zeigt der Bericht von `tclutils::tuappinfo`: die Suchvariablen (`TCLLIBPATH`, `TCL*_TM_PATH`, `LD_LIBRARY_PATH`) Eintrag für Eintrag, `auto_path`, `tcl_pkgPath` und den tm-Pfad mit einer Marke `(missing)` an jedem Ordner, den es nicht gibt, jedes geladene Paket mit seiner Datei und die Liste aus `info loaded`. Das Beispiel der Bibliothek lädt vorher die genannten Pakete:

```
tclsh9.0 examples/demo-tuappinfo.tcl sqlite3
```

5.10 Zusammenfassung

- Debian-Tcl: `auto_path` über `/usr/share/tcltk` und `/usr/lib/tcltk/<multiarch>`, eigene Pakete nach `/usr/local/…/tcltk`.
- Debians Tcl 9 sieht dieselben Paketordner wie 8.6. Ob ein Paket lädt, entscheidet sein Index.
- `libsqlite3-tcl` bis 3.53.4-2 ist nur für 8.6 gebaut, ab 3.54.1-1 auch für Tcl 9. Für ältere: dieselbe Fassung für 8.6 und 9.0 bauen und vorn in `auto_path` legen.
- Tcl 9 unter `/opt` sieht keine Distributionspakete.
- `TCLLIBPATH`: Tcl-Liste mit Leerzeichen, gilt für jedes Tcl der Sitzung.
- `TCL9_0_TM_PATH`, `TCL8_6_TM_PATH`: Doppelpunkt, je Fassung, umgekehrt eingefügt; 9.1 liest auch `TCL9_0_TM_PATH`.
- Ein passendes `.tm` beendet die Suche, solange kein Index gelesen ist. Danach gewinnt bei gleicher Version der Index, nach einem Paket aus einem anderen Namensraum-Ordner auch ein älterer.
- Herkunft eines Pakets: `package ifneeded` mit `package present`; geladene Bibliotheken: `info loaded`.

Nächstes Kapitel: Kodierung und Locale

6 Kodierung und Locale

6.1 Einführung

Unter Linux bestimmt die Locale die Kodierung, nicht das Betriebssystem. Tcl liest beim Start `LC_ALL`, `LC_CTYPE` und `LANG` und setzt daraus encoding system. Ist die Locale `C` oder fehlt sie, ist das nicht UTF-8. Daran hängen drei Dinge: wie `open` eine Datei liest, wie `puts` auf das Terminal schreibt, und unter Tcl 8.6 auch, wie `source` ein Skript liest.

Die Langfassung zum Thema, mit Windows und TIP 656, steht im Buch *Encoding*.

6.2 encoding system folgt der Locale

`serie-linux` (Howto 08) zeigt auf Debian forky mit Tcl 8.6.18:

```
LANG=C          encoding system = iso8859-1
LANG=C.UTF-8    encoding system = utf-8
```

Die Container-Prüfung bestätigt das und ergänzt weitere Werte. Jeder Lauf mit `env -i`, also ohne sonstige Variable. Ausgewertet: `encoding system`, die Länge eines UTF-8-Literals `"Grüße"` im Skript, und die Länge derselben fünf Zeichen, gelesen aus einer UTF-8-Datei mit `open` ohne `fconfigure`.

Umgebung	8.6.14 System	8.6.14 Literal	9.0.4 System	9.0.4 Literal	open, beide
keine Locale	iso8859-1	7	iso8859-1	5	7
LANG=C	iso8859-1	7	iso8859-1	5	7
LANG=POSIX	iso8859-1	7	iso8859-1	5	7
LANG=C.UTF-8	utf-8	5	utf-8	5	5
LANG=de_DE.UTF-8	utf-8	5	utf-8	5	5
LANG=en_US.ISO-8859-1	iso8859-1	7	iso8859-1	5	7
LANG=de_DE	iso8859-1	7	utf-8	5	7 bzw. 5
LANG=C.UTF-8 LC_ALL=C	iso8859-1	7	iso8859-1	5	7
LANG=C LC_CTYPE=C.UTF-8	utf-8	5	utf-8	5	5

Was daraus folgt:

- Die Reihenfolge ist `LC_ALL`, dann `LC_CTYPE`, dann `LANG`. Ein `LC_ALL=C` schlägt jedes `LANG`.
- Die Locale muss nicht installiert sein. Im Container gibt es nur `C`, `C.utf8` und `POSIX` (`locale -a`), trotzdem führt `LANG=de_DE.UTF-8` zu `utf-8`. Tcl liest den Namen.
- Eine Locale ohne Zeichensatz (`de_DE`) behandeln 8.6.14 und 9.0.4 verschieden: 8.6 nimmt iso8859-1, 9.0 nimmt utf-8.

- open folgt in beiden Fassungen `encoding system`.
- Skripte liest Tcl 9 immer als UTF-8. Tcl 8.6 liest sie in `encoding system`.

6.3 Dateien: immer mit Kodierung

Weil die Vorgabe von der Locale abhängt, öffnet ein Programm jede Textdatei mit ausdrücklicher Kodierung. Das gilt unter 8.6 und 9:

```
set ch [open $pfad r]
fconfigure $ch -encoding utf-8
set text [read $ch]
close $ch
```

open selbst hat keine Option für die Kodierung, auch unter Tcl 9 nicht. Die Zeile `fconfigure` bleibt.

Tcl 9 bricht beim Lesen ungültiger Bytes ab, statt sie still umzudeuten (TIP 656, Profil `strict` als Vorgabe für Kanäle). Container-Prüfung mit einer Latin-1-Datei `Gr\xfc\xdfe`, gelesen mit `-encoding utf-8`:

```
8.6.14   Grüße
9.0.4    error reading "file5": invalid or incomplete multibyte or wide character
```

Tcl 8.6 deutet ungültige UTF-8-Bytes als Latin-1. Das sieht hier zufällig richtig aus und verdeckt, dass die Datei gar kein UTF-8 ist. Unter Tcl 9 fällt derselbe Fehler auf. Unter Linux trifft das vor allem alte Dateien und Ausgaben fremder Programme. Die Profile (`-profile replace`, `-profile tcl8`) beschreibt das Buch *Encoding*.

6.4 Skripte: die Wache unter Tcl 8.6

Ein UTF-8-Skript mit Umlauten im Quelltext läuft unter Tcl 8.6 nur richtig, wenn die Locale UTF-8 ist. Unter LANG=C wird aus `Rückgängig` die Zeichenfolge `RÃ¼ckgÃ¤ngig` (`serie-linux`, Howto 08). Das trifft Programme, die aus Cron, aus einer Systemd-Einheit oder aus einem Container ohne Locale starten.

Abhilfe ohne Startparameter ist eine Wache am Dateianfang. Die Datei selbst ist UTF-8. Liest Tcl sie als iso8859-1, ist `"ü"` zwei Zeichen lang. Dann liest die Wache die Datei noch einmal, diesmal als UTF-8:

```
if {[string length "ü"] != 1} {
    source -encoding utf-8 [file normalize [info script]]
    return
}
```

`beispiele/utf8-wache.tcl` zeigt das. Container-Prüfung, unter LANG=C und LANG=C.UTF-8, mit 8.6.14 und 9.0.4: In allen vier Läufen hat `"Rückgängig"` die Länge 10. Unter LANG=C sieht das Terminal die Umlaute trotzdem falsch, weil `stdout` in iso8859-1 schreibt. Der String im Programm stimmt.

Die Wache gehört in das Startskript. Für weitere Dateien gibt es `source -encoding utf-8` direkt, für Pakete den Eintrag in `pkgIndex.tcl`:

```
package ifneeded meinpaket 1.0 \
    [list source -encoding utf-8 [file join $dir meinpaket.tcl]]
```

Alternativ ruft der Startbefehl die Kodierung ausdrücklich: `tclsh8.6 -encoding utf-8 app.tcl`. Das geht nur, wenn man den Aufruf selbst in der Hand hat, also nicht über eine Shebang-Zeile mit `env` (siehe Welches Tcl?).

6.5 Terminal und exec

`stdout`, `stderr` und `stdin` stehen beim Start auf `encoding system` (Container: `fconfigure stdout -encoding` liefert `utf-8` unter `C.UTF-8`). Dasselbe gilt für Kanäle zu Kindprozessen: `exec` und `open "|…"` dekodieren die Ausgabe des Kindes in `encoding system`. `exec printf %s "Grüße"` liefert unter `LANG=C.UTF-8` `Grüße`. Ein Kind, das immer UTF-8 schreibt, liest man unter unbekannter Locale über einen Kanal mit fester Kodierung:

```
set ch [open |[list git log -1 --format=%an] r]
fconfigure $ch -encoding utf-8
set autor [string trim [read $ch]]
close $ch
```

Siehe exec, xdg-open, Prozesse.

6.6 msgcat und Sprache

`msgcat::mclocale` liest ebenfalls die Locale-Variablen, in der Reihenfolge `LC_ALL`, `LC_MESSAGES`, `LANG` (Quelltext von `msgcat.tcl` in Tcl 9.0.4). Im Container mit `LANG=C.UTF-8` ist das Ergebnis `c`. Ein Programm mit deutschen Meldungen in `de.msg` braucht also eine deutsche Locale oder einen ausdrücklichen Aufruf `msgcat::mclocale de`. `LANGUAGE`, unter Linux die Sprachliste für gettext, wertet `msgcat` nicht aus.

6.7 Zusammenfassung

- `encoding system` folgt `LC_ALL`, `LC_CTYPE`, `LANG`. `C` und fehlende Locale ergeben iso8859-1.
- `open` folgt `encoding system`. Dateien immer mit `fconfigure -encoding utf-8`.
- Tcl 9 liest Skripte als UTF-8, Tcl 8.6 in `encoding system`. Für 8.6 die Wache am Anfang des Startskripts.
- `de_DE` ohne Zeichensatz: 8.6 iso8859-1, 9.0 utf-8.
- Ausgaben von Kindprozessen über einen Kanal mit fester Kodierung lesen.

Nächstes Kapitel: exec, xdg-open, Prozesse

7 exec, xdg-open, Prozesse

7.1 Einführung

Unter Linux ist `exec` das Werkzeug für alles, was Tcl nicht selbst kann: Dateien mit dem passenden Programm öffnen, den Browser starten, Systemwerte abfragen. Es gibt keine Shell dazwischen. Tcl startet das Programm direkt, mit den Argumenten als Liste. Das macht `exec` sicher gegen Leerzeichen und Sonderzeichen, aber Shell-Syntax wie `*`, `~` oder `$VAR` wirkt nicht.

7.2 Programm finden: auto_execok

auto_execok sucht ein Programm im PATH und liefert den vollen Pfad als Liste, oder einen leeren String:

```
set p [auto_execok xdg-open]
if {$p eq ""} {
    error "xdg-open fehlt"
}
exec {*}$p $datei &
```

Im Container: `auto_execok sh` liefert `/usr/bin/sh`, `auto_execok xdg-open` liefert `/usr/bin/xdg-open`, `auto_execok lpstat` einen leeren String (kein CUPS). Das Ergebnis ist eine Liste, daher das `{*}`. Unter Windows kann sie mehrere Elemente haben, unter Linux ist es ein Pfad.

`auto_execok` merkt sich jede Antwort im globalen Array `auto_execs`, auch die leere. Wer nach dem Programmstart ein Werkzeug installiert, löscht den Eintrag, sonst bleibt die alte Antwort:

```
unset -nocomplain ::auto_execs(xdg-open)
```

`auto_reset` leert das Array ebenfalls, entfernt aber zugleich alle über `auto_index` nachgeladenen Befehle (Quelltext `auto.tcl` in Tcl 9.0.4). Für diesen Zweck ist es zu grob.

7.3 Fehler von exec auseinanderhalten

`exec` wirft einen Fehler in drei verschiedenen Fällen. Welcher vorliegt, steht in `errorCode`. Container-Prüfung, 8.6.14 und 9.0.4 gleich:

Fall	Meldung	errorCode
Programm fehlt	couldn't execute "gibtsnicht": no such file or directory	POSIX ENOENT {no such file or directory}
Rückgabewert ungleich 0 (false)	child process exited abnormally	CHILDSTATUS <pid> 1
Ausgabe auf stderr	der stderr-Text, hier meldung	NONE
Kind durch Signal beendet	child killed: software termination signal	CHILDKILLED <pid> SIGTERM {software termination signal}

Der dritte Fall ist die bekannte Falle: Ein Programm, das erfolgreich läuft, aber eine Warnung auf stderr schreibt, gilt für `exec` als Fehler. Abhilfe:

```
exec -ignorestderr programm arg      ;# stderr geht an Tcls stderr
exec programm arg 2>@1               ;# stderr in das Ergebnis
exec programm arg 2>/dev/null        ;# stderr verwerfen
```

Ein Muster für alle Fälle:

```
try {
    set aus [exec -ignorestderr {*}$befehl]
} trap {CHILDSTATUS} {msg opts} {
    set rc [lindex [dict get $opts -errorcode] 2]
    puts "Rückgabewert $rc"
} trap {POSIX ENOENT} {} {
    puts "Programm fehlt"
}
```

7.4 Im Hintergrund und über Kanäle

Ein `&` am Ende startet das Programm im Hintergrund. `exec` wartet nicht und liefert die Prozessnummern:

```
set pids [exec xdg-open $datei &]
```

Für lang laufende Programme, deren Ausgabe man lesen will, ohne die Oberfläche anzuhalten, ist ein Kanal das richtige Werkzeug:

```
set ch [open |[list ping -c 3 localhost] r]
fconfigure $ch -blocking 0 -encoding utf-8
fileevent $ch readable [list lesen $ch]
proc lesen {ch} {
    if {[gets $ch zeile] >= 0} { .t insert end $zeile\n }
    if {[eof $ch]} {
        fconfigure $ch -blocking 1
        catch {close $ch} fehler
    }
}
```

Das `fconfigure -blocking 1` vor dem `close` sorgt dafür, dass `close` den Rückgabewert des Kindes meldet. Die Kodierung des Kanals folgt sonst `encoding system`, siehe Kodierung und Locale.

7.4.1 Die Falle mit der eigenen Auswahl

Ein blockierendes `exec` hält die Ereignisschleife an. Meist ist das nur unschön. Es wird zur Verklemmung, wenn das Kind auf das eigene Programm wartet. Container-Prüfung: Ein `wish` markiert Text in einem Entry und ruft dann `exec xclip -o -selection primary`. `xclip` fragt beim Besitzer der Auswahl nach, also bei diesem `wish`. Der steht in `exec` und antwortet nicht. Beide warten, bis `timeout` den Lauf nach 20 Sekunden beendet. Mit einem Kanal und `fileevent` wie oben läuft derselbe Aufruf durch (`beispiele/auswahl-probe.tcl`). Mehr zur Auswahl in Maus, Tastatur, Zwischenablage.

7.5 xdg-open: Datei oder Adresse öffnen

xdg-open ist das Linux-Gegenstück zu `start` unter Windows und `open` unter macOS. Es öffnet eine Datei mit dem eingestellten Programm oder eine Adresse im Browser (`serie-linux`, Howto 07):

```
proc oeffnen {ziel} {
    set p [auto_execok xdg-open]
    if {$p eq ""} { error "xdg-open fehlt (Paket xdg-utils)" }
    exec {*}$p $ziel &
}
oeffnen https://www.tcl-lang.org
oeffnen [file join $env(HOME) bericht.pdf]
```

`xdg-open` gehört zum Paket `xdg-utils`. Es fragt die Desktop-Umgebung, welches Programm zuständig ist. Unter Xfce reicht es an `exo-open` weiter (siehe Xfce aus Tcl). Die Zuordnung für einen Dateityp zeigt `xdg-mime query default <MIME-Typ>`.

Das `&` ist wichtig. Ob `xdg-open` sofort zurückkehrt oder wartet, bis das geöffnete Programm endet, hängt von der Desktop-Umgebung und vom Programm ab; ohne `&` kann die Oberfläche stehen bleiben. Gemessen mit Tcl 8.6:

Aufruf	Arbeitsplatz, Xfce	Container ohne Desktop
exec xdg-open /etc/hostname &	Prozessnummer sofort, Mousepad öffnet	ruft LibreOffice auf und wartet
exec xdg-open https://example.org &	Prozessnummer sofort, Browser öffnet	kein Browser, Exit 3
exec xdg-open /gibtsnicht	CHILDSTATUS … 4	CHILDSTATUS … 2

Die Exitcodes stehen in `xdg-open(1)`: 2 Datei fehlt, 3 kein Programm, 4 Aktion gescheitert. Unter Xfce prüft `xdg-open` die Datei nicht selbst, sondern reicht sie an `exo-open` weiter; dessen Fehler wird 4. Ohne Desktop-Sitzung (`xdg-utils` 1.1.3, Container) prüft es selbst und meldet 2. Code prüft also nur "nicht 0", nicht eine bestimmte Zahl. Die Fenster erscheinen nicht in der Reihenfolge der Aufrufe: Auf dem Arbeitsplatz kam der Browser vor Mousepad, obwohl `/etc/hostname` zuerst geöffnet wurde. Wer danach ein Fenster erwartet, wartet nicht auf eine Reihenfolge. Die Serien starten `xdg-open` unter `SERIES_AUTO` nicht.

7.6 Prozessnummer und Signale

```
pid                     ;# eigene Prozessnummer
pid $ch                 ;# Prozessnummern hinter einem Kanal
```

Ein Signal schickt Tcl nicht selbst. `exec kill -TERM $p` tut es. Ein Tcl-Programm, das auf `SIGTERM` reagieren soll, braucht eine Erweiterung (etwa `TclX` mit `signal`); ohne sie endet es sofort. Das ist nicht geprüft.

Unter Linux hat jedes Programm `/proc/<pid>/` mit `cmdline`, `environ` und `status`. Ob ein Prozess noch lebt:

```
file isdirectory /proc/$p
```

7.7 Zusammenfassung

- `auto_execok` vor jedem `exec` eines Fremdprogramms, Ergebnis mit `{*}` einsetzen.
- `errorCode` trennt: `POSIX ENOENT`, `CHILDSTATUS`, `NONE` (stderr), `CHILDKILLED`.
- `-ignorestderr` für Programme, die auf stderr plaudern.

- Lange Läufe über `open |…` mit `fileevent`. Nie blockierend ein Programm rufen, das das eigene Programm fragt.
- `xdg-open` mit `&`, Paket `xdg-utils`.

Nächstes Kapitel: X11, Fenstermanager und Wayland

8 X11, Fenstermanager und Wayland

8.1 Einführung

Tk spricht unter Linux nur X11. `tk windowingsystem` liefert immer `x11`. Anders als unter Windows und macOS ist das Fenstersystem dabei in zwei Programme geteilt: den X-Server, der zeichnet und Eingaben liefert, und den Fenstermanager, der Rahmen, Titelleiste, Fokus und Iconify beisteuert. Ein Tk-Programm kann ohne Fenstermanager laufen, aber dann fehlt ihm einiges. Dieses Kapitel trennt, was Tk selbst tut, was der Fenstermanager tut, und was eine Wayland-Sitzung daran ändert.

8.2 DISPLAY und der Start ohne Anzeige

Tk verbindet sich beim Start mit dem X-Server aus DISPLAY. Fehlt die Variable oder antwortet der Server nicht, scheitert Tk. Container-Prüfung, 8.6.14 und 9.0.4 gleich:

```
$ env -i wish9.0 probe.tcl
application-specific initialization failed: no display name and no $DISPLAY
    environment variable
$ env -i DISPLAY=:77 wish9.0 probe.tcl
application-specific initialization failed: couldn't connect to display ":77"
```

Bemerkenswert ist, was danach passiert: `wish` bricht **nicht** ab. Es führt das Skript ohne Tk aus, `package require Tk` liefert dann den Fehler, und der Rückgabewert von `wish` ist 0. Ein Testlauf in einer Pipeline, der nur auf den Rückgabewert schaut, meldet Erfolg, obwohl keine Zeile der Oberfläche gelaufen ist. Abhilfe am Anfang des Skripts:

```
if {[catch {package require Tk} fehler]} {
    puts stderr "Tk fehlt: $fehler"
    exit 1
}
```

`tclsh` mit `package require Tk` verhält sich wie erwartet: Der Fehler bricht das Skript ab.

Was Tk über den Server weiß:

```
winfo server .          ;# X11R0 The X.Org Foundation 12101011 (Xvfb)
winfo screen .          ;# :93.0
winfo screenwidth .     ;# 1280
winfo screenmmwidth .   ;# 325
winfo visual .          ;# truecolor
```

8.3 Fensterklasse und WM_CLASS

Jedes Hauptfenster trägt die X-Eigenschaft WM_CLASS mit zwei Teilen: Name und Klasse. Der Fenstermanager, die Taskleiste und `.desktop`-Dateien erkennen ein Programm daran. Tk leitet

beide aus dem Programmnamen ab. Container-Prüfung mit `xprop`, 8.6.14 und 9.0.4 gleich:

Start	tk appname	WM_CLASS
wish9.0 meinprog.tcl	meinprog.tcl	"meinprog.tcl", "Meinprog.tcl"
wish9.0 meinprog.tcl -name meinname	meinname	"meinname", "Meinname"
tclsh9.0 meinprog mit package require Tk	meinprog	"meinprog", "Meinprog"

Die Klasse ist der Name mit großem Anfangsbuchstaben. Einen eigenen Klassennamen für `.` nimmt Tk über die Befehlszeile nicht an: Ein `-class` in `argv` bleibt unbeachtet in `argv` stehen. `-name` dagegen wirkt auch, wenn ein Skript es vor `package require Tk` in `::argv` setzt:

```
set ::argv [list -name meineapp]
set ::argc 2
package require Tk           ;# WM_CLASS "meineapp", "Meineapp"
```

Für ein Programm, das als `meineapp.tcl` startet, heißt die Klasse also `Meineapp.tcl`, mit Endung. Wer eine `.desktop`-Datei mit `StartupWMClass` schreibt, muss genau diesen Wert treffen, siehe Native Dialoge und Desktop-Integration. Weitere Toplevels tragen ihren Fensternamen und ihre Klasse: Ein `toplevel .t -class Zweit` hat im Container `WM_CLASS "t", "Zweit"` (8.6.14 und 9.0.4). Für `.` selbst gibt es diesen Weg nicht.

Tk setzt außerdem `WM_NAME` und `_NET_WM_NAME` aus `wm title` und `_NET_WM_PID`, sobald `WM_CLIENT_MACHINE` gesetzt ist (Quelltext `tkUnixWm.c`). Im Container fehlte `_NET_WM_PID` am Fenster; die Bedingung dafür ist nicht untersucht.

8.4 Was der Fenstermanager beisteuert

Ein reparentierender Fenstermanager hängt jedes Hauptfenster in einen eigenen Rahmen mit Titelleiste. Daraus folgen zwei Koordinaten: `wm geometry` setzt und liefert die **Rahmenecke**, `winfo rootx` und `winfo rooty` die Ecke der **Clientfläche**. Die Differenz ist der Rahmen.

```
wm geometry . 360x220+30+40
update
scan [wm geometry .] {%dx%d+%d+%d} w h gx gy
set titel [expr {[winfo rooty .] - $gy}]
```

Gemessen in `serie-xvfb` (Howto 04 und 13, Tcl/Tk 8.6.14 und 9.0.4):

Umgebung	Titelleiste rooty - gy	wm frame . gegen winfo id .
Xvfb ohne Fenstermanager	0	gleich
Xvfb mit openbox	20 (seitlich 1)	verschieden
Xfce auf dem Arbeitsplatz des Autors	29 (seitlich 5)	verschieden

Unter Xfce gleich bei 8.6.18, 9.0.4 und 9.1.0; Beispiel 9.0.4: `wm frame 0x92e055`, `winfo id 0x7a00009`.

Eine feste Zahl wie 80 für "unter das Hauptfenster" trifft keine dieser Umgebungen. Wer einen

Dialog unter die Titelleiste des Hauptfensters setzen will, nimmt `winfo rooty.`.

Was ohne Fenstermanager fehlt, Container-Prüfung mit `beispiele/wm-probe.tcl`, 8.6.14 und 9.0.4 gleich:

Vorgang	**ohne Fenstermanager**
Fokus nach update	leer, niemand vergibt ihn
wm iconify .	Zustand bleibt normal, Tk wartet rund 2000 ms
wm attributes . -fullscreen 1	Attribut bleibt 0, Größe unverändert
focus -force .	wirkt
grab	wirkt (Tk-intern, serie-xvfb, Howto 11)

Mit openbox wird das Fenster nach `wm iconify` zu `iconic`, und `.` hat nach `update` meist schon den Fokus (`serie-xvfb`, Howto 09 und 10). Vollbild, `-zoomed` und `-type` setzt Tk über die EWMH-Eigenschaften `_NET_WM_STATE` und `_NET_WM_WINDOW_TYPE`. Ob sie wirken, entscheidet der Fenstermanager. `wm attributes. -alpha 0.8` setzt `_NET_WM_WINDOW_OPACITY`, und zwar am Hüllfenster, das Tk um die Clientfläche legt (Quelltext `tkUnixWm.c`). Unter einem reparentierenden Fenstermanager ist das weder `wm frame` noch `winfo id`: Mit openbox steht die Eigenschaft am Hüllfenster und am Rahmen, den openbox damit versorgt, nicht am Client (Container, 8.6.14 und 9.0.4). Das Hüllfenster nennt `xwininfo` als Elternfenster des Clients:

```
regexp {Parent window id: (0x[0-9a-f]+)} [exec xwininfo -tree -id [winfo id .]] ->
    huelle
exec xprop -id $huelle _NET_WM_WINDOW_OPACITY
```

Unter Xfce 4.20 mit Compositor (`use_compositing` wahr) nimmt `-alpha` 0,35 an und liefert 0,35 zurück, und das Fenster ist am Bild durchsichtig; mit `use_compositing` falsch ist es undurchsichtig (8.6.18, 9.0.4, 9.1.0). Am Rahmen und am Client steht die Eigenschaft nicht, am Hüllfenster schon: `1503238553`, also 0,35 × `0xFFFFFFFF`, bei 8.6.18, 9.0.4 und 9.1.0, mit und ohne Compositor. Tk setzt sie also immer; ob man etwas sieht, entscheidet allein der Compositor.

Die unter X11 bekannten Attribute, Container-Prüfung:

```
8.6.14   -alpha 1.0 -topmost 0 -zoomed 0 -fullscreen 0 -type {}
9.0.4    -alpha 1.0 -fullscreen 0 -topmost 0 -type {} -zoomed 0
```

Gleicher Inhalt, andere Reihenfolge. Wer die Ausgabe als Liste vergleicht, vergleicht sie als `dict`.

8.5 Einen Fenstermanager erkennen

`winfo manager.` hilft nicht: Für ein Toplevel ist es immer `wm`, mit und ohne Fenstermanager (`serie-xvfb`, Howto 02). Was trägt, zeigt `serie-xvfb` in Howto 09:

1. `wm frame.` gegen `winfo id.`. Verschieden heißt: Ein reparentierender Fenstermanager läuft.
2. Sind sie gleich, entscheidet eine Iconify-Probe an einem Hilfsfenster. Wird es `iconic`, läuft ein Fenstermanager ohne Rahmen (dwm, xmonad). Bleibt es `normal`, läuft keiner.

Die Ids **numerisch** vergleichen. Tk 8.6 schreibt `winfo id` mit führenden Nullen, `wm frame` ohne (Container: `0x200003` gegen `0x00200003`), Tk 9.0 schreibt beide ohne. Vor dem ersten Mappen ist `wm frame 0x0`, also erst `update`.

```
update
set rahmen [expr {[wm frame .] != [winfo id .]}]
```

Die Iconify-Probe kostet ohne Fenstermanager rund zwei Sekunden. Die Prozeduren `serie::wmRahmen` und `serie::wmDa` in der Serie erlauben deshalb, das Ergebnis über `SERIE_WM=0` oder `1` vorzugeben.

8.6 Wayland

Tk 9.0.4 hat kein Wayland-Backend. Der Quelltext erwähnt Wayland an genau zwei Stellen: in `scaling.tcl` (siehe Schriften, Skalierung, Themes) und in `tkUnixEvent.c`, wo Tk eine unsinnige Bildschirmbreite in Millimetern unter XWayland auf 75 dpi zurücksetzt.

In einer Wayland-Sitzung läuft ein Tk-Programm deshalb über **XWayland**, den X-Server, den der Wayland-Compositor mitbringt. Dann sind `DISPLAY` und `WAYLAND_DISPLAY` beide gesetzt, und `XDG_SESSION_TYPE` ist `wayland`. `tk windowingsystem` bleibt `x11`.

```
set sitzung [expr {[info exists env(XDG_SESSION_TYPE)]
                   ? $env(XDG_SESSION_TYPE) : "unbekannt"}]
```

Was unter XWayland anders ist (globale Fensterpositionen, Fokus aus fremden Programmen holen, Bildschirmfotos, Skalierung bei mehreren Bildschirmen), hängt am Compositor. Auf dem Arbeitsplatz des Autors läuft Xfce unter X11 (`serie-xfce`: `XDG_SESSION_TYPE=x11`, `WAYLAND_DISPLAY` leer); das Debian-Buch meldet für den Debian-Arbeitsplatz ebenfalls `x11`. Eine echte Wayland-Sitzung (GNOME, KDE, Xfce mit Wayland) ist für dieses Buch **nicht geprüft**.

8.6.1 XWayland unter Weston

Geprüft ist XWayland im Container: Weston 13.0.0 verschachtelt im Xvfb (`weston --backend=x11-backend.so --xwayland`), Tk über `DISPLAY=:0`, 8.6.14 und 9.0.4 gleich:

Probe	unter XWayland (Weston)
tk windowingsystem	x11
wm frame . gegen winfo id .	verschieden, Weston setzt einen Rahmen
wm geometry . +100+100	Weston setzt das Fenster selbst, wm geometry meldet +32+32
wm geometry . +400+300	meldet +400+300, das Fenster bleibt stehen
winfo rootx/rooty	6,27 für jedes Fenster, fest
focus -force .	wirkungslos, der Fokus bleibt beim zuletzt gezeigten Fenster
wm iconify .	Zustand bleibt normal
-fullscreen 1	wirkt, ganze Ausgabe
xwd -root	BadMatch, kein Bild

Unter XWayland sind Fensterpositionen also weder setzbar noch lesbar: Tk meldet, was es verlangt hat, nicht, wo das Fenster steht. Code, der Dialoge selbst platziert oder Bildschirmfotos über X holt, verlässt sich darauf nicht. Andere Compositoren können sich anders verhalten.

8.7 Zusammenfassung

- `tk windowingsystem` ist unter Linux immer `x11`, auch unter Wayland (über XWayland). Dort sind Positionen weder setzbar noch lesbar, kein Iconify, kein `focus -force` (Weston).
- Ohne Anzeige meldet `wish` den Fehler, läuft ohne Tk weiter und endet mit 0. Skripte prüfen `package require Tk` selbst.
- `WM_CLASS` ist Programmname und Programmname mit großem Anfang, `-name` ändert beide.
- `wm geometry` ist die Rahmenecke, `winfo rooty` die Clientfläche. Rahmen: 0, 20 oder 29 Pixel, nie fest annehmen.
- Ohne Fenstermanager: kein Fokus, kein Iconify, kein Vollbild. Erkennen über `wm frame` und eine Iconify-Probe.

Nächstes Kapitel: Schriften, Skalierung, Themes

9 Schriften, Skalierung, Themes

9.1 Einführung

Unter X11 zeichnet Tk Schrift mit Xft und sucht sie über fontconfig. Die Größe der Oberfläche hängt an `tk scaling`, und das hängt an der Auflösung, die der X-Server meldet. Farben und Themes kommen aus Tk selbst und aus der X-Ressourcendatenbank, nicht aus den Einstellungen des Desktops. Dieses Kapitel zeigt, woher jeder Wert kommt und was Tk 9 an der Skalierung geändert hat.

9.2 Schriften: Xft und fontconfig

Ob Tk mit Xft gebaut ist, sagt `tk::pkgconfig get fontsystem`. Im Container liefert es unter 8.6.14 und 9.0.4 `xft`. Dann setzt Tk die benannten Schriften auf die fontconfig-Namen `sans-serif` und `monospace` (Quelltext `ttk/fonts.tcl`). Welche Familie dahinter steht, entscheidet fontconfig auf dem Rechner, auf dem das Programm läuft:

```
fc-match sans-serif     # Container: Inter-Regular.otf: "Inter" "Regular"
fc-match monospace      # Container: DejaVuSansMono.ttf: "DejaVu Sans Mono" "Book"
```

Container-Prüfung, beide Fassungen gleich:

Name	Familie	Größe
TkDefaultFont, TkTextFont, TkMenuFont, TkIconFont	Inter	10
TkHeadingFont	Inter, fett	10
TkCaptionFont	Inter, fett	12
TkTooltipFont, TkSmallCaptionFont	Inter	9

Name	Familie	Größe
TkFixedFont	DejaVu Sans Mono	10

`serie-xvfb` (Howto 01) meldet ebenfalls `TkDefaultFont Inter 10`. Auf einem Rechner ohne Inter liefert fontconfig eine andere Familie, meist DejaVu Sans. Der Text darf das nicht voraussetzen.

Die Größen sind **Punkte**, keine Pixel. Ein Wunschname, den es nicht gibt, führt nicht zu einem Fehler: `font actual {GibtsNicht 10}` liefert im Container `Inter`. `Helvetica` wird zu `TeX Gyre Heros`, weil fontconfig dafür einen Ersatz kennt. Wer prüfen will, ob eine Familie wirklich da ist, fragt `font families`:

```
if {"Noto Sans" in [font families]} {
    font configure TkDefaultFont -family "Noto Sans"
}
```

Die Schriften kommen vom **Client**, nicht vom X-Server. Ein Programm, das über `ssh -X` auf einem anderen Rechner läuft, zeichnet mit den Schriften des entfernten Rechners. Dasselbe gilt für Xvfb: Der virtuelle Server hat keine eigenen Schriften im Sinn von Xft.

9.3 tk scaling und DPI

tk scaling ist die Zahl der Pixel je Punkt. Tk rechnet sie beim Start aus der Bildschirmgröße in Pixeln und Millimetern: Pixel ÷ (mm ÷ 25,4) ÷ 72. Bei 100 dpi ist das 1,389. Alle Größen in Punkten, auch die Schriften, wachsen mit diesem Wert.

`serie-xvfb` (Howto 01, Tcl/Tk 8.6.14 und 9.0.4) misst das mit verschiedenen `-dpi`-Werten des Xvfb:

Xvfb	tk scaling
1280x1024, ohne -dpi	1,389 (≈ 100 dpi)
-dpi 50	0,695
-dpi 200	2,770

Container-Prüfung mit einem Label "Hallo Welt" in `TkDefaultFont` und einer `ttk::scrollbar`:

Xvfb	Fassung	tk scaling	::tk::scalingPct	Zeilenhöhe	Textbreite	Scrollbar
100 dpi	8.6.14	1,389	fehlt	18	69	15
100 dpi	9.0.4	1,389	100	18	69	15
192 dpi	8.6.14	2,667	fehlt	33	131	15
192 dpi	9.0.4	2,667	200	33	131	27

Die Schrift wächst in beiden Fassungen mit. Die Scrollbar wächst nur unter Tk 9: Tk 9 rechnet viele Maße in ttk und in den Standarddialogen über ::tk::scalingPct, die Auflösung in Prozent von 96 dpi, auf 25 gerundet und mindestens 100. Unter 8.6 bleiben Pixelmaße wie Scrollbarbreite und Checkbox fest.

9.3.1 Woher Tk 9 die Skalierung nimmt

Ein Desktop stellt die Vergrößerung meist nicht über die DPI des X-Servers ein, sondern über eigene Einstellungen. Tk 9 liest einige davon beim Start (Quelltext `library/scaling.tcl`, 9.0.4), in dieser Reihenfolge:

1. Läuft `xfce4-session`: `xfconf-query -c xsettings -p /Gdk/WindowScalingFactor`. Ab Faktor 2 gilt 200 %, und die Schriftgrößen werden verdoppelt (`::tk::fontScalingFactor` 2).
2. Läuft `mate-session`: `gsettings` für MATE.
3. Läuft `gnome-session`: der Schlüssel `Gdk/WindowScalingFactor` in den xsettings-Overrides.
4. Sonst die X-Ressource Xft.dpi, gelesen mit `xrdb -query`: Prozent = `Xft.dpi` ÷ 96 × 100.
5. Unter GNOME ohne Wayland zuletzt `~/.config/monitors.xml`.

Weicht der gefundene Wert vom gemessenen ab, setzt Tk 9 `tk scaling` neu.
Container-Prüfung: Xvfb mit 100 dpi, dazu `Xft.dpi: 192` über `xrdb -merge`:

Fassung	tk scaling	scalingPct	font actual … -size	Zeilenhöhe	Scrollbar
8.6.14	1,389	fehlt	19	33	15
9.0.4	2,672	200	10	33	27

Unter 8.6 vergrößert Xft die Schrift selbst, weil fontconfig `Xft.dpi` kennt. Tk weiß davon nichts: `tk scaling` bleibt 1,389, und `font actual` meldet die zurückgerechnete Größe 19 statt 10. Pixelmaße bleiben klein. Unter 9.0 stimmen Schrift, Rechenwert und Maße zusammen.

Wer die Skalierung selbst festlegen will, setzt sie vor dem Aufbau der Oberfläche:

```
tk scaling 2.0     ;# 144 dpi; vor dem ersten Widget
```

Unter Tk 9 zieht `tk scaling` den Wert `::tk::scalingPct` sofort nach: `tk scaling 2.0` ergibt 150 (Faktor × 75, auf 25 gerundet; Quelltext `tkCmds.c`). Wie stark ein Standarddialog dabei wächst, hängt an der Schrift. `tk_messageBox` vor und nach `tk scaling 2.0`:

Umgebung	Fassung	scalingPct	Dialog	Zeilenhöhe TkDefaultFont
Container, Xvfb 100 dpi	8.6.14	fehlt	117×95 → 162×114	18 → 25
Container, Xvfb 100 dpi	9.0.4	100 → 150	117×95 → 164×116	18 → 25
Arbeitsplatz, Xfce	8.6.18	fehlt	110×96 → 110×96	34 → 34
Arbeitsplatz, Xfce	9.0.4	100 → 150	110×96 → 110×100	34 → 34
Arbeitsplatz, Xfce	9.1.0	100 → 150	112×96 → 128×119	31 → 31

Im Container wächst die Schrift mit, und mit ihr der Dialog. Auf dem Arbeitsplatz bleibt die Schrift gleich; was dort wächst, rechnet Tk über `scalingPct`: unter 9.0 kaum, unter 9.1 deutlich.

9.4 Themes

Unter X11 bringt Tk vier ttk-Themes mit. Container-Prüfung, beide Fassungen: `ttk::style theme names` liefert `clam alt default classic`, aktiv ist `default`. Es gibt kein natives Linux-Theme wie `aqua` oder `vista`. Das Aussehen von GTK- oder Qt-Programmen übernimmt Tk nicht. Dafür gibt es Zusatzpakete (etwa `ttk::theme::`*-Sammlungen); sie sind nicht Teil dieses Buches.

Der Hintergrund klassischer Widgets ist `#d9d9d9`. `TButton -padding` meldet unter 8.6 `3 3`, unter 9.0 `2.25p`: Tk 9 rechnet Abstände in Punkten, damit sie mit `tk scaling` wachsen.

9.5 X-Ressourcen

Unter X11 liest Tk beim Start die Ressourcendatenbank des X-Servers, die Eigenschaft RESOURCE_MANAGER am Wurzelfenster, die `xrdb` setzt. Fehlt sie, liest Tk `~/.Xdefaults`. Container-Prüfung, 8.6.14 und 9.0.4 gleich:

```
*background:  lightyellow     -> option get . background Background: lightyellow
                                 button -background: lightyellow
*Label.font:  {DejaVu Sans} 14 -> label -font: {DejaVu Sans} 14
*TkTheme:     clam             -> ttk::style theme use: clam
```

*TkTheme wählt also das ttk-Theme, ohne eine Zeile Code. Die Datei `~/.Xdefaults` mit `*background: pink` wirkte im Container, solange keine Ressourcen geladen waren. Mit XENVIRONMENT auf eine Datei gesetzt änderte sich nichts.

Das hat zwei Seiten. Ein Benutzer kann so Farbe, Schrift und Theme eines Tk-Programms anpassen. Ein Programm, das auf einem fremden Rechner seltsam aussieht, hat dort vielleicht eine `~/.Xdefaults` aus alten Zeiten. `xrdb -query` zeigt, was geladen ist.

Ein Programm lädt eigene Vorgaben mit niedriger Priorität, damit der Benutzer sie überschreiben kann:

```
option add *Entry.background white widgetDefault
option readfile [file join $hier app.Xdefaults] startupFile
```

Eine Eigenheit von Xvfb: Ohne `-noreset` setzt der Server seinen Zustand zurück, sobald der letzte Client die Verbindung schließt. Ressourcen, die ein `xrdb` vorher geladen hat, sind dann weg, bevor das Tk-Programm startet. Im Container blieb `xrdb -query` aus diesem Grund leer, bis Xvfb mit `-noreset` lief. Für Tests mit Ressourcen gehört `-noreset` in die Xvfb-Optionen, siehe GUI-Tests mit Xvfb.

9.6 Zusammenfassung

- Xft und fontconfig: `sans-serif` und `monospace`, Familie vom Rechner des Programms. Größen in Punkten.
- `tk scaling` aus Pixeln und Millimetern. Tk 9 liest zusätzlich Xfce-, MATE-, GNOME-Einstellungen und `Xft.dpi` und skaliert auch Pixelmaße.
- Unter 8.6 vergrößert `Xft.dpi` nur die Schrift.
- Themes: `clam alt default classic`, Vorgabe `default`.
- X-Ressourcen wirken: `*background`, `*TkTheme`. Fehlt RESOURCE_MANAGER, gilt

`~/.Xdefaults`.

Nächstes Kapitel: Maus, Tastatur, Zwischenablage

10 Maus, Tastatur, Zwischenablage

10.1 Einführung

Drei Stellen, an denen Linux-Code anders aussieht als Code für Windows oder macOS: das Mausrad, das unter X11 als Maustaste ankommt; die Modifier, deren Namen unter X11 keine feste Taste bedeuten; und die zwei Zwischenablagen PRIMARY und CLIPBOARD. Beim Mausrad und bei `Command` haben sich Tk 8.6 und Tk 9 verschieden entschieden.

Alle Werte dieses Kapitels sind im Container mit `beispiele/rad-und-tasten.tcl` und `beispiele/auswahl-probe.tcl` ermittelt. Die Skripte schicken mit `xdotool` echte X-Ereignisse an das eigene Fenster. Die Tastaturbelegung des Xvfb ist die Vorgabe `pc105`, `us`.

10.2 Das Mausrad

X11 kennt kein Rad-Ereignis. Der X-Server meldet jede Raste als Druck und Loslassen einer Maustaste: 4 nach oben, 5 nach unten, 6 und 7 für das seitliche Kippen. Tk 8.6 reicht das so weiter. Tk 9 übersetzt es in MouseWheel wie unter Windows und macOS (Quelltext `tkEvent.c`: Tasten 4 bis 7 werden beim Drücken zu `MouseWheelEvent` mit `%D` gleich ±120, 6 und 7 bekommen zusätzlich Shift; das Loslassen wird verworfen).

Container-Prüfung:

X-Taste	Tk 8.6.14	Tk 9.0.4
4 (hoch)	<Button-4>, %b=4	<MouseWheel>, %D=120
5 (runter)	<Button-5>, %b=5	<MouseWheel>, %D=-120
6 (links)	<Button-4>, %s=1 (Shift)	<Shift-MouseWheel>, %D=120
7 (rechts)	<Button-5>, %s=1 (Shift)	<Shift-MouseWheel>, %D=-120
8 (Seitentaste "Zurück")	<ButtonPress>, %b=8	<Button-4>, %b=8
9 (Seitentaste "Vor")	<ButtonPress>, %b=9	<Button-5>, %b=9

Die letzten zwei Zeilen sind die Falle. Tk 9 hat die Nummern 4 und 5 freigemacht und an die Tasten 8 und 9 vergeben. Alter Code mit

```
bind .c <Button-4> {.c yview scroll -1 units}
bind .c <Button-5> {.c yview scroll  1 units}
```

scrollt unter Tk 9 nicht mehr mit dem Rad, sondern mit den Seitentasten einer Maus. Ein `event generate .l <Button-4>` liefert unter Tk 9 ebenfalls `%b=8`. Tk 8.6 lehnt `<Button-8>` dagegen ab ("bad button number "8"").

Die Klassenbindungen von `Text` (`bind Text`) zeigen denselben Unterschied:

```
8.6.14  <Shift-Button-5> <Shift-Button-4>
        <Button-5> <Button-4>
        <Shift-MouseWheel> <MouseWheel>
9.0.4   <Shift-Alt-MouseWheel> <Shift-MouseWheel>
        <Alt-MouseWheel> <MouseWheel>
```

Eigener Code für beide Fassungen:

```
proc rad {w delta} {
    $w yview scroll [expr {$delta > 0 ? -1 : 1}] units
}
bind .c <MouseWheel> {rad %W %D}
if {[tk windowingsystem] eq "x11"
        && ![package vsatisfies [package provide Tk] 9-]} {
    bind .c <Button-4> {rad %W 120}
    bind .c <Button-5> {rad %W -120}
}
```

%D ist unter X11 immer ein Vielfaches von 120. Feinere Werte, wie sie Touchpads unter Windows und macOS liefern, kommen unter X11 nicht an. Ob ein Touchpad unter Xfce glatte Werte erzeugt, ist nicht geprüft.

10.3 Modifier und Tasten

X11 hat acht Modifier-Bits: Shift, Lock, Control und Mod1 bis Mod5. Welche Taste welches Bit setzt, legt die Tastaturbelegung fest (`xmodmap -pm`). Im Container: Mod1 trägt Alt_L, Alt_R und Meta_L, Mod2 Num_Lock, Mod4 Super_L und Super_R, Mod5 ISO_Level3_Shift (AltGr).

Tk kennt dazu Namen, die Plattformen verbinden sollen. Container-Prüfung, jede Bindung einzeln gesetzt:

Bindung	8.6.14 greift bei	9.0.4 greift bei
<Control-c>	Strg+C	Strg+C
<Alt-c>	Alt+C	Alt+C
<Mod1-c>	Alt+C	Alt+C
<Mod4-c>	Super+C	Super+C
<Meta-c>	nichts	nichts
<Command-c>	Alt+C	nichts
<Option-c>	nichts	Alt+C
<Super-c>	Fehler beim bind	Fehler beim bind

Unter Tk 8.6 ist Command unter X11 dasselbe wie `Mod1`. Unter Tk 9.0.4 ist es dasselbe wie `Meta` (Quelltext `tkBind.c`), und `Option` ist `Alt`. Welches Bit `Meta` ist, ermittelt Tk aus der Belegung: das Bit, auf dem eine Taste mit dem Keysym `Meta_L` oder `Meta_R` in der ersten Ebene liegt (Quelltext `tkUnixKey.c`). Im Container liegt `Meta_L` auf Keycode 205, und dessen erste Ebene ist `NoSymbol`:

```
$ xmodmap -pke | grep 'keycode 205'
keycode 205 = NoSymbol Meta_L NoSymbol Meta_L
```

Tk findet also kein Meta-Bit. Dann greift `Meta`, und damit `Command`, bei keiner Taste.

Das Debian-Buch schreibt im Kapitel *Tcl unter Linux*: "Eine Bindung `<Command-c>` aus einem Mac-Programm reagiert unter Linux also auf Alt+C." Das gilt für Tk 8.6, geprüft mit 8.6.14. Unter Tk 9.0.4 reagiert sie im Container auf keine Taste. Auf dem Arbeitsplatz des Autors (Xfce, deutsche Belegung) ebenso: `xdotool` sendet Alt+C an ein Fenster mit nur `<Command-c>`. 8.6.18 löst aus und speichert die Bindung als `<Mod1-Key-c>`, 9.0.4 und 9.1.0 bleiben stumm und speichern `<Meta-Key-c>`. `Meta_L` liegt dort wie im Container auf Keycode 205 mit `NoSymbol` in der ersten Ebene. Die Regel des Debian-Buchs bleibt richtig: `Command` nur unter `aqua` binden, unter `x11` `Control`.

```
set mod [expr {[tk windowingsystem] eq "aqua" ? "Command" : "Control"}]
event add <<Speichern>> <$mod-s>
bind . <<Speichern>> speichern
```

Die Super-Taste (Windows-Taste) heißt in `bind` `Mod4`. Der Desktop fängt nur die Super-Kürzel ab, die er selbst belegt hat. Auf dem Arbeitsplatz (Xfce) kommt Super+C bei Tk an, als `<Mod4-c>` mit dem Keysym `Super_L`; die Sitzung belegt dort unter anderem Super+E, Super+P und Super+R. Welche belegt sind, zeigt `xfconf-query -c xfce4-keyboard-shortcuts -l -v`.

Die Taste selbst meldet Tk mit ihrem Keysym (%K): `Control_L`, `Alt_L`, `Super_L`. Beim Drücken eines Modifiers ist `%s` noch 0, das Bit erscheint erst bei der nächsten Taste.

10.4 Zwei Zwischenablagen

X11 hat mehrere "Auswahlen" (selections). Zwei sind für Programme wichtig:

- PRIMARY: der zuletzt markierte Text. Er wird ohne Kopierbefehl gesetzt und mit der mittleren Maustaste eingefügt.
- `CLIPBOARD`: die Zwischenablage im Sinn von Windows und macOS. Gesetzt mit Strg+C oder `<<Copy>>`, eingefügt mit Strg+V oder `<<Paste>>`.

`clipboard` in Tk meint CLIPBOARD, `selection` ohne Option meint PRIMARY. Container-Prüfung, 8.6.14 und 9.0.4 gleich, Text "Auswahl-Ü" im Entry markiert:

```
PRIMARY von innen:    Auswahl-Ü
PRIMARY über xclip:   Auswahl-Ü
CLIPBOARD vorher:     leer (CLIPBOARD selection doesn't exist or form "STRING" not
    defined)
CLIPBOARD nachher:    Auswahl-Ü        (nach event generate .e <<Copy>>)
CLIPBOARD über xclip: Auswahl-Ü
TARGETS:              MULTIPLE TARGETS TIMESTAMP TK_APPLICATION TK_WINDOW
    UTF8_STRING STRING
```

Markieren allein setzt PRIMARY, nicht CLIPBOARD. Tk bietet beide als `UTF8_STRING` und `STRING` an, Umlaute kommen also bei anderen Programmen an.

Der wichtigste Unterschied zu Windows: Eine X11-Auswahl ist kein gespeicherter Inhalt, sondern ein Versprechen des Programms, das sie besitzt. Endet das Programm, ist der Inhalt weg. Im Container lieferte `xclip -o -selection clipboard` nach dem Ende von `wish` nur `Error: target STRING not available`. Ein Zwischenablage-Verwalter wie

`xfce4-clipman` übernimmt den Inhalt beim Kopieren und hält ihn fest, aber nur, wenn es läuft. Auf dem Arbeitsplatz ist das Paket 1.6.6-1 installiert, ein Prozess läuft nicht; nach dem Ende von `wish` meldet `xclip` dort dasselbe wie im Container. Installiert heißt also nicht gestartet: `pgrep xfce4-clipman`.

`serie-linux` (Howto 11) zeigt: PRIMARY existiert erst nach einer X11-Selektion, `selection get -selection PRIMARY` schlägt sonst fehl. Code fragt deshalb mit `catch`:

```
if {[catch {selection get -selection PRIMARY} text]} {
    set text ""
}
```

Wer die Auswahl eines anderen Programms mit einem Werkzeug wie `xclip` liest, darf das nicht mit blockierendem `exec` tun, solange das eigene Programm Besitzer ist. Das verklemmt beide, siehe exec, xdg-open, Prozesse.

10.4.1 Virtuelle Ereignisse

Container-Prüfung, `event info`:

Ereignis	8.6.14	9.0.4 zusätzlich
<<Copy>>	<Control-Key-c> <Key-F16> <Control-Lock-Key-C> <Meta-Key-w> <Lock-Meta-Key-W> <Control-Key-Insert>	<Key-XF86Copy>
<<Paste>>	<Control-Key-v> <Key-F18> <Control-Lock-Key-V> <Control-Key-y> <Control-Lock-Key-Y> <Shift-Key-Insert>	<Key-XF86Paste>
<<PasteSelection>>	<ButtonRelease-2>	wie 8.6

`<<PasteSelection>>` fügt PRIMARY ein, nicht CLIPBOARD. Wer einem Text-Widget das mittlere Einfügen abgewöhnen will, löscht die Bindung für diese eine Klasse:

```
bind Text <<PasteSelection>> {break}
```

10.5 Zusammenfassung

- Mausrad: Tk 8.6 unter X11 `<Button-4>`/`<Button-5>`, Tk 9 überall `<MouseWheel>`. Unter Tk 9 sind `<Button-4>`/`<Button-5>` die Seitentasten 8 und 9.
- `Command` ist unter X11 in 8.6 Alt, in 9.0 und 9.1 keine Taste (Container und Xfce). Unter `x11` `Control` binden.
- Super heißt `Mod4`.
- PRIMARY durch Markieren, CLIPBOARD durch Kopieren. Beide verschwinden mit dem Programm, wenn kein Verwalter sie übernimmt.

Nächstes Kapitel: Native Dialoge und Desktop-Integration

11 Native Dialoge und Desktop-Integration

11.1 Einführung

Unter Windows und macOS ruft `tk_getOpenFile` den Dialog des Systems. Unter Linux gibt es kein "System" in diesem Sinn, sondern GTK, Qt und die Desktop-Portale, und Tk benutzt keines davon. Die Standarddialoge zeichnet Tk selbst. Zur Integration in den Desktop gehören außerdem eine `.desktop`-Datei, ein Symbol, Benachrichtigungen und, ab Tk 9, ein Symbol in der Leiste und der Druck über CUPS.

11.2 Die Standarddialoge sind Tk-Dialoge

```
set datei [tk_getOpenFile -title "Öffnen" \
    -filetypes {{Tcl {.tcl}} {Alle *}}]
set ordner [tk_chooseDirectory -mustexist 1]
tk_messageBox -message "Gespeichert." -type ok
```

Unter X11 führt `tk_getOpenFile` zu `::tk::dialog::file::` (Container-Prüfung, Rumpf der Prozedur unter 8.6.14 und 9.0.4):

```
if {$::tk_strictMotif} {
    return [::tk::MotifFDialog open {*}$args]
} else {
    return [::tk::dialog::file:: open {*}$args]
}
```

Der Dialog sieht deshalb auf jedem Linux gleich aus, unabhängig von Xfce, GNOME oder KDE. Er kennt keine Lesezeichen des Dateimanagers und keine Netzwerkorte von GVfs. `tk_messageBox`, `tk_chooseColor` und `tk fontchooser` sind ebenfalls Tk-Dialoge.

Eine Eigenheit des X11-Dateidialogs: Er zeigt versteckte Dateien (Punktdateien) in der Grundeinstellung **an**. Im Quelltext von `tkfbox.tcl` (8.6.14 und 9.0.4) steht `showHiddenVar 1` und `showHiddenBtn 0`. Wer sie ausblenden und einen Schalter anbieten will, lädt den Dialog einmal und setzt beide Variablen:

```
catch {tk_getOpenFile -gibtsnicht}      ;# lädt tkfbox.tcl
set ::tk::dialog::file::showHiddenVar 0
set ::tk::dialog::file::showHiddenBtn 1
```

`serie-linux` (Howto 11) öffnet die Dialoge nur, wenn `SERIES_AUTO` nicht gesetzt ist. Im Container sind sie nicht geöffnet worden.

Wer den Dialog des Desktops will, ruft ein Hilfsprogramm, etwa `zenity --file-selection` (GTK) oder `kdialog --getopenfilename` (KDE), über `exec` und liest die Ausgabe. Beide fehlten im Container. Auf dem Arbeitsplatz (Xfce) ist `zenity` 4.2.2 installiert, `kdialog` fehlt. `zenity --file-selection` aus `wish` 8.6.18, 9.0.4 und 9.1.0, gleich:

Aufruf	Datei gewählt	Abbrechen
exec zenity …	catch 1, Ergebnis Pfad plus GTK-Warnung, -errorcode NONE	catch 1, CHILDSTATUS … 1
exec -ignorestderr zenity …	catch 0, Ergebnis nur der Pfad	catch 1, child process exited abnormally

GTK schreibt bei jedem Aufruf eine Warnung nach stderr (`Gtk-WARNING … Failed to measure available space`). Ohne `-ignorestderr` macht `exec` daraus einen Fehler, auch wenn eine Datei gewählt wurde. Also:

```
proc dateiWaehlen {} {
    if {[catch {exec -ignorestderr zenity --file-selection} pfad]} {
        return ""           ;# Abbrechen: Exit 1
    }
    return $pfad
}
```

Welches Hilfsprogramm da ist, vorher prüfen:

```
foreach p {zenity kdialog} {
    if {[auto_execok $p] ne ""} { set hilfe $p; break }
}
```

11.3 Die .desktop-Datei

Ein Programm erscheint im Startmenü über eine .desktop-Datei nach der Desktop Entry Specification. Für einen Benutzer liegt sie in `~/.local/share/applications/`, für alle in `/usr/share/applications/`. Vorlage `beispiele/meinapp.desktop`:

```
[Desktop Entry]
Type=Application
Name=MeineApp
Name[de]=Meine Anwendung
Comment=Beispiel aus "Tcl/Tk unter Linux"
Exec=/opt/meineapp/bin/meineapp %F
Icon=meineapp
Terminal=false
Categories=Utility;
StartupWMClass=Meineapp
```

Hinweise zu den Schlüsseln:

- `Exec` mit vollem Pfad. Ein Programm aus dem Startmenü bekommt den `PATH` der Sitzung, nicht den aus `~/.bashrc`. Ein `/opt/tcl9/bin`, das nur in der Shell vorn steht, fehlt dort. `%F` reicht ausgewählte Dateien weiter.
- `Icon` ohne Endung und ohne Pfad sucht im Symbolthema, etwa `~/.local/share/icons/hicolor/48x48/apps/meineapp.png`. Ein voller Pfad geht auch.
- StartupWMClass verbindet das laufende Fenster mit dem Eintrag, damit die Leiste das richtige Symbol zeigt. Der Wert ist die Klasse aus `WM_CLASS`. Ein mit `wish9.0 meineapp.tcl` gestartetes Programm hat die Klasse `Meineapp.tcl`; mit `-name meineapp` wird daraus `Meineapp`. Siehe X11, Fenstermanager und Wayland.

Prüfen mit `desktop-file-validate` aus dem Paket `desktop-file-utils`. `serie-xfce` (Howto 07) schreibt eine minimale Datei (`Type`, `Name`, `Exec=true`, `Terminal`, `Categories`) und prüft sie damit; unter Xfce 4.20 ohne Meldung. Im Container fehlte das Werkzeug. Die Vorlage oben, `beispiele/meinapp.desktop`, prüft es auf dem Arbeitsplatz ohne Meldung, Exit 0.

11.4 Fenstersymbol

```
image create photo ikon -file [file join $hier meineapp.png]
wm iconphoto . -default ikon
```

`-default` gilt für alle späteren Toplevels. Unter X11 legt Tk das Bild in die Eigenschaft `_NET_WM_ICON`; die Leiste nimmt es, wenn sie über `StartupWMClass` keinen besseren Eintrag findet. Im Container lief der Aufruf ohne Fehler; sichtbar ist das Symbol ohne Leiste nicht. Mehrere Größen in einem Aufruf (`wm iconphoto. -default ikon16 ikon32 ikon48`) überlassen die Wahl dem Desktop.

11.5 Benachrichtigungen

Zwei Wege: das Programm `notify-send` und, ab Tk 9, `tk sysnotify`.

`serie-xfce` (Howto 06) ruft unter Xfce 4.20:

```
exec {*}[auto_execok notify-send] -t 1000 -e --app-name meineapp \
    "Titel" "Text"
```

Den Dienst stellt dort `xfce4-notifyd`. `notify-send` braucht den Sitzungs-DBus. Im Container ohne Sitzung endet es mit

```
Failed to execute child process "dbus-launch" (No such file or directory)
```

und Rückgabewert 1.

tk sysnotify lädt unter X11 zur Laufzeit `libnotify.so.4` (oder ältere Fassungen) per `Tcl_LoadFile`; Tk ist nicht dagegen gelinkt (Quelltext `tkUnixSysNotify.c`). Gelingt das, gibt es den Befehl `::tk::sysnotify::_sysnotify`, und die Meldung geht an libnotify. Fehlt die Bibliothek, zeichnet Tk ein eigenes, randloses Fenster oben rechts, das nach einer Weile ausblendet (`systray.tcl`, Prozedur `_notifywindow`).

Container-Prüfung mit 9.0.4: `libnotify4` ist installiert, `_sysnotify` ist vorhanden. `tk sysnotify Titel Text` kehrt ohne Fehler zurück, es erscheint aber nichts, und es bleibt kein Fenster zurück: Ohne DBus hat libnotify niemanden, dem es die Meldung geben kann. Ein Programm erfährt davon nichts. Mit Xfce ist `tk sysnotify` nicht geprüft.

11.6 Symbol in der Leiste

`tk systray` legt unter X11 ein Symbol im Benachrichtigungsbereich an. Tk benutzt dafür das Protokoll des System-Tray (XEmbed), übernommen aus dem Paket tktray (Quelltext `tkUnixSysTray.c`). Container-Prüfung mit 9.0.4:

```
image create photo ikon -width 16 -height 16
tk systray exists                       ;# 0
tk systray create -image ikon -text Probe
tk systray exists                       ;# 1
```

`exists` meldet 1, obwohl im Container kein Benachrichtigungsbereich läuft. Der Wert sagt nur, dass `create` gelaufen ist, nicht, dass jemand das Symbol zeigt. Unter Tk 8.6 gibt es den Befehl nicht ("unknown or ambiguous subcommand "systray""). Auf dem Arbeitsplatz (Xfce, 9.0.4 und 9.1.0) ebenso `exists` 1 nach `create`; der Text aus `-text` ist der Tooltip und erscheint nicht als Fenstername im Fensterbaum. Ob das Xfce-Panel das Symbol zeigt, ist nicht am Bild geprüft.

11.7 Drucken

`tk print` gibt es ab Tk 9. Unter X11 zeigt es einen eigenen Tk-Dialog und druckt über die Befehle `lpstat` und `lpr` von CUPS (Quelltext `print.tcl`). Ein Canvas geht als PostScript an den Drucker, ein Text-Widget als Text. Container-Prüfung ohne CUPS:

```
tk print                    wrong # args: should be "tk print w"
::tk::print::cups           Tcl-Ensemble (lpstat/lpr)
::tk::print::cups getprinters
    Unable to obtain the list of printers. Command "lpstat" not found.
    Please install the CUPS package for your system.
```

Die Druckerliste entsteht aus `lpstat -e`, der Standarddrucker aus `lpstat -d`. Druck über CUPS, PDF-Wege und Etikettendrucker behandelt das Buch *Printing*.

11.8 Zusammenfassung

- `tk_getOpenFile` und Verwandte sind unter X11 Tk-Dialoge, keine GTK- oder Qt-Dialoge. Versteckte Dateien sind sichtbar, bis `showHiddenVar` 0 ist.
- `.desktop` in `~/.local/share/applications`, `Exec` mit vollem Pfad, `StartupWMClass` gleich der Klasse aus `WM_CLASS`.
- `notify-send` und `tk sysnotify` brauchen den Sitzungs-DBus. `tk sysnotify` schweigt, wenn er fehlt.
- `tk systray exists` sagt nichts über die Sichtbarkeit.
- `tk print` unter X11: CUPS-Befehle `lpstat` und `lpr`.

Nächstes Kapitel: Xfce aus Tcl

12 Xfce aus Tcl

12.1 Einführung

Xfce ist der Desktop auf dem Arbeitsplatz des Autors. Ein Tcl-Programm spricht mit ihm über Kommandozeilenwerkzeuge: Einstellungen über `xfconf-query`, bevorzugte Programme über `exo-open`, Meldungen über `notify-send`. Alle drei brauchen den Sitzungs-DBus. Dieses Kapitel folgt `serie-xfce`, geprüft mit Xfce **4.20** unter X11 auf Debian. Im Container gibt es kein Xfce; jeder Wert hier stammt aus der Serie.

12.2 Xfce erkennen

`serie-xfce` (Howto 01 und 04):

```
$env(XDG_CURRENT_DESKTOP)       ;# XFCE
$env(XDG_SESSION_TYPE)          ;# x11
$env(XDG_MENU_PREFIX)           ;# xfce-
$env(DBUS_SESSION_BUS_ADDRESS)  ;# unix:path=/run/user/UID/bus
exec xfce4-about --version      ;# … Xfce 4.20
```

`XDG_CURRENT_DESKTOP` kann eine Liste mit Doppelpunkten sein (`ubuntu:GNOME`). Eine Prüfung sucht daher ein Element:

```
proc desktop {} {
    if {![info exists ::env(XDG_CURRENT_DESKTOP)]} { return "" }
    return [split [string toupper $::env(XDG_CURRENT_DESKTOP)] :]
}
if {"XFCE" in [desktop]} { ... }
```

Ohne Desktop-Sitzung bricht die Serie ab, bevor sie etwas prüft. Im Container meldet Howto 01:

```
SKIP: serie-xfce braucht XFCE (XDG_CURRENT_DESKTOP=)
```

12.3 xfconf: Einstellungen lesen

Xfce speichert seine Einstellungen in xfconf, einem Dienst am Sitzungs-DBus. Er ist in Kanäle geteilt, jeder Kanal in Eigenschaften mit Pfaden (`serie-xfce`, Howto 02):

```
exec xfconf-query -l                            ;# Kanäle
exec xfconf-query -c xfwm4 -l                   ;# Eigenschaften
exec xfconf-query -c xfwm4 -p /general/theme
exec xfconf-query -c xfwm4 -p /general/workspace_count
```

Vorhandene Kanäle unter anderem `xfwm4`, `thunar`, `xfce4-panel`, `xfce4-session`, `xsettings`. Die Werte (Theme, Zahl der Arbeitsflächen) sind Sache des Benutzers. Die Serie prüft deshalb nur Typ und Länge, nicht einen bestimmten Wert. Ohne Sitzung scheitert jeder Aufruf mit "Verbindung ist gescheitert".

Ein Leser mit Vorgabe:

```
proc xfconf {kanal pfad {vorgabe ""}} {
    set p [auto_execok xfconf-query]
    if {$p eq ""} { return $vorgabe }
    if {[catch {exec {*}$p -c $kanal -p $pfad 2>/dev/null} wert]} {
        return $vorgabe
    }
    return $wert
}
set skala [xfconf xsettings /Gdk/WindowScalingFactor 1]
```

Der Kanal `xsettings` ist für Tk wichtig: Tk 9 liest beim Start unter Xfce genau `/Gdk/WindowScalingFactor` und setzt bei 2 die Oberfläche auf 200 % (Quelltext `scaling.tcl`, siehe Schriften, Skalierung, Themes). Tk erkennt Xfce dabei nicht an XDG_CURRENT_DESKTOP, sondern daran, dass `ps -e` einen Prozess `xfce4-session` zeigt. Auf dem Arbeitsplatz findet `ps -e` ihn aus `wish9.0` und `wish9.1`. `WindowScalingFactor` ist dort 1, also bleibt alles beim Wert aus der Anzeige: `tk scaling` 1,332, `scalingPct` 100, unter 9.0 `::tk::fontScalingFactor` 1.

12.4 xfconf: Einstellungen schreiben

```
exec xfconf-query -c serie-xfce -p /howto/demo -n -t string -s ok
exec xfconf-query -c serie-xfce -p /howto/demo
exec xfconf-query -c serie-xfce -p /howto -r -R
```

`-n` legt eine neue Eigenschaft an, `-t` nennt ihren Typ, `-r -R` löscht rekursiv. Die Serie (Howto 03) schreibt nur in einen eigenen Testkanal und räumt ihn danach weg. Einstellungen des

Benutzers in `xfwm4` oder im Panel schreibt ein Programm nur, wenn das seine Aufgabe ist.

Ein eigener Kanal ist ein bequemer Ort für die Einstellungen eines Xfce-Programms, aber er bindet das Programm an Xfce. Für ein Programm, das auch unter anderen Desktops läuft, ist eine Datei unter `XDG_CONFIG_HOME` (siehe Pfade, Umgebung, XDG) die bessere Wahl.

12.5 Bevorzugte Programme: exo-open

`xdg-open` fragt unter Xfce `exo-open`. Direkt aufgerufen startet `exo-open` die in Xfce eingestellten Programme nach Rolle (`serie-xfce`, Howto 05 und 09):

```
exec {*}[auto_execok exo-open] --launch FileManager &
exec {*}[auto_execok exo-open] --launch TerminalEmulator &
exec {*}[auto_execok exo-open] --launch WebBrowser https://www.tcl-lang.org &
```

Unter Xfce 4.20: Dateimanager ist Thunar 4.20, `xdg-mime query default inode/directory` liefert `thunar.desktop`. Textdateien öffnet `org.xfce.mousepad.desktop`. Das Terminal ist `xfce4-terminal` 1.2.0.

Ein Ordner im Dateimanager:

```
exec {*}[auto_execok xdg-open] $ordner &
```

Thunar direkt: `exec thunar $ordner &`. Die Serie öffnet unter `SERIES_AUTO` kein Fenster und prüft nur, ob die Werkzeuge da sind.

12.6 Benachrichtigungen

Den Dienst stellt `xfce4-notifyd`. Er liegt auf dem Arbeitsplatz unter `/usr/lib/x86_64-linux-gnu/xfce4/notifyd/`, also nicht im `PATH`. `auto_execok xfce4-notifyd` findet ihn nicht; ob der Dienst läuft, fragt man über `notify-send` selbst oder über DBus. Der Aufruf von `notify-send` und das Verhalten von `tk sysnotify` stehen in Native Dialoge und Desktop-Integration.

12.7 Fenstermanager, Panel, Sitzung

`serie-xfce` (Howto 10 und 11):

```
exec xfwm4 --version
exec xfce4-panel --version
exec xfce4-session --version
```

xfwm4 ist der Fenstermanager der Sitzung. Er reparentiert: Die Titelleiste ist auf dem Arbeitsplatz 29 Pixel hoch (`serie-xvfb`, Howto 04), gemessen als `winfo rooty . - gy`. Mit einem anderen Theme ändert sich der Wert. Ein Tk-Programm unter Xfce bekommt Fokus, Iconify und Vollbild vom Fenstermanager; unter Xvfb ohne Fenstermanager fehlt das alles, siehe X11, Fenstermanager und Wayland.

Das Panel-Layout liegt im Kanal `xfce4-panel`. Die Serie liest dort nicht hinein.

12.8 Zusammenfassung

- Xfce erkennen: `XFCE` in `XDG_CURRENT_DESKTOP`, als Liste mit `:` lesen.

- Einstellungen: `xfconf-query -c Kanal -p Pfad`. Schreiben nur in eigene Kanäle.
- Tk 9 liest unter Xfce `xsettings /Gdk/WindowScalingFactor`.
- Programme nach Rolle: `exo-open --launch`. Ordner: Thunar.
- Alles hängt am Sitzungs-DBus. Ohne ihn scheitert jeder Aufruf.

Nächstes Kapitel: GUI-Tests mit Xvfb

13 GUI-Tests mit Xvfb

13.1 Einführung

Tk-Tests brauchen eine Anzeige, auch auf einem Server ohne Bildschirm oder in einer CI-Pipeline. Unter Linux liefert sie Xvfb, ein X-Server, der nur in den Speicher zeichnet. Xvfb hat keinen Fenstermanager. Was das für Fokus, Rahmen und Iconify bedeutet, zeigt das nächste Kapitel. Dieses Kapitel richtet die Anzeige ein: Xvfb und `xvfb-run`, Bildschirmgröße und DPI, Schriften, und den Rückgabewert, auf den sich eine Pipeline verlassen kann.

Grundlage ist `serie-xvfb`, geprüft mit Tcl/Tk 8.6.14 und 9.0.4 unter Xvfb ohne Fenstermanager und mit openbox. Im Container liefen alle 20 Howtos mit beiden Fassungen: 19 OK, 1 SKIP (Howto 10, Iconify, braucht einen Fenstermanager), 0 FAIL.

13.2 Xvfb und xvfb-run

Debian-Paket `xvfb`. Es bringt den Server `Xvfb` und das Hilfsskript xvfb-run (`serie-xvfb`, Howto 17). Fehlt es, meldet eine Testsuite SKIP, nicht FAIL:

```
if {[auto_execok xvfb-run] eq ""} {
    puts "SKIP: xvfb-run fehlt"
    exit 0
}
```

`xvfb-run` startet einen Server, setzt `DISPLAY`, startet das Programm und beendet den Server danach (`serie-xvfb`, Howto 18):

```
xvfb-run -a wish9.0 test.tcl
xvfb-run -a -s "-screen 0 800x600x16 -dpi 50" wish9.0 test.tcl
xvfb-run -a sh -c 'openbox >/dev/null 2>&1 & sleep 1; wish9.0 test.tcl'
```

- `-a` sucht ein freies Display, beginnend bei `:99`.
- `-s "…"` reicht Optionen an Xvfb: `-screen 0` BxHxT (Breite, Höhe, Farbtiefe) und `-dpi N`.
- Ohne `-s` nahm das `xvfb-run` des Messrechners `-screen 0 1280x1024x24`. Ältere Fassungen nahmen `640x480x8`. Deshalb immer angeben.
- Ein Fenstermanager startet im selben `sh -c` vorab. `sleep 1` gibt ihm Zeit, sich anzumelden. openbox ist das Debian-Paket `openbox`.

Ein Skript, das nicht selbst endet, hält `xvfb-run` offen. Testskripte enden mit `exit`; die Serie erzwingt das über `SERIES_AUTO=1`.

`beispiele/gui-test-xvfb.sh` fasst das zusammen: feste Größe, feste DPI, Fenstermanager über `WM=openbox`, SKIP ohne `xvfb-run`.

13.2.1 Eigener Server statt xvfb-run

Für viele kurze Läufe lohnt ein dauerhafter Server:

```
Xvfb :93 -screen 0 1280x1024x24 -nolisten tcp -noreset &
export DISPLAY=:93
```

Die Option -noreset ist wichtig. Ohne sie setzt Xvfb seinen Zustand zurück, sobald der letzte Client die Verbindung schließt. Im Container ging so jede mit `xrdb` geladene Ressource verloren, bevor der Test startete; `xprop -root` zeigte eine eben gesetzte Eigenschaft schon nicht mehr. Mit `-noreset` blieben sie stehen.

13.3 Bildschirm, DPI und Schriften

`winfo screenwidth`, `screenheight` und `screendepth` liefern genau die Werte aus `-screen` (`serie-xvfb`, Howto 01). `tk scaling` folgt `-dpi`: 1,389 ohne Angabe (Xvfb meldet dann 100 dpi), 0,695 bei `-dpi 50`, 2,770 bei `-dpi 200`.

Die Schriften kommen über fontconfig vom Rechner, auf dem der Test läuft, nicht von Xvfb. Ihre Größe ist in Punkten angegeben, die Pixelgröße wächst also mit `-dpi`. Ein Test, der feste Pixel erwartet (Breite eines Labels, Klickpunkt auf einem Reiter), kippt bei anderer DPI oder auf einem Rechner mit anderer Standardschrift. Solche Tests rechnen relativ:

```
set breite [font measure TkDefaultFont "Hallo Welt"]
set soll   [winfo reqwidth .l]
```

Unter Tk 9 wachsen zusätzlich die Pixelmaße von ttk mit der DPI (im Container: Scrollbar 15 Pixel bei 100 dpi, 27 bei 192 dpi). Ein Test, der unter 8.6 grün ist, kann unter 9 an derselben Stelle kippen, siehe Schriften, Skalierung, Themes.

Für reproduzierbare Läufe gehören Größe, Tiefe und DPI deshalb fest in den Testaufruf, und die Schrift fest in den Test:

```
font configure TkDefaultFont -family "DejaVu Sans" -size 10
```

13.4 Rückgabewert und Abbruch

Ein `wish` ohne Anzeige endet mit 0, nachdem es das Skript ohne Tk ausgeführt hat (Container-Prüfung, siehe X11, Fenstermanager und Wayland). Für eine Pipeline ist das gefährlich: Ein falsch gesetztes `DISPLAY` sieht wie ein bestandener Test aus. Testskripte beginnen deshalb mit:

```
if {[catch {package require Tk} fehler]} {
    puts stderr "FAIL: $fehler"
    exit 1
}
```

Auch `tcltest` setzt den Rückgabewert nicht von selbst. `tcltest::runAllTests` liefert in der Fassung aus Tcl 9.0.4 den Wert 1, wenn eine Testdatei fehlschlägt (Quelltext `tcltest.tcl`). `all.tcl` reicht ihn weiter:

```
exit [tcltest::runAllTests]
```

Die Einzelheiten stehen im Buch *Testen*.

Für Hänger hilft ein äußeres `timeout`:

```
timeout 60 xvfb-run -a wish9.0 tests/all.tcl
```

13.5 tclsh oder wish

`wish` lädt Tk beim Start, `tclsh` erst mit `package require Tk`. Danach verhalten sich beide gleich (`serie-xvfb`, Howto 14): `.` existiert, ist aber beim Lauf des Skripts **noch nicht gemappt**, `winfo ismapped .` ist 0. Gemappt wird erst beim nächsten `update` oder wenn die Ereignisschleife läuft.

Unter `tclsh` läuft die Ereignisschleife nicht von selbst. Ein Test unter `tclsh` braucht `update` oder `vwait`, ein Programm unter `tclsh` ein `vwait forever` am Ende. `serie-xvfb` wählt den Interpreter über die erste Zeile des Howtos: Steht `wish` darin, startet `run-all.tcl` das passende `wish` zur Fassung des laufenden `tclsh`.

13.6 Zusammenfassung

- Debian-Paket `xvfb`, `xvfb-run -a -s "-screen 0 BxHxT -dpi N"`.
- Ohne `xvfb-run`: SKIP, nicht FAIL.
- Eigener Xvfb mit `-noreset`, sonst verliert er Ressourcen.
- DPI bestimmt `tk scaling`, Schriften kommen vom Client. Größe, DPI und Schrift im Test festlegen.
- `wish` ohne Anzeige endet mit 0. Tests prüfen `package require Tk` selbst und enden mit einem Rückgabewert.

Nächstes Kapitel: Was der Test selbst herstellt

14 Was der Test selbst herstellt

14.1 Einführung

Auf dem Arbeitsplatz gibt der Fenstermanager einem neuen Fenster den Fokus, hängt es in einen Rahmen und kann es verkleinern. Unter Xvfb ohne Fenstermanager tut das niemand. Ein Test, der sich auf eines davon verlässt, ist auf dem Arbeitsplatz grün und in der Pipeline rot, oder umgekehrt. Die Regel aus `serie-xvfb` (Howto 16): Was die Umgebung beisteuert, darf der Test nicht annehmen. Er stellt es selbst her, mit `update`, `focus -force` und `winfo rooty`. Dann gilt er unter Xvfb ohne Fenstermanager und mit openbox gleich.

Alle Aussagen in diesem Kapitel sind in `serie-xvfb` mit Tcl/Tk 8.6.14 und 9.0.4 geprüft, ohne Fenstermanager und mit openbox. Die Werte ohne Fenstermanager hat die Container-Prüfung mit `beispiele/wm-probe.tcl` bestätigt. Ausnahme ist der Xfce-Fall im Abschnitt *Der Rahmen kommt spät*; er stammt aus Testläufen auf einem Arbeitsplatz.

14.2 Übersicht

	Xvfb ohne Fenstermanager	Xvfb mit openbox
winfo manager .	wm	wm
wm frame . gegen winfo id .	gleich	verschieden

	Xvfb ohne Fenstermanager	Xvfb mit openbox
winfo ismapped, winfo viewable nach update	1	1
focus -force	wirkt	wirkt
Fokus nach update ohne focus -force	leer	.
Titelleiste rooty - gy	0	20
wm iconify	bleibt normal, rund 2 s	iconic
lokaler grab	wirkt	wirkt

14.3 Mappen: erst update

Ein Fenster, das das Skript eben erzeugt hat, ist noch nicht auf dem Bildschirm. Gemappt wird es beim nächsten `update` (`serie-xvfb`, Howto 03 und 14):

```
toplevel .t
winfo ismapped .t        ;# 0
update
winfo ismapped .t        ;# 1, auch ohne Fenstermanager
```

winfo viewable verlangt zusätzlich, dass alle Vorfahren gemappt sind. Nach `wm withdraw .` ist ein Frame in `.` nicht `viewable`, auch wenn er selbst gemappt ist (Howto 15).

14.4 Tasten erzeugen: Fokus herstellen

`event generate` mit einem Tastenereignis geht an das Widget mit dem **Tastaturfokus**, nicht an das Fenster, das man nennt (Howto 07):

```
entry .e1; entry .e2; pack .e1 .e2
update
focus -force .e1
update
event generate .e2 <Key-x>      ;# landet bei .e1
```

Daraus folgen drei Fallen, alle geprüft ohne Fenstermanager und mit openbox:

1. **Vor update:** Ein neues Toplevel ist nicht gemappt und hat keinen Fokus. `event generate .t <Escape>` kommt nicht an (Howto 05). Das ist kein Xvfb-Unterschied, es gilt überall.
2. **Zurückgezogen:** Ein Fenster nach `wm withdraw` bekommt die Taste nicht, auch nicht nach `focus -force .`. Erst `wm deiconify`, `update` und `focus -force` (Howto 06).
3. **Ohne Fenstermanager:** Nach `update` ist `focus` leer. Niemand vergibt den Fokus. Eine erzeugte Taste kommt erst nach `focus -force` an (Howto 07 und 20). Mit openbox hat `.` nach `update` meist schon den Fokus. Der Test wartet trotzdem nicht darauf.

Das Muster für jeden Tastentest steht in der Serie als `serie::fokusVorbereiten`:

```
proc fokusVorbereiten {{w .}} {
    update
    focus -force $w
    update
```

```
}
fokusVorbereiten .
event generate . <Control-s>
```

Container-Prüfung mit `beispiele/wm-probe.tcl`: Taste ohne `focus -force` 0-mal angekommen, mit `focus -force` 1-mal, unter 8.6.14 und 9.0.4.

14.5 Grab

Ein lokaler grab ist Tk-intern und braucht keinen Fenstermanager (Howto 11). Die Serie macht das Fenster vorher sichtbar:

```
toplevel .dlg
update
grab .dlg
grab current            ;# .dlg
```

Ein Fehler kommt ohne `update` nicht. Container-Prüfung, 8.6.14 und 9.0.4: `grab .d` auf ein frisch erzeugtes, noch nicht sichtbares Toplevel kehrt ohne Fehler zurück, `grab current` ist `.d`, `grab status .d` ist `local`; dasselbe nach `wm withdraw .d`. Ein lokaler Grab auf ein unsichtbares Fenster sperrt also still die übrige Oberfläche. Ein globaler Grab (`grab -global`) hängt am X-Server. Auf dem Arbeitsplatz (Xfce, 8.6.18, 9.0.4, 9.1.0) kehrt er ohne Fehler zurück, `grab status` ist `global`, `grab release` hebt ihn auf; die Sitzung blieb bedienbar. Im Test nur mit `grab release` im `finally`.

14.6 Lage von Dialogen

`wm geometry +x+y` setzt die **Rahmenecke** eines Fensters, `winfo rootx/rooty` liefert die Ecke der **Clientfläche** (Howto 04 und 12). Ein Dialog, der direkt unter der Titelleiste des Hauptfensters liegen soll:

```
set x [winfo rootx .]
set y [winfo rooty .]
wm geometry .dlg +$x+$y
```

Der Rahmen des Dialogs beginnt dann an der Oberkante der Clientfläche des Hauptfensters; seine eigene Clientfläche liegt um die eigene Titelleiste tiefer. Ohne Fenstermanager ist `winfo rooty .dlg` gleich `y`, unter openbox `y + 20`.

Eine feste Zahl ist immer falsch irgendwo (Howto 13). Gemessene Titelleisten: ohne Fenstermanager 0, openbox 20, Xfce auf dem Arbeitsplatz des Autors 29. Code wie

```
wm geometry .dlg +[expr {$gx}]+[expr {$gy + 80}]
```

stimmt nur, wenn die Titelleiste zufällig 80 Pixel hat. Die Demo der Serie misst den Rahmen und prüft, dass 80 hier falsch wäre.

`scan [wm geometry.] {%dx%d%[+-]%d%[+-]%d}` liest auch die Form `-X-Y`. Dann zählt X von rechts, und die Rechnung oben gilt nicht. Vorher mit `+X+Y` setzen.

14.6.1 Der Rahmen kommt spät

Wann der Fenstermanager seinen Rahmen anlegt, hängt vom Fenstermanager ab. Unter Xvfb mit openbox ist er bei einem einfachen Toplevel schon da, wenn das erste `update` nach `wm`

`geometry` zurückkehrt; das gilt auch für ein transientes. Ein Probeskript mit einem Dialog bei `+300+250` lieferte mit 8.6.14 und 9.0.4:

Zeitpunkt	ohne Fenstermanager	openbox
nach update	winfo rootx/rooty 300,250	301,270
10 bis 1000 ms danach	unverändert	unverändert
<Configure> mit Versatz	keines	vor <Map>, nach 17 bis 28 ms

Auf einem Xfce-Arbeitsplatz mit wish 9.1.0 kam der Versatz später. Ein Dialog, den der Code nach 100 ms ein einziges Mal korrigiert hatte, lag am Ende 5 Pixel rechts und 28 bis 29 Pixel unter dem Ziel. Das geschah nicht in jedem Lauf: in beiden Läufen der ganzen Suite, beim Einzellauf der Testdatei im dritten von drei. Unter Xvfb, mit und ohne openbox, kam es nie vor.

Für den Code heißt das: Eine Korrektur nach fester Wartezeit reicht nicht. Er beobachtet `<Configure>` am Dialog eine Weile, etwa eine Sekunde nach dem Erscheinen, und korrigiert jedes Mal kurz danach. Dabei rechnet er von der Rahmenecke aus `wm geometry` aus, nicht von `winfo rootx`; sonst schießt die Korrektur ohne Rahmen über das Ziel hinaus.

Für den Test heißt das: Unter Xvfb tritt der Fall nicht von selbst auf. Der Test stellt ihn her und verschiebt den Dialog nach dem Erscheinen selbst, ausgehend von der Rahmenecke:

```
after 300 {
    regexp {\+(-?\d+)\+(-?\d+)$} [wm geometry .d] -> fx fy
    wm geometry .d +[expr {$fx + 5}]+[expr {$fy + 29}]
}
```

Wer hier `winfo rootx` statt der Rahmenecke nimmt, verschiebt unter openbox um den Rahmen zusätzlich, im Probelauf 6 und 49 statt 5 und 29 Pixel. Beim mittigen transienten Dialog aus dem nächsten Abschnitt ergibt auch die Rahmenecke 6 und 49, weil `wm geometry` dort nicht stimmt. Einem Test, der nur prüft, dass die Korrektur greift, schadet das nicht; einer, der den Versatz genau vergleicht, setzt den Dialog vorher nicht genau mittig oder nicht transient.

`beispiele/spaeter-rahmen.tcl` zeigt beides nebeneinander. Mit 8.6.14 und 9.0.4, ohne Fenstermanager und mit openbox, war das Ergebnis gleich, ebenso auf dem Debian-Arbeitsplatz unter Xvfb ohne Fenstermanager mit 8.6.18, 9.0.4 und 9.1.0:

```
einmal     Abstand der Mitten: 5,29
beobachten Abstand der Mitten: 0,0
```

14.6.2 openbox setzt mittige Dialoge selbst

Eine Ausnahme zeigte sich unter openbox bei einem transienten Dialog, den das Programm genau mittig über die Clientfläche des Elternfensters setzt. Elternfenster `.p` mit `wm geometry .p 800x500+500+300`, Dialog 418×92 Pixel, transient zu `.p`; abgelesen 50 ms nach dem Setzen, mit 8.6.14 und 9.0.4:

verlangte Rahmenecke	wm geometry	Rahmen laut xwininfo	Clientfläche
+692+524 (mittig)	+692+524	691,504	692,524
+680+524	+680+524	680,524	681,544
+692+524, nicht transient	+692+524	nicht abgelesen	693,544

Im ersten Fall liegt der Rahmen mittig über dem Rahmen von `.p`, nicht an der verlangten Stelle. Offenbar setzt openbox einen transienten Dialog, der mittig verlangt wird, selbst mittig. Die Clientfläche liegt dann sofort am Ziel. `wm geometry` meldet aber weiter die verlangte Lage, nach 950 ms noch immer; die Rahmenecke daraus ist um 1 und 20 Pixel falsch. Erst nachdem das Programm die Lage selbst wieder setzt, stimmen die Werte wieder.

14.7 Tests, die einen Fenstermanager brauchen

Iconify, echte Rahmenlage und Fokuswechsel durch den Fenstermanager lassen sich ohne ihn nicht prüfen. Solche Tests melden SKIP, wenn keiner läuft. Die Erkennung (Howto 09) steht in X11, Fenstermanager und Wayland: `wm frame` gegen `winfo id`, sonst eine Iconify-Probe.

In der Serie heißen die Wächter `serie::brauchtWM` (irgendein Fenstermanager) und `serie::brauchtRahmen` (ein reparentierender). Die Iconify-Probe kostet ohne Fenstermanager rund zwei Sekunden, weil Tk auf eine Zustandsänderung wartet (Container: 2002 und 2003 ms). Die Serie lässt sich das Ergebnis deshalb über `SERIE_WM` vorgeben:

```
SERIE_WM=0 xvfb-run -a wish9.0 run-all.tcl
```

Für Tests mit Fenstermanager startet `xvfb-run` openbox vorab, siehe GUI-Tests mit Xvfb. Mit xfwm4 unter Xvfb sind die Howtos nicht gelaufen.

Unter Windows und macOS gibt es immer einen Fenstermanager. Dort skippt nichts; die X11-spezifischen Howtos melden SKIP, weil `tk windowingsystem` nicht `x11` ist. Unter Windows sind die Howtos nicht ausprobiert.

14.8 Zusammenfassung

- `update` vor jeder Prüfung von Sichtbarkeit, Lage oder Fokus.
- Erzeugte Tasten gehen an das Fokus-Widget. `focus -force`, dann `update`, dann `event generate`.
- Lokaler `grab` braucht nur ein sichtbares Fenster.
- Dialoglage aus `winfo rooty`, nie aus `wm geometry` plus fester Zahl.
- Ein Rahmen kann nach dem Mappen kommen. `<Configure>` eine Weile beobachten; im Test die späte Verschiebung selbst erzeugen.
- openbox setzt einen mittig verlangten transienten Dialog selbst; `wm geometry` meldet dann nicht die Rahmenecke.
- Iconify und Rahmen nur mit Fenstermanager; ohne ihn SKIP, vorgegeben über `SERIE_WM`.

Nächstes Kapitel: Tk bauen: Abhängigkeiten

15 Tk bauen: Abhängigkeiten

15.1 Einführung

Wer Tcl/Tk 9 unter Linux selbst baut, braucht einen Übersetzer und die X11-Entwicklungspakete. Den Bau selbst, mit allen Schaltern, beschreiben *Tcl/Tk 9.0 bauen* und *Tcl/Tk 9.1 bauen*. Dieses Kapitel prüft nur, welche Bibliotheken ein Linux-Tk tatsächlich braucht und mitbringt, und wo das Debian-Paket von einem Quellbau abweicht. Grundlage ist `serie-tcltk-bauen`, geprüft auf Debian forky (September 2026) mit `tclsh8.6` (8.6.18) und `tclsh9.0` (9.0.4), je 12 von 12

Howtos bestanden. Im Container lief die Serie unter 8.6.14 mit 11 OK und 1 SKIP (Howto 10: Ubuntu hat Itcl für 8.6 nicht installiert), unter 9.0.4 mit 11 OK und 1 FAIL; der FAIL steht unten.

15.2 Was Debian zum Bau von Tk braucht

`apt-cache showsrc tk8.6` nennt als Build-Depends vier X-Entwicklungspakete und `tcl8.6-dev` (Howto 02):

```
libx11-dev libxss-dev libxext-dev libxft-dev tcl8.6-dev
```

Dazu die Werkzeugkette aus `build-essential`. `libxft-dev` zieht fontconfig und freetype nach. Für freetype heißt das Paket heute `libfreetype-dev`; `libfreetype6-dev` ist nur noch ein Übergangspaket.

Ein Quellbau von Tk 9 braucht laut *Tcl/Tk 9.0 bauen* (ausprobiert auf Ubuntu 24.04) zusätzlich libxi-dev: `configure` hing dort an `X11/extensions/XInput2.h`. Debian listet `libxi-dev` für `tk8.6` nicht, und keine gelinkte Tk-Bibliothek braucht `libXi` (siehe unten). Im Quelltext von Tk 9.0.4 kommt `XInput2` nicht vor (Container-Suche über alle Dateien). Wo der Bedarf herkam, ist nicht geklärt. Die Empfehlung der Serie bleibt: Für einen Selbstbau von Tk 9 das Paket hinstellen; es schadet nicht, und `configure` bricht ohne den Kopf nicht spät ab.

15.3 TK_LIBS und TCL_LIBS

Was eine Installation linkt, steht in `tkConfig.sh` und `tclConfig.sh` (Howto 03 und 04). Debian forky:

```
TK_LIBS  (8.6)  -lX11 -lXss -lXext -lXft -lfontconfig -lfreetype … -lz …
TCL_LIBS (8.6)  -ldl -lz -lpthread -lm
TCL_LIBS (9.0)  -ldl -lz -lpthread -lm -ltommath
```

Container-Prüfung:

```
TK_LIBS  (Quellbau 9.0.4)  -lX11 -lXss -lXext -lXft -lfontconfig -lfreetype
    -lfontconfig -ldl -lz -lpthread -lm
TCL_LIBS (Ubuntu 8.6.14)   -ldl -lz -lpthread -lm
TCL_LIBS (Quellbau 9.0.4)  -ldl -lz -lpthread -lm
```

Hier liegt der FAIL im Container: Howto 03 erwartet unter 9.0 `-ltommath`. Das Debian-Paket `tcl9.0` linkt gegen die Systembibliothek libtommath; ein Quellbau übersetzt die mitgelieferte Fassung hinein und braucht kein `-ltommath`. Beides ist richtig, die Demo prüft nur die Debian-Lage.

`zlib1g-dev` ist für Tcl Pflicht. Fehlt es, bleibt ein Selbstbau oft mit einer undeutlichen `configure`-Meldung stehen (Howto 03).

15.4 Was wirklich gelinkt ist

TK_LIBS ist die Absicht, `ldd` an der Bibliothek ist das Ergebnis (Howto 07). Debian forky, `libtk8.6.so` und `libtcl9tk9.0.so`: Xft, Xss, X11, Xext, fontconfig, freetype, Xrender, zlib, kein `libXi`. `libpng16` führt `ldd` bei allen drei Tk-Fassungen auf, aber nur über freetype (siehe unten).

`ldd` zeigt auch, was eine Bibliothek über andere nachzieht. Xrender kommt über Xft, nicht als

eigenes `-lXrender`. Genauer ist die Liste der direkten Abhängigkeiten:

```
readelf -d /opt/tcl9/lib/libtcl9tk9.0.so | grep -E 'NEEDED|RUNPATH'
```

Container-Prüfung:

Bibliothek	direkt gebraucht (NEEDED)	RUNPATH
libtk8.6.so (Ubuntu)	Xft, fontconfig, X11, Xss, m, c	keiner
libtcl9tk9.0.so (Quellbau)	Xft, fontconfig, X11, Xss, m, c	/opt/tcl9/lib
libtcl8.6.so (Ubuntu)	z, m, c	keiner
libtcl9.0.so (Quellbau)	z, m, c	/opt/tcl9/lib

Im Container führt `ldd libpng16` für **beide** Tk-Fassungen auf, als Abhängigkeit von freetype. Auf dem Arbeitsplatz (Debian forky) ebenso: `readelf -d` zeigt bei `libtk8.6.so`, `libtcl9tk9.0.so` und `libtcl9tk9.1.so` kein `libpng` unter `NEEDED`; `libfreetype.so.6` hat es dort direkt. Tk 9.1 braucht zusätzlich direkt HarfBuzz und Xrender.

Den `RUNPATH /opt/tcl9/lib` setzt der Quellbau. Er sorgt dafür, dass `tclsh9.0` seine Bibliothek ohne `LD_LIBRARY_PATH` findet, bindet sie aber an diesen Ort. Das wird beim Weitergeben wichtig, siehe Weitergeben.

15.5 tclConfig.sh unter Debian

Unter Debian und Ubuntu ist `/usr/lib/tcl8.6/tclConfig.sh` ein Wrapper von zwei Zeilen (Howto 09; im Container genauso):

```
#! /bin/sh
. /usr/lib/`dpkg-architecture -qDEB_HOST_MULTIARCH`/tcl8.6/tclConfig.sh
```

Die Werte stehen in `/usr/lib/x86_64-linux-gnu/tcl8.6/tclConfig.sh`. Wer `TCL_LIBS` mit `grep` in der Wrapper-Datei sucht, findet nichts. Ein Quellbau nach `/opt/tcl9` legt `tclConfig.sh` direkt nach `/opt/tcl9/lib`; `configure --with-tcl=` will genau diesen Ordner. Für TEA-Erweiterungen siehe *Tcl-Erweiterungen*.

15.6 Threads und shared

- Tcl 8.6 ist immer mit Threads gebaut: `TCL_THREADS=1`. Tcl 9 hat keinen Schalter mehr; `--enable-threads` erzeugt nur `WARNING: unrecognized options` (Howto 05).
- Ein normaler Linux-Bau ist shared: `TCL_SHARED_BUILD=1`, `libtcl9.0.so`, `info sharedlibextension` ist `.so` (Howto 06; Container: alle drei bestätigt). Statisch (`--disable-shared`) ist die Ausnahme für Zipkits.

15.7 Zwei Fallen beim Bau

Aus `serie-tcltk-bauen`, Howto 11:

- **Leere .o:** Ein abgebrochener `make` hinterlässt Objektdateien mit null Byte und neuem Zeitstempel. `make` übersetzt sie nicht noch einmal, der Linker scheitert später mit `file format not recognized`. Suchen mit `find. -name '*.o' -size 0`.
- **LD_LIBRARY_PATH bei Tcl 9:** Das frisch gebaute `tclsh` im Bauordner findet

`libtcl9.0.so` nicht, solange sie nicht installiert ist. Die Meldung schlägt `TCLSH_NATIVE` vor; gemeint ist der Bibliothekspfad: `env LD_LIBRARY_PATH=$PWD make`.

Die `configure`-Zeilen der Serie (Howto 12):

```
cd tcl9.0.4/unix
./configure --prefix=/opt/tcl90 --enable-shared --enable-64bit
env LD_LIBRARY_PATH=$PWD make
make install

cd ../../tk9.0.4/unix
./configure --prefix=/opt/tcl90 --exec-prefix=/opt/tcl90 \
    --with-tcl=/opt/tcl90/lib --enable-64bit
env LD_LIBRARY_PATH=/opt/tcl90/lib make
make install
```

`configure` läuft im Ordner `unix/`, nicht in der Wurzel des Baums.

15.8 Zusammenfassung

- Debian baut Tk gegen `libx11-dev libxss-dev libxext-dev libxft-dev`. Für einen Tk-9-Quellbau `libxi-dev` dazustellen.
- Gelinkt sind Xft, fontconfig, X11, Xss; über diese Xext, Xrender, freetype, zlib, `libpng`. Kein `libXi`. Tk 9.1 direkt auch HarfBuzz und Xrender.
- `-ltommath` nur beim Debian-Paket `tcl9.0`, nicht beim Quellbau.
- `tclConfig.sh` unter Debian ist ein Wrapper; Werte in der Multiarch-Datei.
- Quellbau setzt RUNPATH auf das Prefix.

Nächstes Kapitel: Weitergeben

16 Weitergeben

16.1 Einführung

Ein Tcl-Programm verlässt den eigenen Rechner auf drei Wegen: als Skript für das Tcl des Zielrechners, als Systempaket (`.deb`, `.rpm`) oder als eine einzige ausführbare Datei mit zipfs. Das Buch *Ausliefern* beschreibt alle drei ausführlich, mit Zipkits, Starkits und Installern. Dieses Kapitel nennt nur, was unter Linux dabei auffällt: welches Tcl der Zielrechner hat, woran ein Image hängt, und was für den Desktop dazugehört. Grundlage für zipfs ist `serie-tcl9-app`, im Container vollständig nachgelaufen (18 von 18 OK, `tclsh9.0` 9.0.4).

16.2 Welches Tcl hat der Zielrechner?

Ein Skript braucht ein passendes Tcl auf dem Zielrechner. Unter Linux ist das meist 8.6 aus dem Paket:

- Debian Stable (trixie): Tcl 8.6.16 und das Paket `tcl9.0` 9.0.1.
- Debian Testing (forky) und unstable: 8.6.18, `tcl9.0` 9.0.4 und `tcl9.1` 9.1.0.
- Ubuntu 24.04: 8.6.14. Im Container kennt `apt-cache policy` kein Paket `tcl9.0`.

Ein Programm, das Tcl 9 verlangt, läuft also auf den meisten Linux-Rechnern nicht mit dem System-Tcl. Es bringt sein Tcl mit, oder es bleibt 8.6-fähig. Für ein `.deb` mit `Depends: tcl9.0`

gilt: Das Paket muss es auf dem Zielsystem geben. *Ausliefern*, Kapitel *Linux: deb und rpm*, rät deshalb, vorher `apt-cache policy tcl9.0 tcl8.6` auf dem Ziel zu prüfen.

Ein Skript, das unter beiden Fassungen laufen soll, beachtet die Unterschiede dieses Buches: Tilde (siehe Pfade, Umgebung, XDG), Kodierung der Skripte (Kodierung und Locale), Mausrad (Maus, Tastatur, Zwischenablage).

16.3 Ein Image mit zipfs mkimg

Tcl 9 kann ein Verzeichnis als ZIP an einen Interpreter hängen. Das Ergebnis ist eine ausführbare Datei, die beim Start ihr Archiv unter `//zipfs:/app` einhängt und `main.tcl` daraus ausführt (`serie-tcl9-app`, Howto 06 und 07):

```
zipfs mkimg app.bin app.vfs app.vfs "" [info nameofexecutable]
file attributes app.bin -permissions 0o755
```

Das vierte Argument `""` ist das Passwort. Es muss stehen, wenn als fünftes die Basis folgt, sonst landet der Pfad im Passwortfeld.

Container-Prüfung mit einem `main.tcl`, das seine Lage druckt, gebaut auf `/opt/tcl9/bin/tclsh9.0`:

```
Größe     16374 Byte
lib       //zipfs:/lib/tcl/tcl_library
mounts    //zipfs:/lib/tcl  /opt/tcl9/lib/libtcl9.0.so
          //zipfs:/app      …/app.bin
script    //zipfs:/app/main.tcl
```

16 KB, weil das Image nur das kleine `tclsh9.0` und das eigene Archiv enthält. Die Tcl-Bibliothek steckt in `libtcl9.0.so` und wird von dort eingehängt. Das Image hängt deshalb an genau dieser Datei:

```
ldd app.bin | grep tcl
#   libtcl9.0.so => /opt/tcl9/lib/libtcl9.0.so
readelf -d /opt/tcl9/bin/tclsh9.0 | grep RUNPATH
#   Library runpath: [/opt/tcl9/lib]
```

Auf einem Rechner ohne `/opt/tcl9/lib/libtcl9.0.so` startet es nicht. Das ist nicht auf einem zweiten Rechner geprüft, folgt aber aus `RUNPATH` und `NEEDED`. Ein solches Image ist zum Üben gut, zum Weitergeben nicht.

`serie-tcl9-app` betont für statische Basen das Gegenteil: Liegt die Tcl-Bibliothek im Archiv der ausführbaren Datei selbst, ersetzt `mkimg` dieses Archiv, und `tcl_library` muss ins eigene `app.vfs` kopiert werden (Howto 15). Beim dynamischen Quellbau unter `/opt/tcl9` liegt sie in `libtcl9.0.so` und bleibt erhalten. Welche Lage vorliegt, zeigt `zipfs mount` in der Basis.

Für GUI-Programme ist die Basis `wish9.0`, nicht `tclsh9.0` (Howto 09). Das Image hängt dann zusätzlich an `libtcl9tk9.0.so` und an den X-Bibliotheken des Zielrechners (siehe Tk bauen: Abhängigkeiten).

16.4 Zipkit: statische Basis

Zum Weitergeben braucht das Image eine Basis, die nicht auf `/opt/tcl9` zeigt: ein statisch

gebautes `tclsh` oder `wish` mit angehängter Tcl-Bibliothek, ein Zipkit (`serie-tcl9-app`, Howto 13). Wie man es baut oder bezieht (BAWT, eigener Bau mit `--disable-shared`), steht in *Ausliefern*, Kapitel *Zipkits bauen und füllen*, und in *Tcl/Tk 9.0 bauen*, Kapitel *Statisch bauen*.

Auch ein statisches Tk hängt an den X-Bibliotheken des Systems (`libX11`, `libXft`, `libfontconfig`). Die gibt es auf jedem Linux-Desktop, aber nicht auf jedem Server. Und es hängt an der C-Bibliothek: Der Quellbau im Container verlangt Symbole bis `GLIBC_2.38` (`objdump -T`), Ubuntu 24.04 hat 2.39. Auf einem Ziel mit älterer glibc startet er nicht. Wer für viele Distributionen baut, baut auf der ältesten. Das ist nicht geprüft.

Weil ein Image nur Basis plus ZIP ist, lässt sich derselbe ZIP-Inhalt mit einer anderen Basis für eine andere Plattform zusammensetzen, solange er keine `.so` enthält (Howto 16 und 17):

```
cat zipkit-linux-x86_64 app.zip > app
chmod +x app
```

16.5 Desktop: Starter und Symbol

Ein Programm für den Desktop braucht neben der ausführbaren Datei:

- eine `.desktop`-Datei in `~/.local/share/applications/` oder `/usr/share/applications/`,
- ein Symbol im Symbolthema, etwa `/usr/share/icons/hicolor/48x48/apps/meineapp.png`,
- eine Fensterklasse, die zu `StartupWMClass` passt.

Alle drei beschreibt Native Dialoge und Desktop-Integration. Bei einem Image nimmt Tk den Namen nicht von der Datei, sondern vom Startskript im Archiv. Container-Prüfung, Image `meineapp.bin` auf Basis `wish9.0`: `tk appname` ist `main.tcl`, die Klasse `Main.tcl`. Jedes Image mit `main.tcl` heißt also für den Fenstermanager gleich. Mit `./meineapp.bin -name meineapp` wird daraus `Meineapp`. Das `-name` gehört deshalb in die `Exec`-Zeile. Bei einer `wish`-Basis ist Tk schon geladen, wenn `main.tcl` läuft; dort hilft nur die Befehlszeile. Bei einer `tclsh`-Basis kann `main.tcl` es vor `package require Tk` in `::argv` setzen (siehe X11, Fenstermanager und Wayland).

16.6 Zusammenfassung

- Ziel-Tcl ist meist 8.6. Tcl 9 gibt es als Paket in Debian Stable (`tcl9.0` 9.0.1), Testing und unstable (`tcl9.0` 9.0.4, `tcl9.1` 9.1.0); Ubuntu 24.04 hat es nicht.
- `zipfs mkimg` auf einem dynamischen `tclsh9.0` ergibt ein kleines Image, das an `/opt/tcl9/lib/libtcl9.0.so` hängt. Zum Weitergeben eine statische Basis (Zipkit).
- Auch statisch bleiben X-Bibliotheken und glibc Sache des Zielrechners.
- Zum Desktop gehören `.desktop`, Symbol und passende Fensterklasse.
- Einzelheiten: *Ausliefern*.

Nächstes Kapitel: FAQ

17 FAQ

Kurze Antworten mit Verweis auf das Kapitel, das sie begründet.

17.1 Plattform und Interpreter

Wie erkenne ich Linux im Code?

`$tcl_platform(os) eq "Linux"`. `platform` ist auch auf macOS `unix`. Siehe tcl_platform und Distribution.

Wie erkenne ich Debian oder Ubuntu?

`/etc/os-release` lesen, `ID` und `ID_LIKE` prüfen. `/etc/debian_version` gibt es auch auf Ubuntu.

tclsh9.0 findet Tcllib nicht, tclsh8.6 schon. Warum?

Ein Tcl 9 unter `/opt` hat `/usr/share/tcltk` nicht in `auto_path`. Reine Tcl-Pakete gezielt einhängen, nicht über `TCLLIBPATH` für alle Fassungen. Siehe Suchpfade.

wish startet Tk 8.6, obwohl /opt/tcl9/bin vorn im PATH steht.

Ein Quellbau legt `wish9.0` an, aber keinen Link `wish`. Mit Versionsnummer aufrufen. Siehe Welches Tcl?.

file normalize ~ liefert unter Tcl 9 einen Pfad im aktuellen Ordner.

Tcl 9 löst die Tilde nicht mehr auf. `$env(HOME)` oder `file home` nehmen. Siehe Pfade, Umgebung, XDG.

Wohin mit den Einstellungen meines Programms?

`$XDG_CONFIG_HOME/<programm>/`, sonst `~/.config/<programm>/`.

17.2 Kodierung

Aus Cron oder systemd kommen Umlaute als Ã¼.

Dort fehlt die Locale, `encoding system` ist iso8859-1. Tcl 8.6 liest dann auch das Skript falsch. Dateien mit `fconfigure -encoding utf-8` öffnen, für 8.6 die Wache an den Skriptanfang. Siehe Kodierung und Locale.

Tcl 9 bricht beim Lesen einer alten Datei ab.

Die Datei ist nicht UTF-8. Tcl 8.6 hat das verdeckt. Die richtige Kodierung angeben, oder mit `-profile replace` lesen.

17.3 Prozesse

Wie öffne ich eine Datei oder Adresse mit dem Standardprogramm?

`exec {*}[auto_execok xdg-open] $ziel &`. Unter Xfce geht das an `exo-open`. Siehe exec, xdg-open, Prozesse.

exec meldet einen Fehler, obwohl das Programm gelaufen ist.

Es hat auf stderr geschrieben. `errorCode` ist dann `NONE`. `-ignorestderr` oder `2>@1`.

Mein Programm hängt beim Lesen der Zwischenablage mit xclip.

Es ist selbst Besitzer der Auswahl und blockiert in `exec`. Über einen Kanal mit `fileevent` lesen, oder Tk direkt fragen: `selection get`.

17.4 Oberfläche

Läuft Tk unter Wayland?

Über XWayland. `tk windowingsystem` bleibt `x11`. Unter Weston sind Fensterpositionen

dort weder setzbar noch lesbar; eine echte Wayland-Sitzung ist nicht geprüft. Siehe X11, Fenstermanager und Wayland.

Unter Tk 9 scrollt mein Canvas nicht mehr mit dem Rad, aber mit den Seitentasten der Maus.

Tk 9 meldet das Rad als `<MouseWheel>`. `<Button-4>` und `<Button-5>` sind dort die Tasten 8 und 9. Siehe Maus, Tastatur, Zwischenablage.

<Command-c> aus meinem Mac-Code tut unter Linux nichts.

Unter Tk 9 ist `Command` unter X11 `Meta`; im Container und auf dem Xfce-Arbeitsplatz war dafür keine Taste belegt. Unter Tk 8.6 war es Alt. Unter `x11` `Control` binden.

Die Zwischenablage ist leer, nachdem mein Programm beendet ist.

Unter X11 hält das Programm den Inhalt selbst. Ohne Zwischenablage-Verwalter ist er mit dem Programm weg.

Auf dem 4K-Bildschirm ist alles winzig.

`tk scaling` folgt der DPI des X-Servers. Tk 9 liest zusätzlich die Skalierung von Xfce, MATE, GNOME und `Xft.dpi`. Unter 8.6 von Hand `tk scaling` setzen. Siehe Schriften, Skalierung, Themes.

Mein Programm hat auf einem Rechner plötzlich gelbe Knöpfe.

X-Ressourcen. `xrdb -query` und `~/.Xdefaults` prüfen.

Warum sieht der Dateidialog nicht wie in GTK-Programmen aus?

Unter X11 zeichnet Tk ihn selbst. Siehe Native Dialoge und Desktop-Integration.

Die Leiste zeigt für mein Programm ein falsches oder gar kein Symbol.

`StartupWMClass` in der `.desktop`-Datei passt nicht zur Klasse. `wish9.0 meineapp.tcl` hat die Klasse `Meineapp.tcl`, ein Image mit `main.tcl` die Klasse `Main.tcl`. Mit `-name` festlegen.

17.5 Tests

Die CI meldet Erfolg, aber kein Test lief.

`wish` ohne Anzeige endet mit 0. Am Testanfang `package require Tk` prüfen und `exit 1`. Siehe GUI-Tests mit Xvfb.

Unter Xvfb kommt event generate nicht an.

Ohne Fenstermanager vergibt niemand den Fokus. `update`, `focus -force`, `update`. Siehe Was der Test selbst herstellt.

wm iconify dauert zwei Sekunden und ändert nichts.

Kein Fenstermanager. Tk wartet vergeblich auf die Zustandsänderung.

17.6 Bauen und Weitergeben

configure von Tk findet XInput2.h nicht.

`libxi-dev` installieren. Siehe Tk bauen: Abhängigkeiten.

Mein zipfs mkimg-Programm startet auf einem anderen Rechner nicht.

Die Basis war ein dynamisches `tclsh9.0` und hängt an `/opt/tcl9/lib/libtcl9.0.so`. Eine statische Basis (Zipkit) nehmen. Siehe Weitergeben.

Nächstes Kapitel: Fallen

18 Fallen

Kurze Liste. Jede Zeile beschreibt, was man **nicht** tun sollte. Die Begründung steht im genannten Kapitel.

18.1 Plattform und Interpreter

- Linux mit `tcl_platform(platform) eq "unix"` erkennen. Das trifft auch macOS. `os` nehmen.
- Die Distribution aus `tcl_platform(osVersion)` lesen. Das ist der Kernel, im Container sogar der des Wirts.
- Debian an `/etc/debian_version` erkennen. Ubuntu hat die Datei auch (Container: `trixie/sid`).
- `tcl_platform(threaded)` ohne `info exists` lesen. Unter Tcl 9 fehlt der Schlüssel.
- `$env(USER)` für den Benutzer halten. Er kann fehlen. `tcl_platform(user)` ist verlässlich.
- `wish` ohne Versionsnummer aufrufen und Tk 9 erwarten. Ein Quellbau legt keinen Link `wish` an.
- Ein eigenes Tcl nach `/usr` installieren. `/opt` nehmen.
- Rechte aus `file attributes -permissions` als Zeichenkette vergleichen. 8.6 liefert `041777`, 9.0 `0o41777`.
- `[lindex [file volumes] 0]` für `/` halten. Unter Tcl 9 steht `//zipfs:/` davor.
- Unter Debian `tcl9.1` ohne `tk9.1` installieren und Tk 9.1 erwarten. `package require Tk` lädt dann Debians Tk 9.0.
- Ein Tcl unter `/opt` für getrennt halten, ohne `ldd` zu prüfen. Es kann Debians `libtcl` gleichen Namens laden.
- Ein Alias als Startdatei nehmen. In Skripten und Makefiles gilt er nicht.

18.2 Pfade und Suchpfade

- `file normalize ~` unter Tcl 9. Das ergibt `./~`. `$env(HOME)` oder `file home`.
- `~/Documents` fest eintragen. Der Ordner ist lokalisiert; `xdg-user-dir DOCUMENTS` fragen.
- Einstellungen als Punktdatei direkt in `$HOME` ablegen. XDG: `~/.config/<programm>/`.
- TCLLIBPATH mit Doppelpunkten trennen. Es ist eine Tcl-Liste.
- TCL9_0_TM_PATH mit Leerzeichen trennen. Hier gilt der Doppelpunkt.
- TCLLIBPATH für alle Fassungen auf einen Ordner mit 9er-Paketen setzen. `tclsh8.6` sieht ihn auch.
- Erwarten, dass ein Tcl 9 unter `/opt` die Pakete aus `/usr/share/tcltk` sieht.
- Eigene Pakete für das System-Tcl nach `/usr/share/tcltk` legen. Das gehört `apt`; `/usr/local/share/tcltk` nehmen.
- `libsqlite3-tcl` bis 3.53.4-2 unter Debians Tcl 9 erwarten. Es ist nur für 8.6 gebaut; erst 3.54.1-1 (unstable) bringt Tcl 9 mit.
- Ein zweites sqlite3 mit niedrigerer Nummer daneben legen. Die höchste Nummer gewinnt, der Ordner zählt nur bei gleicher.
- Ein Paket als `.tm` und daneben als `pkgIndex.tcl`. Welche Kopie lädt, hängt davon ab, was vorher geladen wurde; siehe *Welche Kopie gewinnt* im Kapitel *Suchpfade*.
- Ein eigenes sqlite3 nur für Tcl 9 vor Debians Ordner legen. Unter 8.6 verdrängt der Index dann Debians Fassung.
- `package ifneeded` für den Beweis halten, welche Bibliothek geladen ist. `info loaded`

nehmen.
- Einen Paketentwurf mit `dpkg -i` testen. Auspacken mit `dpkg-deb -x` und Laden über `auto_path` reicht und ändert nichts unter `/usr`.
- Beim Test eines ausgepackten `libsqlite3-tcl` das passende `libsqlite3-0` vergessen. Die dynamisch gelinkte Anbindung braucht dieselbe SQLite-Fassung.

18.3 Kodierung

- Annehmen, Linux sei immer UTF-8. Ohne Locale ist `encoding system` iso8859-1.
- Eine Datei ohne `fconfigure -encoding` öffnen.
- Ein UTF-8-Skript mit Umlauten unter Tcl 8.6 aus Cron starten, ohne Wache oder `-encoding utf-8`.
- Sich darauf verlassen, dass `LANG=de_DE` unter 8.6 und 9 gleich wirkt. 8.6: iso8859-1, 9.0: utf-8.
- `LANGUAGE` für `msgcat` setzen. `msgcat` liest `LC_ALL`, `LC_MESSAGES`, `LANG`.

18.4 Prozesse

- `exec` ohne `auto_execok` auf ein Programm, das fehlen kann.
- Jeden `exec`-Fehler als Rückgabewert ungleich 0 deuten. Auch eine Zeile auf stderr ist ein Fehler.
- `xdg-open` ohne `&` aufrufen.
- Mit blockierendem `exec` ein Programm rufen, das die eigene X-Auswahl liest. Beide warten aufeinander.
- `auto_reset` aufrufen, nur um `auto_execok` neu suchen zu lassen. Es entfernt auch nachgeladene Befehle.

18.5 X11 und Fenstermanager

- Sich auf den Rückgabewert von `wish` verlassen. Ohne Anzeige endet es mit 0.
- Einen Fenstermanager mit `winfo manager .` erkennen. Das ist immer `wm`.
- `wm frame .` und `winfo id .` als Zeichenketten vergleichen. 8.6 schreibt führende Nullen.
- Einen Dialog mit `-geometry +0+80` unter das Hauptfenster setzen. Rahmen sind 0, 20 oder 29 Pixel hoch. `winfo rootx .`.
- `wm geometry` für die Lage der Clientfläche halten. Es ist die Rahmenecke.
- Die Lage eines Dialogs einmal nach fester Wartezeit korrigieren. Manche Fenstermanager legen den Rahmen später an.
- Unter openbox `wm geometry` eines mittig gesetzten transienten Dialogs für die Rahmenecke halten. openbox setzt ihn selbst; Tk meldet die verlangte Lage.
- `-fullscreen`, `-zoomed`, `iconify` ohne Fenstermanager erwarten.
- `StartupWMClass=meineapp` schreiben und `wish meineapp.tcl` starten. Die Klasse ist `Meineapp.tcl`.
- Unter Wayland `tk windowingsystem` gleich `wayland` erwarten. Es ist `x11`.

18.6 Schrift, Skalierung, Themes

- Eine Schriftfamilie voraussetzen. fontconfig ersetzt still.
- Pixelmaße in Tests fest verdrahten. Sie hängen an DPI, Schrift und unter Tk 9 an der Skalierung.
- Unter Tk 8.6 annehmen, `Xft.dpi` wirke auf `tk scaling`. Es wirkt nur auf die Schrift.
- Ein natives Linux-Theme erwarten. Es gibt `clam alt default classic`.
- Xvfb ohne `-noreset` starten und mit `xrdb` Ressourcen laden. Der Server vergisst sie, sobald

der letzte Client geht.

18.7 Maus, Tastatur, Zwischenablage

- Unter Tk 9 das Rad über `<Button-4>`/`<Button-5>` binden. Das sind dort die Seitentasten 8 und 9.
- Unter Tk 8.6 auf X11 nur `<MouseWheel>` binden. Das Rad kommt dort als `<Button-4>`/`<Button-5>`.
- `<Command-c>` unter X11 binden. 8.6: Alt, 9.0 und 9.1: keine Taste (Container und Xfce).
- `<Super-c>` binden. Der Name ist `Mod4`.
- Markieren für Kopieren halten. Markieren setzt PRIMARY, Kopieren CLIPBOARD.
- Erwarten, dass die Zwischenablage das Programmende überlebt.
- `selection get -selection PRIMARY` ohne `catch`.

18.8 Desktop

- Den GTK-Dateidialog erwarten. Tk zeichnet seinen eigenen, und der zeigt versteckte Dateien.
- In `Exec=` einer `.desktop`-Datei auf den `PATH` der Shell bauen.
- `zenity` ohne `-ignorestderr` aufrufen. Die GTK-Warnung auf stderr wird ein Fehler, auch wenn eine Datei gewählt wurde.
- `tk systray exists` für "Symbol sichtbar" halten.
- `xfce4-clipman` installiert für "Zwischenablage bleibt" halten. Es muss laufen.
- `tk sysnotify` für "Meldung angezeigt" halten. Ohne DBus schweigt es.

18.9 Tests

- Einen Tastentest ohne `update` und `focus -force`.
- `event generate .w <Key>` an ein Widget ohne Fokus schicken und die Bindung von `.w` erwarten. Die Taste geht ans Fokus-Widget.
- Ohne `xvfb-run` FAIL melden. SKIP ist richtig.
- `xvfb-run` ohne `-s` aufrufen und die Bildschirmgröße einer anderen Fassung erwarten.
- `exit [tcltest::runAllTests]` vergessen. Sonst endet `all.tcl` mit 0.

18.10 Bauen und Weitergeben

- `libxi-dev` beim Tk-9-Quellbau weglassen.
- `--enable-threads` an Tcl 9 übergeben. Unbekannter Schalter.
- TCL_LIBS in `/usr/lib/tcl8.6/tclConfig.sh` suchen. Das ist ein Wrapper.
- Ein `zipfs mkimg` auf einem dynamischen `tclsh9.0` weitergeben. Es hängt an `/opt/tcl9/lib`.
- Ein Image auf einer neueren glibc bauen als das Ziel hat.
- Das Passwortargument von `zipfs mkimg` weglassen, wenn die Basis folgt.
- Ein TEA-Paket gegen Debians Tcl mit `make install-binaries` installieren. Es will nach `/usr/bin`; `install-lib-binaries` nehmen.
- Eine Erweiterung für Tcl 9 gegen 9.1 bauen und unter 9.0 erwarten. Gegen 9.0 bauen.

Nächstes Kapitel: CLI

19 CLI

Kurzreferenz der Befehle dieses Buches und der Beispielskripte. Die Ergebnisse stammen aus

der Container-Prüfung am 2026-10-20 (Ubuntu 24.04, Tcl/Tk 8.6.14 und 9.0.4, Xvfb ohne Fenstermanager), wo nicht anders angegeben.

19.1 Zur Laufzeit

```
array get tcl_platform
info patchlevel
info nameofexecutable
puts $tcl_library
puts $auto_path
tcl::tm::path list
tcl::pkgconfig get libdir,runtime
zipfs mount                         ;# nur Tcl 9
encoding system
tk windowingsystem                  ;# x11
winfo server .
tk scaling
tk::pkgconfig get fontsystem        ;# xft
ttk::style theme names              ;# clam alt default classic
```

19.2 Shell

```
type -a tclsh tclsh8.6 tclsh9.0 wish9.0
dpkg -S /usr/bin/tclsh8.6
cat /etc/os-release
locale -a
echo "$XDG_SESSION_TYPE $DISPLAY $WAYLAND_DISPLAY $XDG_CURRENT_DESKTOP"
xdg-user-dir DOCUMENTS
xdg-mime query default inode/directory
fc-match sans-serif
xrdb -query
xmodmap -pm
xprop WM_CLASS                      # danach ins Fenster klicken
readelf -d /opt/tcl9/lib/libtcl9tk9.0.so | grep -E 'NEEDED|RUNPATH'
xvfb-run -a -s "-screen 0 1280x1024x24 -dpi 96" wish9.0 test.tcl
xfconf-query -c xsettings -p /Gdk/WindowScalingFactor
```

19.3 Prüfskript

`beispiele/messprobe-linux.tcl` druckt alle Werte, die die Tabellen dieses Buches verwenden: Plattform und `os-release`, Interpreter und Suchpfade, Locale, Tilde und `file volumes`, XDG-Variablen, `exec`-Fehlerfälle, Tk mit Bildschirm, Skalierung und Mausrad-Bindungen, Schriften, Themes und X-Ressourcen, Fensterattribute, `tk systray`, `tk sysnotify`, `tk print` und die Zwischenablage. Jede Abfrage läuft für sich; fehlt ein Werkzeug, steht die Fehlermeldung in der Zeile.

```
tclsh8.6 messprobe-linux.tcl            # nur Tcl-Teil
env -i HOME=$HOME PATH=/usr/bin:/bin DISPLAY=$DISPLAY LANG=C.UTF-8 \
    /opt/tcl9/bin/wish9.0 messprobe-linux.tcl
wish9.0 messprobe-linux.tcl -keys       # danach Fenster für Tasten
```

Unterschiede zwischen 8.6.14 und 9.0.4 im Container (gleiche Werte weggelassen):

```
                       8.6.14                          9.0.4
threaded               1                               (Schlüssel fehlt)
tcl_library            /usr/share/tcltk/tcl8.6         //zipfs:/lib/tcl/tcl_library
tk_library             /usr/share/tcltk/tk8.6          //zipfs:/lib/tk/tk_library
zipfs mount            (kein zipfs)                    //zipfs:/lib/tk
    …/libtcl9tk9.0.so
                                                       //zipfs:/lib/tcl
    …/libtcl9.0.so
file home              Fehler                          /root
file normalize ~       /root                           <Arbeitsordner>/~
file volumes           /                               //zipfs:/ /
file tempdir           Fehler                          /tmp/tcl_…
Rechte /tmp            041777                          0o41777
tk::scalingPct         fehlt                           100
tk Unterbefehle        ohne print/sysnotify/systray    mit print, sysnotify, systray
Mausrad Text           Button-4/5, MouseWheel          nur MouseWheel
TButton -padding       3 3                             2.25p
<<Copy>>               ohne XF86Copy                   mit <Key-XF86Copy>
systray                fehlt                           angelegt, exists=1
sysnotify              fehlt                           libnotify geladen
tk::print::cups        fehlt                           Tcl-Ensemble (lpstat/lpr)
```

Gleiche Werte in beiden: `os` Linux, `machine` x86_64, `encoding system` utf-8 unter `C.UTF-8`, `windowingsystem` x11, Bildschirm 1280x1024 px, 325x260 mm, `tk scaling` 1,389, `TkDefaultFont` Inter 10, Themes `clam alt default classic`, Hintergrund #d9d9d9, Klasse `Messprobe-linux.tcl`.

19.4 Weitere Beispielskripte

Datei	Zweck	Kapitel
xdg-ordner.tcl	XDG-Ordner und Benutzerordner, 8.6 und 9	Pfade, Umgebung, XDG
utf8-wache.tcl	UTF-8-Skript, das unter LANG=C mit 8.6 stimmt	Kodierung und Locale
rad-und-tasten.tcl	Mausrad und Modifier mit xdotool auslösen und auslesen	Maus, Tastatur, Zwischenablage
auswahl-probe.tcl	PRIMARY und CLIPBOARD, ohne Verklemmung	Maus, Tastatur, Zwischenablage
wm-probe.tcl	Fokus, Rahmen, Iconify, Vollbild, WM_CLASS	X11, Fenstermanager; Tests
spaeter-rahmen.tcl	Dialoglage bei spätem Rahmen, einmal korrigiert gegen beobachtet	Tests
meinapp.desktop	Vorlage für den Starter	Native Dialoge und Desktop-Integration

Datei	Zweck	Kapitel
gui-test-xvfb.sh	Test unter xvfb-run, optional mit openbox	GUI-Tests mit Xvfb

Alle `.tcl`-Skripte enden selbst und laufen damit auch unter `xvfb-run`. `rad-und-tasten.tcl` braucht `xdotool` und meldet ohne es SKIP.

19.5 Howto-Serien

Drei der Serien sind veröffentlicht (Kapitel 1). Festgehalten ist hier, wie alle fünf im Container liefen und mit welchem Ergebnis:

```
cd serie-linux && tclsh8.6 run-all.tcl
cd serie-xvfb  && xvfb-run -a tclsh9.0 run-all.tcl
cd serie-xvfb  && xvfb-run -a sh -c 'openbox >/dev/null 2>&1 & sleep 1;
    ./run-all.tcl'
cd serie-xfce  && tclsh8.6 run-all.tcl          # nur in einer Xfce-Sitzung
cd serie-tcltk-bauen && tclsh9.0 run-all.tcl
cd serie-tcl9-app    && tclsh9.0 run-all.tcl
```

Ergebnisse im Container am 2026-10-20 (Läufe mit `env -i`):

Serie	8.6.14	9.0.4
serie-linux	10 OK, 2 FAIL (03: kein USER; 05: kein xdg-user-dir)	9 OK, 3 FAIL (zusätzlich 02: Tilde)
serie-xvfb (ohne Fenstermanager)	19 OK, 1 SKIP (10: Iconify)	19 OK, 1 SKIP
serie-tcltk-bauen	11 OK, 1 SKIP (10: Itcl fehlt)	11 OK, 1 FAIL (03: kein -ltommath)
serie-tcl9-app	nicht anwendbar	18 OK
serie-xfce	SKIP (kein Xfce)	nicht gelaufen

Die FAILs sind Unterschiede der Umgebung, keine Fehler der Serien: `env -i` setzt kein `USER`, Ubuntu hat kein `xdg-user-dirs` installiert, Tcl 9 löst die Tilde nicht auf (siehe Pfade, Umgebung, XDG), und der Quellbau linkt libtommath fest hinein (siehe Tk bauen: Abhängigkeiten).

Nächstes Kapitel: Glossar

20 Glossar

20.1 Zeichen

.desktop-Datei
: Starter nach der Desktop Entry Specification, in `~/.local/share/applications/` oder `/usr/share/applications/`. Schlüssel `Exec`, `Icon`, `StartupWMClass`.

//zipfs:/
Wurzel des ZIP-Dateisystems von Tcl 9. Beim Quellbau liegt die Tcl-Bibliothek unter `//zipfs:/lib/tcl`, angehängt an `libtcl9.0.so`. Unter Tcl 9 auch in `file volumes`.

20.2 A

auto_execok
Sucht ein Programm im `PATH`, liefert eine Liste oder leer. Merkt sich jede Antwort in `auto_execs`.

auto_path
Suchliste für `package require`. Debian: `/usr/share/tcltk` und `/usr/lib/tcltk/<multiarch>`. Quellbau unter `/opt/tcl9`: nur Zipfs und `/opt/tcl9/lib`.

20.3 C

CLIPBOARD
Die X11-Auswahl für Kopieren und Einfügen. Gesetzt durch `<<Copy>>` oder `clipboard append`. Verschwindet mit dem Besitzer, wenn kein Verwalter sie übernimmt.

Command
Modifier-Name in `bind`. Unter Aqua die Befehlstaste. Unter X11 in Tk 8.6 gleich Mod1 (Alt), in Tk 9.0.4 gleich Meta.

Container-Prüfung
In diesem Buch: Prüfung in einem Ubuntu-24.04-Container am 2026-10-20 mit Tcl/Tk 8.6.14 (Paket) und 9.0.4 (`/opt/tcl9`), Xvfb ohne Fenstermanager.

20.4 D

DISPLAY
Umgebungsvariable mit der Adresse des X-Servers, etwa `:0` oder `:99`. Ohne sie startet Tk unter X11 nicht.

DBus
Nachrichtenbus der Sitzung. `xfconf-query`, `notify-send` und libnotify brauchen ihn. Adresse in `DBUS_SESSION_BUS_ADDRESS`.

20.5 E

encoding system
Vorgabe-Kodierung von Tcl. Unter Linux aus `LC_ALL`, `LC_CTYPE`, `LANG`. `C` und keine Locale: iso8859-1.

EWMH
Extended Window Manager Hints, die `_NET_WM_*`-Eigenschaften. Darüber setzt Tk Vollbild, `-zoomed`, `-type`, Deckkraft und Fenstersymbol.

exo-open
Xfce-Werkzeug, das bevorzugte Programme nach Rolle startet (`--launch FileManager`). `xdg-open` reicht unter Xfce dorthin weiter.

20.6 F

Fenstermanager

Programm, das unter X11 Rahmen, Titelleiste, Fokus und Iconify beisteuert. Unter Xfce xfwm4. Xvfb hat keinen.

fontconfig
Bibliothek, die Schriftnamen auf Dateien abbildet. Tk fragt nach `sans-serif` und `monospace`. `fc-match` zeigt das Ergebnis.

forky
Codename von Debian 14, zurzeit Testing. Messsystem der Serien.

20.7 L

libnotify
Bibliothek für Desktop-Meldungen. Tk 9 lädt `libnotify.so.4` zur Laufzeit für `tk sysnotify`.

Locale
Sprach- und Zeichensatzeinstellung aus `LC_ALL`, `LC_CTYPE`, `LANG`. Bestimmt unter Linux `encoding system`.

20.8 M

Mod1, Mod4
X11-Modifier-Bits. In der Vorgabebelegung Mod1 = Alt, Mod4 = Super.

MouseWheel
Rad-Ereignis in Tk. Unter X11 erst ab Tk 9; Tk 8.6 meldet das Rad als `<Button-4>` und `<Button-5>`.

Multiarch
Debian-Ordnung für Bibliotheken je Architektur, etwa `/usr/lib/x86_64-linux-gnu`. Dort liegt die echte `tclConfig.sh`.

20.9 O

os-release
Datei `/etc/os-release` mit `ID`, `ID_LIKE`, `VERSION_CODENAME`. Quelle für die Distribution.

20.10 P

PRIMARY
Die X11-Auswahl des zuletzt markierten Textes. Eingefügt mit der mittleren Maustaste (`<<PasteSelection>>`).

20.11 R

Reparenting
Ein Fenstermanager hängt das Fenster in einen eigenen Rahmen. Erkennbar daran, dass `wm frame.` und `winfo id.` verschieden sind.

RUNPATH
Suchpfad für Bibliotheken, in einer ELF-Datei eingetragen. Der Quellbau setzt `/opt/tcl9/lib`.

20.12 S

scalingPct
`::tk::scalingPct`, Tk 9: Auflösung in Prozent von 96 dpi, auf 25 gerundet, mindestens 100.

StartupWMClass
Schlüssel in einer `.desktop`-Datei. Muss der Klasse aus `WM_CLASS` entsprechen.

20.13 T

TCLLIBPATH
Umgebungsvariable, deren Einträge vorn an `auto_path` kommen. Tcl-Liste, getrennt durch Leerzeichen.

TCL9_0_TM_PATH
Zusätzliche tm-Pfade für Tcl 9.0, auch von 9.1 gelesen. Getrennt durch Doppelpunkt, eingefügt in umgekehrter Reihenfolge. Für 8.6: `TCL8_6_TM_PATH`.

tk scaling
Pixel je Punkt. Bei 100 dpi 1,389.

20.14 W

Wayland
Anzeigeprotokoll als Nachfolger von X11. Tk 9.0.4 hat kein Wayland-Backend und läuft dort über XWayland.

WM_CLASS
X-Eigenschaft mit Name und Klasse eines Fensters. Tk: Programmname und Programmname mit großem Anfang.

20.15 X

X-Ressourcen
Einstellungsdatenbank des X-Servers (`xrdb`). Tk liest sie beim Start. `*TkTheme` wählt das ttk-Theme.

X11
Fenstersystem von Tk unter Linux. `tk windowingsystem` liefert `x11`.

XDG
Freedesktop-Spezifikationen. Hier vor allem Base Directory (`XDG_CONFIG_HOME` und Verwandte) und `user-dirs`.

xdg-open
Öffnet Datei oder Adresse mit dem Standardprogramm. Paket `xdg-utils`.

xdg-user-dir
Liefert den lokalisierten Pfad eines Benutzerordners (`DOCUMENTS`). Paket `xdg-user-dirs`.

Xfce
Desktop auf dem Arbeitsplatz des Autors, Fassung 4.20.

xfconf
Einstellungsdienst von Xfce, gelesen und geschrieben mit `xfconf-query`.

Xft
X FreeType, die Schriftdarstellung von Tk unter X11. `tk::pkgconfig get fontsystem` liefert `xft`.

Xft.dpi

X-Ressource für die Schrift-DPI. Tk 9 leitet daraus `tk scaling` ab, Tk 8.6 nicht.

Xvfb
X-Server ohne Bildschirm und ohne Fenstermanager. Für Tests. `xvfb-run` startet ihn für einen Befehl.

XWayland
X-Server, den ein Wayland-Compositor für X11-Programme mitbringt.

20.16 Z

Zipkit
Statisch gebauter Interpreter mit angehängter Tcl-Bibliothek, Basis für weitergebbare `zipfs mkimg`-Programme.

Nächstes Kapitel: Fehlermeldungen

21 Fehlermeldungen

Meldungen, die bei den Prüfungen für dieses Buch aufgetreten sind, mit Ursache und Abhilfe. Wo nicht anders angegeben, im Container unter 8.6.14 und 9.0.4 gleich.

21.1 no display name and no $DISPLAY environment variable

```
application-specific initialization failed: no display name and no $DISPLAY
    environment variable
```

DISPLAY fehlt. `wish` führt das Skript danach ohne Tk aus und endet mit 0. Abhilfe: Anzeige setzen oder `xvfb-run`. Im Skript `package require Tk` mit `catch` prüfen. Siehe X11, Fenstermanager und Wayland.

21.2 couldn't connect to display

```
application-specific initialization failed: couldn't connect to display ":77"
```

DISPLAY ist gesetzt, aber kein Server antwortet. Typisch nach einem beendeten Xvfb, dessen Sockel unter `/tmp/.X11-unix/` liegen blieb, oder bei `ssh` ohne `-X`.

21.3 couldn't execute … no such file or directory

```
couldn't execute "gibtsnicht": no such file or directory
```

exec findet das Programm nicht. `errorCode` ist `POSIX ENOENT {no such file or directory}`. Vorher `auto_execok`.

21.4 child process exited abnormally

Das Kind endete mit einem Wert ungleich 0. `errorCode` ist `CHILDSTATUS <pid> <wert>`. Kein Fehler im Sinn des Programms, wenn der Wert eine Antwort ist (etwa `grep` ohne Treffer: 1).

21.5 child killed: software termination signal

Das Kind wurde mit `SIGTERM` beendet. `errorCode`: `CHILDKILLED <pid> SIGTERM {software termination signal}`.

21.6 (Text auf stderr)

exec meldet als Fehlertext, was das Kind auf stderr geschrieben hat, bei Rückgabewert 0. `errorCode` ist `NONE`. Abhilfe: `-ignorestderr` oder `2>@1`. Siehe exec, xdg-open, Prozesse.

21.7 invalid or incomplete multibyte or wide character

```
error reading "file5": invalid or incomplete multibyte or wide character
```

Nur Tcl 9: Ein Kanal mit `-encoding utf-8` hat Bytes gelesen, die kein UTF-8 sind. Tcl 8.6 deutet sie still als Latin-1. Siehe Kodierung und Locale.

21.8 unknown or ambiguous subcommand "home"

```
unknown or ambiguous subcommand "home": must be atime, attributes, …
```

`file home`, `file tildeexpand` und `file tempdir` gibt es erst ab Tcl 9. Unter 8.6 `$env(HOME)` nehmen.

21.9 unknown or ambiguous subcommand "systray"

```
unknown or ambiguous subcommand "systray": must be appname, busy, caret,
    fontchooser, inactive, scaling, useinputmethods, or windowingsystem
```

`tk systray`, `tk sysnotify` und `tk print` gibt es erst ab Tk 9.

21.10 bad button number "8"

Tk 8.6 kennt in `bind` nur die Tasten 1 bis 5. Tk 9 nimmt auch 6 bis 9.

21.11 bad event type or keysym "Super"

Es gibt keinen Modifier `Super`. Die Super-Taste heißt in `bind` `Mod4`.

21.12 CLIPBOARD selection doesn't exist or form "STRING" not defined

```
CLIPBOARD selection doesn't exist or form "STRING" not defined
PRIMARY selection doesn't exist or form "STRING" not defined
```

Niemand besitzt die Auswahl. Bei CLIPBOARD: Es wurde nichts kopiert, oder das kopierende Programm ist beendet. Bei PRIMARY: Es ist nichts markiert. `selection get` und `clipboard get` immer mit `catch`.

21.13 Error: target STRING not available

Meldung von `xclip`, wenn der Besitzer der Auswahl nicht mehr läuft. Im Container nach dem Ende von `wish`.

21.14 Unable to obtain the list of printers

```
Unable to obtain the list of printers. Command "lpstat" not found. Please install
    the CUPS package for your system.
```

Tk 9, `::tk::print::cups getprinters` ohne CUPS. Paket `cups-client` oder `cups` installieren. Nur 9.0.4.

21.15 Failed to execute child process "dbus-launch"

```
Failed to execute child process "dbus-launch" (No such file or directory)
```

`notify-send` ohne Sitzungs-DBus. Rückgabewert 1. In einer Desktop-Sitzung tritt das nicht auf; in Cron, `ssh` oder Container schon. `tk sysnotify` meldet in derselben Lage nichts.

21.16 Verbindung ist gescheitert

`xfconf-query` ohne Sitzungs-DBus (`serie-xfce`, Referenz; deutsche Meldung auf dem Arbeitsplatz des Autors). Gilt für jeden Aufruf.

21.17 SKIP: serie-xfce braucht XFCE

```
SKIP: serie-xfce braucht XFCE (XDG_CURRENT_DESKTOP=)
```

Keine Xfce-Sitzung. Die Serie prüft dann nichts.

21.18 WARNING: unrecognized options

`configure` von Tcl 9 kennt den Schalter nicht mehr; Threads sind immer an (`serie-tcltk-bauen`, Howto 05). Schalter weglassen.

21.19 file format not recognized

Der Linker trifft auf eine leere `.o` aus einem abgebrochenen `make` (`serie-tcltk-bauen`, Howto 11). `find. -name '*.o' -size 0 -delete`, dann `make` erneut.

21.20 XInput2.h fehlt

Fehlender Kopf `X11/extensions/XInput2.h` beim Tk-Quellbau, gemeldet in *Tcl/Tk 9.0 bauen* auf Ubuntu 24.04. Paket `libxi-dev`. Den genauen Wortlaut der Meldung nennt das Baubuch nicht; hier ist er nicht nachgeprüft.

21.21 this extension is compiled for Tcl 8.6

```
% package require sqlite3
this extension is compiled for Tcl 8.6
    while executing
"load /usr/lib/tcltk/sqlite3/libsqlite3.53.4.so"
```

Unter Debians `tclsh9.0` und `tclsh9.1` mit `libsqlite3-tcl` bis 3.53.4-2; diese Fassungen sind nur für Tcl 8.6 gebaut. Seit 3.54.1-1 bringt das Paket eine Bibliothek für Tcl 9 mit, und die Meldung entfällt. Hintergrund und ein Weg für ältere Fassungen in Suchpfade, Abschnitt *sqlite3 unter Debians Tcl 9*.

21.22 Keine Berechtigung beim Installieren nach /usr/bin

```
/bin/bash ../tclconfig/install-sh -c  -m 755 ../compat/sqlite3/spaceanal.tcl
    "/usr/bin/sqlite3_analyzer"
cp: reguläre Datei '/usr/bin/_inst.669779_' kann nicht angelegt werden: Keine
    Berechtigung
```

Ein TEA-Paket, gebaut gegen Debians `tclConfig.sh`, übernimmt den Präfix `/usr`. `make install-binaries` will dann Programme nach `/usr/bin` legen. Bibliothek und Index sind zu diesem Zeitpunkt schon installiert. Abhilfe: `make install-lib-binaries`.

Nächstes Kapitel: Quellen

22 Quellen

22.1 Einführung

Das Buch erklärt im Text, woher eine Aussage stammt. Dieser Anhang nennt die Quellen vollständig. Fremde Texte sind zusammengefasst, nicht übersetzt oder abgeschrieben. Kurze wörtliche Stellen, etwa Fehlermeldungen und Befehle, stehen im Code-Block.

22.2 Eigene Läufe

22.2.1 Howto-Serien des Autors

Die Serien sind Begleitmaterial des Autors; `serie-linux`, `serie-xvfb` und `serie-xfce` liegen unter Serien/. Jede enthält `README.md` mit dem Abschnitt "Gemessen", `TUTORIAL.md`, `howto/00-referenz.md` und je Howto eine Erklärung `NN-*.md` mit Demo `NN-*.tcl`; `run-all.tcl` startet alle Demos.

Serie	Howtos	Stand laut README	verwendet in
serie-linux	12	Linux, Tcl 8.6.18, Debian forky; ohne Datum	tcl_platform, Pfade, Kodierung, exec, Dialoge
serie-xvfb	20	Tcl/Tk 8.6.14 und 9.0.4 unter Xvfb, ohne Fenstermanager und mit openbox; ohne Datum	X11, beide Testkapitel
serie-xfce	12	Xfce 4.20, Debian, X11; ohne Datum	Xfce, Desktop-Integration
serie-tcltk-bauen	12	Debian forky, 2026-09-15, tclsh8.6 und tclsh9.0, je 12/12	Welches Tcl, Tk bauen, Suchpfade
serie-tcl9-app	18	Tcl 9.0.x	Weitergeben

Die Werte für Xfce mit 29 Pixel Titelleiste stammen aus `serie-xvfb` (Howto 04 und 13), geprüft auf dem Arbeitsplatz des Autors.

22.2.2 Container-Prüfung

Am 2026-10-20 in einem Container mit Ubuntu 24.04.5 LTS, Kernel 6.18.44, x86_64, glibc 2.39.

- Tcl/Tk 8.6.14 aus den Ubuntu-Paketen `tcl8.6`, `tk8.6`.
- Tcl/Tk 9.0.4 als Quellbau unter `/opt/tcl9`, Quellen daneben unter `tcl9.0.4` und `tk9.0.4`.
- Xvfb 21.1.12 ohne Fenstermanager, 1280x1024x24, 100 dpi; für die Skalierung zusätzlich 1920x1080 mit `-dpi 192`; für X-Ressourcen mit `-noreset`.
- Werkzeuge: `xdotool`, `xclip`, `xprop`, `xrdb`, `xmodmap`, `readelf`, `objdump`, `notify-send` 0.8.3 (libnotify 0.8.3).
- Läufe mit `env -i` für Suchpfade und Kodierung.
- Skripte: `beispiele/messprobe-linux.tcl`, `rad-und-tasten.tcl`, `auswahl-probe.tcl`, `wm-probe.tcl`, `spaeter-rahmen.tcl`, `utf8-wache.tcl`,

`xdg-ordner.tcl`; dazu alle Serien außer `serie-xfce` (siehe CLI).

22.2.3 Prüfung auf lxpro

Auf dem Arbeitsplatz des Autors, Debian unstable (forky), Oktober 2026, ausgeführt vom Autor:

- Debian-Pakete `tcl8.6` 8.6.18, `tcl9.0` 9.0.4, `tcl9.1` 9.1.0, `tk9.0` 9.0.4, `tk9.1` 9.1.0, `libsqlite3-tcl` 3.53.4-2, ab 2026-11-03 3.54.1-1 aus unstable.
- BAWT-Builds 9.0.1 und 9.0.4 unter `/opt/Tcl`, Quellbau 9.1.0 unter `/opt/tcl91`.
- sqlite3 3.53.4 aus `sqlite3.53.4.tar.gz` der Tcl-9.1.0-Distribution, gebaut gegen Debians `tcl8.6-dev`, `tcl9.0-dev` und `tcl9.1-dev`.
- Berichte mit `examples/demo-tuappinfo.tcl` aus tclutils 0.69.0.
- Messung der offenen Punkte am 2026-11-02 unter Xfce 4.20, X11, `wish` 8.6.18, 9.0.4 und 9.1.0: `os-release`, `wm frame`, `-alpha`, `scalingPct` am Dialog, `<Command-c>`, Super, `xfce4-clipman`, `zenity`, `desktop-file-validate`, `tk systray`, Xfce-Erkennung, `grab -global`, `readelf -d`.

22.2.4 Was im Container fehlte

Für XWayland nachinstalliert: `weston` 13.0.0, `xwayland`, `x11-apps` (Ubuntu-Pakete), Weston verschachtelt im Xvfb.

Im Container nicht vorhanden und deshalb nicht geprüft: Desktop-Sitzung, Sitzungs-DBus, Xfce, Fenstermanager, Wayland, CUPS, `xdg-user-dirs`, `desktop-file-utils`, `zenity`, `kdialog`, Debian selbst.

22.3 Tcl und Tk

- **Tcl/Tk-Quelltext 9.0.4**, gelesen im Container:
  - `tk/generic/tkEvent.c`: Umsetzung der Tasten 4 bis 7 in `MouseWheel`.
  - `tk/generic/tkBind.c`: Modifier-Tabelle (`Command`, `Option`), Tastennummern ab 4 bei `event generate`.
  - `tk/unix/tkUnixKey.c`: Ermittlung der Meta- und Alt-Bits.
  - `tk/unix/tkUnixWm.c`: `WM_CLASS`, `_NET_WM_*`.
  - `tk/unix/tkUnixEvent.c`: Rückfall auf 75 dpi unter XWayland.
  - `tk/unix/tkUnixSysNotify.c`, `tk/unix/tkUnixSysTray.c`, `tk/library/systray.tcl`: `tk sysnotify`, `tk systray`.
  - `tk/library/print.tcl`: `tk print` unter X11.
  - `tk/library/scaling.tcl`: Skalierung unter X11.
  - `tk/library/ttk/fonts.tcl`: Standardschriften.
  - `tk/library/tkfbox.tcl`: Dateidialog, versteckte Dateien.
  - `tcl/library/init.tcl`, `tcl/library/auto.tcl`: `auto_execok`, `auto_reset`.
  - `tcl/library/msgcat/msgcat.tcl`: Locale-Variablen.
  - `tcl/library/tcltest/tcltest.tcl` (2.5.11): Rückgabewert von `runAllTests`.
  - `tcl/unix/Makefile.in`, `tk/unix/Makefile.in`: installierte Programme.
  https://core.tcl-lang.org/tcl/, https://core.tcl-lang.org/tk/
- **Tcl/Tk-Handbuchseiten** 9.0: `bind`, `event`, `selection`, `clipboard`, `wm`, `winfo`, `tk`, `file`, `exec`, `encoding`, `zipfs`, `tm`. https://www.tcl-lang.org/man/tcl9.0/
- **TIP 602**: Tilde nicht mehr im Dateisystem auflösen.
  https://core.tcl-lang.org/tips/doc/trunk/tip/602.md
- **TIP 656**: Kodierungsprofile, `strict` als Vorgabe.

https://core.tcl-lang.org/tips/doc/trunk/tip/656.md
- **Tcl/Tk 9.1.0**, Download-Seite mit Release-Notes und den mitgelieferten Paketen, darunter `sqlite3.53.4.tar.gz`. https://sourceforge.net/projects/tcl/files/Tcl/9.1.0/

22.4 Debian

- **tcltk-defaults**: Standardfassung von Tcl/Tk in Debian.
  https://tracker.debian.org/pkg/tcltk-defaults

22.5 Spezifikationen

- **XDG Base Directory Specification**, freedesktop.org.
  https://specifications.freedesktop.org/basedir-spec/latest/
- **Desktop Entry Specification**, freedesktop.org.
  https://specifications.freedesktop.org/desktop-entry-spec/latest/
- **Extended Window Manager Hints (EWMH)**, freedesktop.org.
  https://specifications.freedesktop.org/wm-spec/latest/
- **xdg-user-dirs**, freedesktop.org. https://www.freedesktop.org/wiki/Software/xdg-user-dirs/
- **os-release**, Handbuchseite `os-release(5)` von systemd.
- Handbuchseiten: `Xvfb(1)`, `xvfb-run(1)`, `xrdb(1)`, `xprop(1)`, `xmodmap(1)`, `xclip(1)`, `xdotool(1)`, `xdg-open(1)`, `notify-send(1)`, `fc-match(1)`, `readelf(1)`, `ld.so(8)`, `locale(7)`.

22.6 Nachbarbücher derselben Reihe

Buch	Verwendet für
Debian unter Linux (book-linux-debian), Kapitel Tcl unter Linux, Debian erkennen, Suite und Codename	Lage der Tcl-Installationen auf lxpro und decoder, Regeln für /opt, os-release
Tcl/Tk unter macOS (book-macos-tcltk)	Aufbau und Stil; Vergleich mit Aqua
Tcl/Tk 9.0 bauen (book-tcltk9.0-bauen), Tcl/Tk 9.1 bauen (book-tcltk9.1-bauen)	Unix-Abhängigkeiten, libxi-dev, mehrere Installationen, statisch bauen
Tcl-Anwendungen ausliefern (book-tcl-ausliefern), im Text Ausliefern	zipfs, Zipkits, .deb und .rpm
C-Erweiterungen (book-tcl-erweiterungen)	TEA gegen tclConfig.sh
Pakete und Module (book-package-module)	auto_path, .tm
Tcl/Tk-Versionen nebeneinander (book-tcltk-versionen)	gleiche Paketversion, Binärerweiterungen je Fassung
tclutils (book-tclutils)	tuappinfo
Tcl/Tk 9.1 (book-tcltk9.1)	tk fileicon, BiDi unter X11
Encoding (book-encoding)	Profile, TIP 656

Buch	Verwendet für
Testen (book-tcl-testen)	tcltest
Printing (book-printing)	Druck über CUPS

22.7 Widersprüche zwischen den Quellen

Wo Ergebnisse voneinander abweichen, steht das im Kapitel. Die wichtigsten:

- *Debian unter Linux*, Kapitel *Tcl unter Linux*: "<Command-c> ... reagiert unter Linux also auf Alt+C." Gilt für Tk 8.6; unter Tk 9.0 und 9.1 keine Taste, im Container und auf dem Xfce-Arbeitsplatz. Siehe Maus, Tastatur, Zwischenablage.
- `serie-linux`, README: `file volumes` liefert `/`. Unter Tcl 9 zusätzlich `//zipfs:/`.
- `serie-tcltk-bauen`, Howto 03: `TCL_LIBS` von 9.0.4 enthält `-ltommath`. Gilt für das Debian-Paket, nicht für den Quellbau.
- `serie-tcltk-bauen`, Howto 04 und 07: Tk 9 linkt zusätzlich `libpng`. Im Container und auf dem Arbeitsplatz erscheint `libpng16` bei allen Tk-Fassungen nur über freetype (`readelf -d`).
- *Tcl/Tk 9.0 bauen*: `libxi-dev` nötig wegen `XInput2.h`. Im Quelltext von Tk 9.0.4 kommt `XInput2` nicht vor.

22.8 Marken

Linux ist eine Marke von Linus Torvalds. Debian ist eine Marke von Software in the Public Interest, Inc. Ubuntu ist eine Marke von Canonical Ltd. Xfce, X.Org und freedesktop.org sind Namen der jeweiligen Projekte. Tcl und Tk stehen unter einer BSD-artigen Lizenz. Dieses Buch steht in keiner Verbindung zu diesen Organisationen.

Ende des Buches.

23 Testumgebung

23.1 Einführung

Dieser Anhang nennt die Rechner und Versionen, mit denen die Beispiele des Buchs ausprobiert sind, die Prüfskripte dazu und die Punkte, die noch offen sind. Die Kapitel nennen davon nur, was für das Verständnis nötig ist. Wie die Werte entstanden sind, steht in Anhang E: Quellen.

23.2 Rechner und Versionen

Zwei Umgebungen, beide x86_64 mit X11:

Was	Debian-Arbeitsplatz des Autors	Container
System	Debian forky (Testing)	Ubuntu 24.04.5 LTS, Kernel 6.18, glibc 2.39
Tcl 8.6	8.6.18 aus dem Paket	8.6.14 aus den Paketen tcl8.6, tk8.6

Was	Debian-Arbeitsplatz des Autors	Container
Tcl/Tk 9	9.0.4 und 9.1.0 aus den Paketen	9.0.4, Quellbau unter /opt/tcl9
Anzeige	Xfce 4.20 unter X11 (Fenstermanager xfwm4)	Xvfb ohne Fenstermanager, 1280x1024x24, 100 dpi
Quelle der Werte	die fünf Howto-Serien	beispiele/und die Serien außer serie-xfce

Im Container laufen die Interpreter für Suchpfade und Kodierung mit leerer Umgebung (`env -i`), damit keine Variable aus der Sitzung die Werte verfälscht. Aussagen über Tk-Interna (Mausrad, `tk print`, `tk systray`, Skalierung) sind in den Tk-9.0.4-Quellen nachgelesen und im Container nachgeprüft. Die Werte mit Fenstermanager stammen aus `serie-xvfb` (openbox) und vom Arbeitsplatz des Autors (Xfce).

Im Container gibt es keine Desktop-Sitzung, keinen Sitzungs-DBus, kein Xfce, keinen Fenstermanager, kein Wayland und kein CUPS. Die Xfce-Werte stammen deshalb nur aus `serie-xfce`.

23.3 Prüfskripte

Skript	prüft	Kapitel
beispiele/messprobe-linux.tcl	alle Werte der Tabellen dieses Buchs	alle, siehe CLI
beispiele/rad-und-tasten.tcl	Mausrad und Modifier mit xdotool	Maus, Tastatur, Zwischenablage
beispiele/auswahl-probe.tcl	PRIMARY und CLIPBOARD, ohne Verklemmung	Maus, Tastatur, Zwischenablage
beispiele/wm-probe.tcl	Fokus, Rahmen, Iconify, Vollbild, WM_CLASS	X11, Fenstermanager; Tests
beispiele/spaeter-rahmen.tcl	Dialoglage bei spätem Rahmen	Tests
beispiele/utf8-wache.tcl	UTF-8-Skript unter LANG=C mit 8.6	Kodierung und Locale
beispiele/xdg-ordner.tcl	XDG-Ordner und Benutzerordner, 8.6 und 9	Pfade, Umgebung, XDG
serie-*/run-all.tcl	alle Howtos einer Serie (drei veröffentlicht, Kapitel 1)	siehe Kapitel 1

23.4 Noch nicht getestet

Kapitel	offen
X11, Fenstermanager und Wayland	eine echte Wayland-Sitzung (verschachteltes Weston geprüft)
Native Dialoge und Desktop-Integration	tk systray im Xfce-Panel am Bild
Was der Test selbst herstellt	die Howtos unter Windows

Ein Ergebnis zu einem dieser Punkte gehört in das jeweilige Kapitel und wird hier gestrichen.

24 Lizenz

```
Copyright (c) 2026 Gregor Ebbing

Text: Creative Commons Namensnennung - Weitergabe unter gleichen
Bedingungen 4.0 International (CC BY-SA 4.0)
Code: MIT-Lizenz (unten)
Übernommenes Material: Lizenz des Originals

---

Geltungsbereich

Die folgenden Lizenzen gelten nur für Inhalte, an denen der Autor die
Rechte hat. Übernommene Inhalte anderer Projekte sind davon
ausgenommen (siehe "Übernommenes Material").

---

Text

Alle vom Autor verfassten Texte dieses Buchs stehen, soweit nicht
anders gekennzeichnet, unter der Creative-Commons-Lizenz
"Attribution-ShareAlike 4.0 International" (CC BY-SA 4.0). Das gilt
insbesondere für die Kapitel, Anhänge, README und weitere
Dokumentation.

Sie dürfen das Material in jedwedem Format oder Medium vervielfältigen
und weiterverbreiten sowie es remixen, verändern und darauf aufbauen,
auch zu kommerziellen Zwecken, unter den folgenden Bedingungen:

* Namensnennung: Sie müssen angemessene Urheber- und Rechteangaben
  machen, einen Link zur Lizenz beifügen und angeben, ob Änderungen
  vorgenommen wurden.

* Weitergabe unter gleichen Bedingungen: Wenn Sie das Material
  remixen, verändern oder anderweitig direkt darauf aufbauen, dürfen
  Sie Ihre Beiträge nur unter derselben Lizenz wie das Original
  verbreiten.

Verbindlich ist der vollständige Lizenztext:
https://creativecommons.org/licenses/by-sa/4.0/legalcode.de
Englische Originalfassung:
```

```
https://creativecommons.org/licenses/by-sa/4.0/legalcode

---

Code

Die vom Autor geschriebenen Programme und Skripte zu diesem Buch
(Beispiele, Prüfskripte, Werkzeuge) stehen unter der MIT-Lizenz.

Codebeispiele innerhalb der Texte stehen zusätzlich unter der
MIT-Lizenz. Sie dürfen ohne die Bedingungen der CC-Lizenz in eigenen
Code übernommen werden.

---

Übernommenes Material

Ausschnitte aus anderen Projekten, etwa Beispiele aus den
Handbuchseiten von Tcl/Tk, tcllib oder Tablelist, sind mit ihrer
Quelle gekennzeichnet und bleiben unter der Lizenz ihres Originals,
wie sie das jeweilige Projekt angibt (bei Tcl/Tk und tcllib die Datei
license.terms). Größere Übernahmen stehen, falls vorhanden, mit Quelle,
Rechteinhaber und Lizenz in THIRD-PARTY.md.

---

MIT License

Copyright (c) 2026 Gregor Ebbing

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

Stichwortverzeichnis

-

-noreset 49

.

.desktop 43

/

/etc/os-release 8
/proc 11
/usr/share/tcltk 19

:

::tk::scalingPct 35

A

auto_execok 27
auto_path 18

C

Command 39
CUPS 45

D

DISPLAY 30

E

encoding system 24
EWMH 32

F

Fenstermanager 30

G

grab 52

I

info loaded 23

L

libsqlite3-tcl 20
libxi-dev 55

M

MouseWheel 38

P

PRIMARY 40

R

reparentierender 31
Ressourcendatenbank 37

S

StartupWMClass 43

T

tcl9.0 11
TCL9_0_TM_PATH 22
tcl_platform 8
TCLLIBPATH 21
threaded 9
tk scaling 35
tk sysnotify 44

W

Wayland 33
winfo viewable 51
WM_CLASS 30

X

X11 7
XDG 7
XDG Base Directory 15
xdg-open 28
Xfce 45
xfconf 46
Xft 34
Xft.dpi 36
xfwm4 47
Xvfb 48
xvfb-run 48

Z

Zipfs 13
zipfs 57
Zipkit 59

~

~/.config/user-dirs.dirs 17